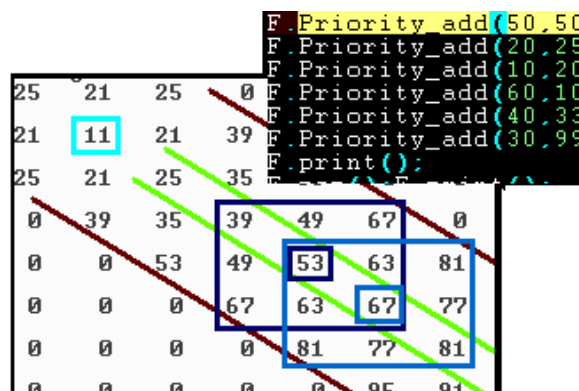




ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ ΡΑΔΙΟΗΛΕΚΤΡΟΛΟΓΙΑΣ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Υλοποίηση Αλγορίθμου Εύρεσης Βέλτιστης Διαδρομής Κινούμενης Μονάδας



Κριτοπούλου Παρασκευή

Επιβλέπων Καθηγητής: Σπυρίδων Νικολαΐδης

Θεσσαλονίκη 2015

ΠΕΡΙΕΧΟΜΕΝΑ

1 PATH FINDING	5
2 ΑΝΑΠΑΡΑΣΤΑΣΗ ΤΟΥ ΧΩΡΟΥ	7
2.1 Τύποι Πλέγματος.....	7
2.1.1 Τετραγωνικό Πλέγμα.....	8
2.1.2 Εξαγωνικό Πλέγμα	8
2.1.3 Τριγωνικό Πλέγμα	8
2.2 Συστατικά Μέρη Κόμβου Πλέγματος	9
2.3 Πολυγωνικό Πλέγμα	9
2.4 Συστήματα Συντεταγμένων στο Πλέγμα	10
2.4.1 Τετραγωνικό Πλέγμα.....	10
2.4.2 Εξαγωνικό Πλέγμα	11
2.4.3 Τριγωνικό Πλέγμα.....	11
2.5 Σχέσεις Μεταξύ Μερών Πλέγματος.....	12
2.6 Το Πλέγμα στον Αλγόριθμο	13
2.7 Μετασχηματισμός Συντεταγμένων	15
2.7.1 Τετραγωνικό Πλέγμα	15
2.7.2 Εξαγωνικό Πλέγμα.....	16
2.7.3 Τριγωνικό Πλέγμα.....	16
3 ΕΙΔΗ ΑΛΓΟΡΙΘΜΩΝ PATH FINDING	19
3.1 Αλγόριθμος Dijkstra's	20
3.1.1 Αλγόριθμος Dijkstra's μετ' Εμποδίων.....	20
3.2 Greedy Best First Search Αλγόριθμος.....	21
3.2.1 Greedy Best First Search και Εμπόδια.....	21
3.3 Βελτιωτικός Αλγόριθμος Contraction Hierarchies.....	23
4 A* (STAR) ALGORITHM	25
4.1 Μέθοδος Ανάπτυξης Μονοπατιού.....	25
4.2 Κόστος Κίνησης	26
4.2.1 Ανοιχτή Λίστα.....	27
4.2.2 Μεταβλητός Βαθμός Προσπέλασης	28
4.2.3 Διαμόρφωση Εδάφους.....	29
4.2.4 Διαγώνιος Βηματισμός.....	29
5 ΕΥΡΕΤΙΚΗ (HEURISTICS).....	31
5.1 Ευρετική σε Ιδανικές Συνθήκες	31
5.1.1 Υπολογισμός Επακριβούς Ευρετικής	31
5.2 Ευρετική σε Χάρτη Πλέγματος.....	32
5.2.1 Απόσταση Manhattan.....	32
5.2.2 Απόσταση Diagonal	33
5.2.3 Ευκλείδεια Απόσταση	34
5.2.4 Ευρετική & Πολλαπλοί Στόχοι	35
5.3 Πολλαπλά και Ισότιμα Η.....	35

6	ΕΥΡΕΤΙΚΗ ΑΝΑΖΗΤΗΣΗ ΚΑΙ ΑΠΟΔΟΣΗ ΑΛΓΟΡΙΘΜΟΥ	41
6.1	A* Algorithm	42
7	ΑΝΟΙΧΤΗ ΚΑΙ ΚΛΕΙΣΤΗ ΛΙΣΤΑ	45
7.1	Δομές Υλοποίησης Λίστας	46
7.1.1	Unsorted Arrays & Linked Lists	46
7.1.2	Sorted Arrays	46
7.1.3	Sorted Linked Lists	46
7.1.4	Binary Heap	46
7.1.5	Sorted Skip Lists	46
7.1.6	Indexed Arrays	47
7.1.7	Hash Tables	47
7.1.8	Splay Trees	48
7.1.9	HOT Queues	48
7.1.10	Pairing Heaps	49
7.2	Σύγκριση Δομών Δεδομένων	49
8	ΠΑΡΑΛΛΑΓΕΣ ΤΟΥ A*	51
8.1	Dynamic Weighting	51
8.2	Bandwidth Search	51
8.3	Bidirectional Search	51
8.4	Dynamic A* and Lifelong Planning A*	52
8.5	Jump Point Search	52
8.6	Theta*	52
9	ΚΙΝΗΣΗ	55
9.1	Κίνηση Μεταξύ Επιφανειών	55
9.1.1	Κίνηση σε Ακμές	56
9.1.2	Κίνηση σε Κορυφές	56
9.1.3	Πολυγωνικοί Χάρτες	56
9.2	Σημεία Πλοήγησης	57
9.3	Skip Link	58
9.4	Εξομάλυνση Μονοπατιού	58
10	ΚΙΝΟΥΜΕΝΑ ΕΜΠΟΔΙΑ	59
10.1	Επανεκτίμηση Διαδρομής	59
10.2	Συναρμογή Διαδρομών	59
10.3	Επαγρύπνηση για Αλλαγές στο Χάρτη	61
10.4	Πρόβλεψη Κίνησης Εμποδίων	61
11	ΧΩΡΟΣ ΣΤΗ ΜΝΗΜΗ	63
11.1	Θέσεις & Κατευθύνσεις	63
11.2	Συμπίεση Διαδρομής	63
11.2.1	Συμπίεση Θέσης	64
11.2.2	Συμπίεση Κατεύθυνσης	64
11.3	Περιορισμός Μήκους Διαδρομής	64
12	ΣΥΝΟΨΗ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ	65

13 ΥΛΟΠΟΙΗΣΗ	67
13.1 Path Finding	67
13.1.1 Ψευδοκώδικας	67
13.1.2 Διάγραμμα ροής	68
13.2 Προγραμματισμός	69
13.2.1 C++	70
13.2.2 VHDL	74
13.3 Ρομποτική	77
13.3.1 Αντιστοίχιση Διαδρομής με Ρομποτική Κίνηση	78
13.3.2 Αλληλεπίδραση με τον Κύριο Βρόχο του Αλγορίθμου	79
13.3.3 Ομαδική Κίνηση	79
ΠΑΡΑΡΤΗΜΑ Ι – ΠΡΟΓΡΑΜΜΑ C++	81
ΠΑΡΑΡΤΗΜΑ ΙΙ – ΠΡΟΣΟΜΟΙΩΣΗ	95
ΠΑΡΑΡΤΗΜΑ ΙΙΙ – VHDL	107
ΒΙΒΛΙΟΓΡΑΦΙΑ	115
ΔΙΚΤΥΑΚΟΙ ΤΟΠΟΙ	115

ΕΙΣΑΓΩΓΗ

Η συγκεκριμένη εργασία πραγματεύεται το σχεδιασμό ενός αλγορίθμου ικανού να υπολογίζει το βέλτιστο μονοπάτι, το οποίο εκτείνεται από μια αρχική θέση, την αφετηρία, έως κάποιο στόχο, αποφεύγοντας όποια εμπόδια μπορεί να υπάρχουν στο ενδιάμεσο διάστημα.

Κατά την ανάπτυξη του αλγορίθμου αυτού, απαντώνται δύο προβλήματα προς αντιμετώπιση:

- Το πρόβλημα της **κίνησης** (Movement), το οποίο αφορά τη διαδικασία της κίνησης πάνω σε ένα επιλεγμένο μονοπάτι, και
- Το πρόβλημα της **εύρεσης του συντομότερου μονοπατιού** (Pathfinding) αφορά την έξυπνη και εργονομική αποφυγή εμποδίων.

Το **Path Finding** επιτρέπει τον μακροπρόθεσμο σχεδιασμό, χωρίς να εντοπίζει και να αντιμετωπίζει τα όποια προβλήματα τη τελευταία στιγμή. Σε γενικές γραμμές η εκτέλεσή του είναι χρονοβόρα, μα αποφέρει το βέλτιστο αποτέλεσμα. Ο Movement είναι ταχύτερος, αλλά μακροπρόθεσμα μπορεί να οδηγήσει σε αδιέξοδο. Όμως σε μεταβλητό φυσικό χώρο, ο μακροπρόθεσμος σχεδιασμός είναι μάταιος.

Το ιδανικό είναι κατά τον εντοπισμό διαδρομών σε μία περιοχή είναι να λαμβάνονται υπόψη και τα δύο αυτά προβλήματα. Path Finding για τη συνολική εικόνα, για αργά μεταβαλλόμενα εμπόδια και μακρά μονοπάτια. Movement για τοπική έρευνα, ταχείες αλλαγές και μικρά μονοπάτια.

Πρέπει να διατηρείται ισορροπία ανάμεσα στα δύο αυτά άκρα, του σχεδιασμού με Path Finding και της αντίδρασης με Movement. Αφενός, ένας αξιόλογος **αλγόριθμος αναζήτησης** συνδυασμένος με έναν ασήμαντο αλγόριθμο κίνησης προσδιορίζει το μονοπάτι στην αρχή της κίνησης με το όχημα να ακολουθεί το μονοπάτι χωρίς να παρεκκλίνει. Αφετέρου, ένα σύστημα που βασίζεται μόνο σε έναν αλγόριθμο κίνησης δεν προϋπολογίζει το μονοπάτι που θα ακολουθήσει, αλλά αποφασίζει σε κάθε του βήμα την επόμενη κίνηση, λαμβάνοντας υπόψη μόνο το άμεσο περιβάλλον. Συνδυασμός και των δύο αλγορίθμων, Movement και Path Finding, επιτυγχάνει τα καλύτερα αποτελέσματα.

1. PATH FINDING

Το **Path Finding** ή **Pathing** είναι ο υπολογισμός της συντομότερης διαδρομής μεταξύ δύο σημείων. Στην ουσία αυτή η μέθοδος ερευνά ένα γράφημα έχοντας ως αφετηρία μία ακμή, τη **μεθόριο**, που αναδιπλώνεται και επεκτείνεται εξερευνώντας τις γειτονικές περιοχές έως ότου προσεγγίσει τον προορισμό. Κύριο κριτήριο βηματισμού αποτελεί η εύρεση της βέλτιστης διαδρομής.

Τα κυριότερα προβλήματα του Path finding είναι :

- 1^ο η εύρεση προσπελάσιμου μονοπατιού ανάμεσα σε δύο περιοχές του γραφήματος
- 2^ο το πρόβλημα του συντομότερου μονοπατιού - δηλαδή η εύρεση της βέλτιστης διαδρομής.

Ο χαρακτηρισμός βέλτιστη διαδρομή περιγράφει το μονοπάτι το οποίο δεν αντιστοιχεί απλά στη συντομότερη απόσταση μεταξύ αφετηρίας και στόχου, αλλά και στον μικρότερο δυνατό χρόνο που μπορεί να δαπανηθεί για τη μετακίνηση από την αφετηρία στο στόχο. Ο χρόνος αυτός εξαρτάται άμεσα από τη διαμόρφωση του εδάφους.

Οι βασικότεροι αλγόριθμοι που έχουν αναπτυχθεί, αντιμετωπίζουν το πρώτο πρόβλημα εξαντλώντας όλες τις πιθανές διαδρομές. Ωστόσο, η εύρεση της βέλτιστης διαδρομής αποτελεί το πιο σύνθετο πρόβλημα προς αντιμετώπιση και για τον προσδιορισμό της δεν είναι απαραίτητο να μελετηθούν όλες οι δυνατές διαδρομές προκειμένου να βρεθεί η βέλτιστη.

Αλγόριθμοι όπως ο A^* και ο *Dijkstra's*, αποκλείουν μεθοδικά τις μη συμφέρουσες διαδρομές, είτε μέσω **ευρετισμού** (heuristics) είτε μέσω δυναμικού προγραμματισμού. Οι αλγόριθμοι αυτοί κατατάσσονται ανάμεσα στους καλύτερους γενικούς αλγόριθμους που λειτουργούν σε γράφημα δίχως προ-επεξεργασία. Ωστόσο, σε πρακτικά συστήματα χάραξης πορείας, μπορούν να επιτευχθούν ιδανικότερες αποδόσεις στο χρόνο επεξεργασίας της διαδρομής από αλγόριθμους οι οποίοι μπορούν να προ-επεξεργαστούν το γράφημα.

Οι περισσότεροι αλγόριθμοι Path Finding που εφαρμόζονται στην τεχνητή νοημοσύνη (AI) ή στους Αλγορίθμους Έρευνας, έχουν σχεδιαστεί για τυχαία γραφήματα και όχι για πλέγματα. Ένα Path Finding θα πρέπει αρχικά να αποκτήσει κάποιες ιδιότητες κοινής λογικής οι οποίες θα βελτιώσουν το χρόνο εκτέλεσής του, για παράδειγμα:

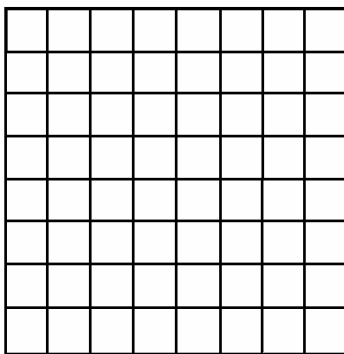
- **Σχετικά με το χώρο:** όσο απομακρύνονται δύο αντικείμενα, τόσο περισσότερο χρόνο θα διαρκέσει η κίνηση από το ένα στο άλλο.
- **Σχετικά με τις κατευθύνσεις:** εάν ο προορισμός βρίσκεται νότια, η βέλτιστη διαδρομή είναι πιο πιθανό να βρεθεί προς το Νότο από ότι προς το Βορρά.
- **Σχετικά με τη συμμετρία:** κίνηση προς το Βορρά, και στη συνέχεια τη Δύση, είναι ίδια με τη κίνηση δυτικά και στη συνέχεια βόρεια.

Προκειμένου να γίνει εφικτός ο υπολογισμός της διαδρομής, πρέπει πρώτα να αναπαρασταθεί ο χώρος μέσα στον οποίο θα κινηθεί η μονάδα. Για την αναπαράσταση του χώρου χρησιμοποιούνται τα πλέγματα. Οι αλγόριθμοι Path Finding εργάζονται σε διαγράμματα, δηλαδή σύνολα από κορυφές που τις συνδέουν ακμές.

2. ΑΝΑΠΑΡΑΣΤΑΣΗ ΤΟΥ ΧΩΡΟΥ (ΧΑΡΤΗΣ)

Σε έναν αλγόριθμο εύρεσης διαδρομής, ο τρόπος της απεικόνισης του χώρου επιδρά καθοριστικά στην αποδοτικότητα και την ποιότητα του τελικού μονοπατιού. Οι αλγόριθμοι αυτοί αναπαριστούν τον χώρο ως ένα πλέγμα κόμβων και προσδιορίζουν το μονοπάτι που πρόκειται να ακολουθηθεί μέσα σε αυτό.

Το πλέγμα δομείται από την επανάληψη απλών σχημάτων. Με αυτό τον τρόπο, η περιοχή διερεύνησης ανάγεται σε έναν απλό διδιάστατο πίνακα. Κάθε στοιχείο του πίνακα αποτελεί έναν κόμβο του πλέγματος και αντιστοιχεί σε μια θέση στο φυσικό χώρο. Η κατάσταση κάθε κόμβου μπορεί να χαρακτηριστεί ως προσπελάσιμη ή μη-προσπελάσιμη, ανάλογα με το αν υπάρχει εμπόδιο σε αυτή τη θέση ή όχι. Από άποψη μετακίνησης, ο κόμβος αποτελεί τη βασική μονάδα κίνησης, το βήμα.



Αναπαράσταση χώρου ως πλέγμα τετραγωνικών κόμβων

Το μονοπάτι αποκαλύπτεται βρίσκοντας από ποιούς κόμβους πρέπει να διέλθει η κινούμενη μονάδα προκειμένου να οδηγηθεί από την αφετηρία στο στόχο. Μόλις το μονοπάτι προσδιοριστεί, η κινούμενη μονάδα μπορεί να κινηθεί από τον ένα κόμβο στον άλλο ακολουθώντας τη διαδρομή, έως ότου καταλήξει στον προορισμό της.

Η σχέση μεταξύ της απόστασης που διανύεται και του χρόνου εύρεσης της διαδρομής στους αλγορίθμους Path Finding, συνήθως δεν είναι γραμμική. Διπλασιασμός της απόστασης συνεπάγεται λίγο περισσότερο από διπλασιασμό του χρόνου εύρεσης του μονοπατιού. Επομένως, όσο λιγότεροι οι κόμβοι που συνιστούν μία περιοχή, τόσο γρηγορότερα θα ολοκληρώνεται η αναζήτηση. Όμως ταυτόχρονα, όσο μεγαλύτερη η ακρίβεια ταύτισης των κόμβων με τις θέσεις στις οποίες θα κινηθεί η μονάδα, τόσο υψηλότερη η ποιότητα του μονοπατιού.

2.1 ΤΥΠΟΙ ΠΛΕΓΜΑΤΟΣ

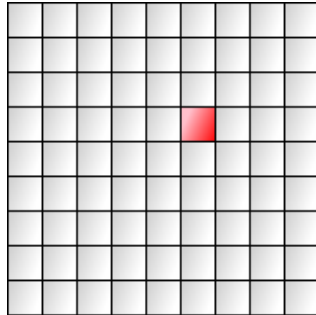
Η συνηθέστερη μέθοδος απεικόνισης λοιπόν, ενός χάρτη, μίας επιφάνειας, ενός πίνακα και γενικά αφηρημένων χώρων, είναι το πλέγμα.

Οι κόμβοι του πλέγματος μπορούν να αναπαρασταθούν με τη μορφή τετράγωνων, εξάγωνων, τριγώνων ή ακόμη και ασύμμετρων πολυγώνων. Η κίνηση στον κόμβο μπορεί

να αφορά κορυφές, ακμές και επιφάνειες. Τα συστήματα συντεταγμένων βοηθούν στον υπολογισμό αυτής της κίνησης.

2.1.1. ΤΕΤΡΑΓΩΝΙΚΟ ΠΛΕΓΜΑ

Το περισσότερο χρησιμοποιούμενο είδος πλέγματος είναι το τετραγωνικό πλέγμα και απαντάται στη πλειοψηφία των εφαρμογών, κυρίως λόγω της ευχρηστίας του.



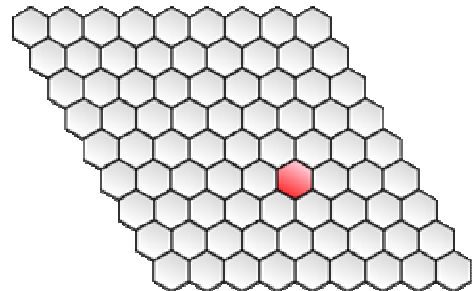
Τετραγωνικό πλέγμα

Οι άξονες είναι κάθετοι μεταξύ τους, επομένως κάθε τοποθεσία μπορεί να προσδιοριστεί από τις γνωστές καρτεσιανές συντεταγμένες (x, y) . Το σύστημα των ορθογώνιων συντεταγμένων λειτουργεί με τον ίδιο τρόπο, ακόμη και αν οι άξονες βρίσκονται υπό γωνία σε μια ισομετρική ή αξονομετρική προβολή.

2.1.2. ΕΞΑΓΩΝΙΚΟ ΠΛΕΓΜΑ

Η επιλογή του εξαγωνικού πλέγματος σχετίζεται με τη παραγωγή μικρότερων στρεβλώσεων στις διανυόμενες αποστάσεις συγκριτικά με το τετραγωνικό πλέγμα.

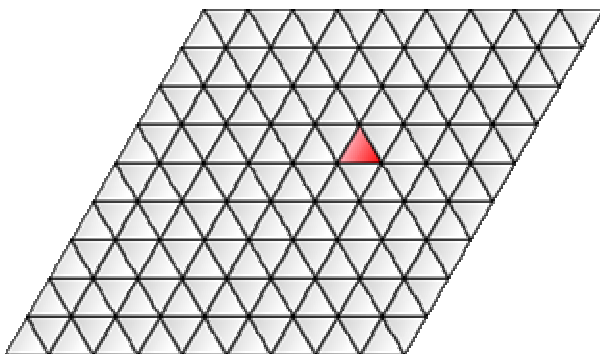
Αυτό οφείλεται στο ότι η απόσταση μεταξύ όλων των γειτονικών κόμβων σε ένα εξαγωνικό πλέγμα είναι η ίδια, ενώ στην περίπτωση του τετραγωνικού πλέγματος η απόσταση διαφέρει για κίνηση σε οριζόντιο και κάθετο επίπεδο, με τη κίνηση ανάμεσα σε κόμβους που γειτνιάζουν διαγωνίως.



Εξαγωνικό πλέγμα

2.1.3. ΤΡΙΓΩΝΙΚΟ ΠΛΕΓΜΑ

Αυτό το είδος πλέγματος χρησιμοποιείται κυρίως στην απεικόνιση του τριδιάστατου χώρου. Μεγάλο μειονέκτημα του τριγωνικού πλέγματος είναι η μεγάλη περίμετρος και η μικρή επιφάνεια έκαστου κόμβου, τα αντίθετα χαρακτηριστικά με λίγα λόγια από αυτά που διαθέτει ο εξάγωνος κόμβος.



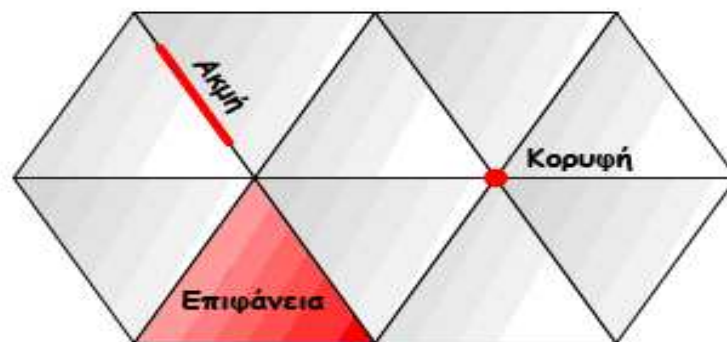
Τριγωνικό πλέγμα

Μικρή επιφάνεια, πρακτικά σημαίνει ότι είναι δυσκολότερο να τοποθετηθεί η κινούμενη μονάδα ολόκληρη μέσα σε έναν κόμβο. Στον τριδιάστατο χώρο, το τρίγωνο είναι το μόνο σχήμα που παραμένει επίπεδο, καθώς τα τετράγωνα και τα εξάγωνα στρεβλώνονται.

Εφόσον το σύστημα συντεταγμένων στο τετραγωνικό πλέγμα είναι απλό, μπορεί να υποτεθεί ότι το εξαγωνικό και το τριγωνικό πλέγμα, προέρχονται από το τετραγωνικό αφού εφαρμοστούν πάνω του κάποιοι μετασχηματισμοί.

2.2 ΣΥΣΤΑΤΙΚΑ ΜΕΡΗ ΚΟΜΒΟΥ ΠΛΕΓΜΑΤΟΣ

Κάθε κόμβος του πλέγματος χαρακτηρίζεται από μία επιφάνεια, καθώς και από ακμές και κορυφές οι οποίες οριοθετούν την επιφάνεια αυτή. Η επιφάνεια είναι διδιάστατη και περικλείεται από τις ακμές. Κάθε ακμή είναι μία ευθεία η οποία ενώνει δύο κορυφές. Η ακμή θεωρείται μονοδιάστατη, ενώ η κορυφή θεωρείται σημείο μηδενικής διάστασης.



Συστατικά μέρη ενός κόμβου του πλέγματος

Αν και στο Path Finding μπορούν να χρησιμοποιηθούν όλα τα μέρη του κόμβου για τον προσδιορισμό της κίνησης, **συνήθως χρησιμοποιούνται οι επιφάνειες**.

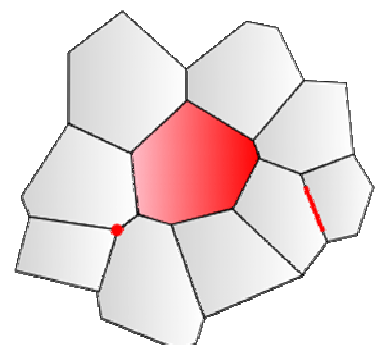
Πιο συγκεκριμένα, για τον προσδιορισμό κτιρίων, τον καθορισμό της μορφολογίας του εδάφους και την ιδιοκτησία της γης χρησιμοποιούνται οι επιφάνειες. Για την οριοθέτηση συνόρων χρησιμοποιούνται οι ακμές. Οι κορυφές καθορίζουν το υψόμετρο. Οδοί και σιδηροδρομικές γραμμές μπορούν να χρησιμοποιήσουν είτε επιφάνειες είτε ακμές ως μέσα αναπαράστασης.

Στον σχεδιασμό συστημάτων συντεταγμένων, η αναλογία των μερών που διαθέτει ο κόμβος, η βασική μονάδα του πλέγματος, είναι καταλυτικός.

2.3 ΠΟΛΥΓΩΝΙΚΟ ΠΛΕΓΜΑ

Το πολυγωνικό πλέγμα αποτελείται από κόμβους των οποίων η μορφή είναι ακανόνιστη και δεν επαναλαμβάνεται. Οπότε και οι αποστάσεις μεταξύ των κόμβων του πολυγωνικού πλέγματος δεν είναι σταθερές.

Στα πολυγωνικά πλέγματα είναι δύσκολη η χρήση συντεταγμένων για τον προσδιορισμό της κίνησης. Σε αυτού του είδους τα πλέγματα οι αλγόριθμοι δεν χρησιμοποιούν



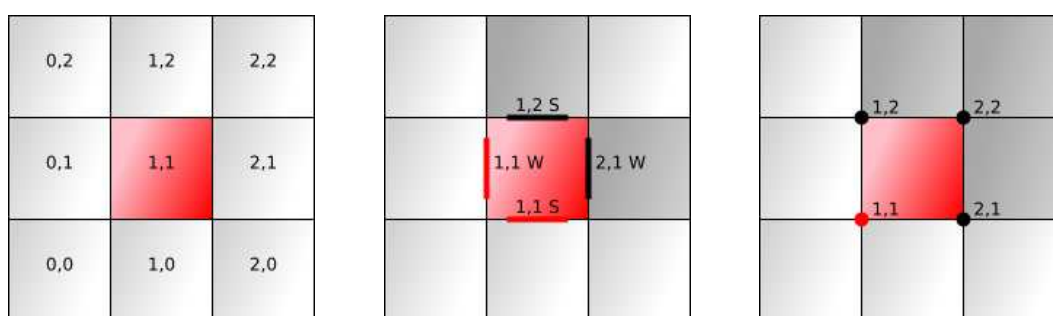
συντεταγμένες, αλλά τις επιφάνειες, ακμές και κορυφές των κόμβων για να ορίσουν την κίνηση.

2.4 ΣΥΣΤΗΜΑΤΑ ΣΥΝΤΕΤΑΓΜΕΝΩΝ ΣΤΟ ΠΛΕΓΜΑ

Όπως προαναφέρθηκε, κάθε πλέγμα διαθέτει τρία μέρη, επομένως είναι αναγκαίο να βρεθεί μία μέθοδος προσδιορισμού τους στο πλαίσιο ενός συστήματος συντεταγμένων. Αρχίζοντας με απλές αριθμητικές συντεταγμένες που αντιστοιχούν στους άξονες του πλέγματος, ο αριθμός κάθε μέρους του πλέγματος [επιφάνεια (E), ακμές(A) και κορυφές(K)], πληροφορεί για το πόσα από τα μέρη του πλέγματος περιγράφονται με τις ίδιες ακριβώς συντεταγμένες.

2.1.1. ΤΕΤΡΑΓΩΝΙΚΟ ΠΛΕΓΜΑ

Στο παρακάτω σχήμα μελετάται αρχικά το σύστημα συντεταγμένων των επιφανειών. Η καταμέτρηση των [E, A, K] είναι [1, 2, 1]. Συνεπώς, μόνο στην περίπτωση των ακμών χρειάζεται περεταίρω προσδιορισμός επειδή περισσότερες από μία ακμές έχουν την ίδια συντεταγμένη, και για το λόγο αυτό χρησιμοποιούνται τα γράμματα S (South) και W (West).



Σύστημα συντεταγμένων τετραγωνικού πλέγματος [E, A, K]

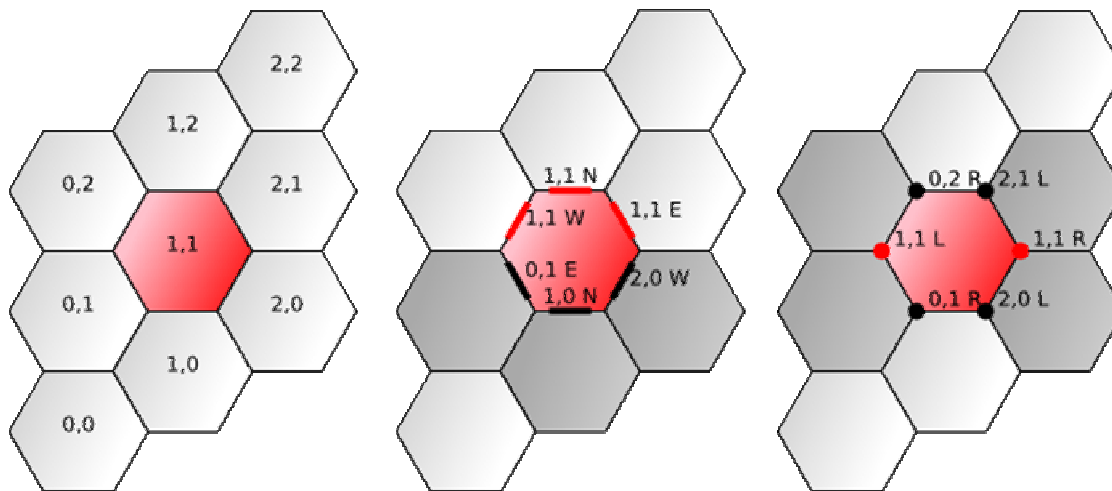
Σε κάθε επιφάνεια ανατίθεται αυθαίρετα μια κορυφή η οποία μοιράζεται τις συντεταγμένες της, στην προκειμένη περίπτωση η νοτιοδυτική γωνία της επιφάνειας.

Σε κάθε επιφάνεια ανατίθενται αυθαίρετα δύο ακμές οι οποίες μοιράζονται τις συντεταγμένες της. Στην προκειμένη περίπτωση το νότιο και δυτικό άκρο (S και W.)

Συγκρίνοντας το διάγραμμα επιφανείας με το διάγραμμα ακμών και κορυφών αντίστοιχα, γίνεται εμφανής ο συσχετισμός των συντεταγμένων. Υπάρχει μία επιφάνεια, δύο ακμές, και μία κορυφή που μοιράζονται τις συντεταγμένες (1, 1). Υπάρχει λοιπόν ταύτιση με την καταμέτρηση [1,2,1] των E, A, K.). Η ανάθεση των ακμών στο παραπάνω σχήμα, υποδηλώνεται από το χρώμα με το οποίο σημειώνονται στο χάρτη καθώς αυτό είναι όμοιο με το χρώμα του κόμβου στον οποίο ανήκουν.

2.1.2. ΕΞΑΓΩΝΙΚΟ ΚΑΙ ΤΡΙΓΩΝΙΚΟ ΠΛΕΓΜΑ

Επειδή το εξαγωνικό πλέγμα προκύπτει από το τετραγωνικό, οι συντεταγμένες της εξαγωνικής επιφάνειας μπορούν να είναι ίδιες με αυτές της τετράγωνης επιφάνειας. Η καταμέτρηση των E, A, K είναι [1, 3, 2].



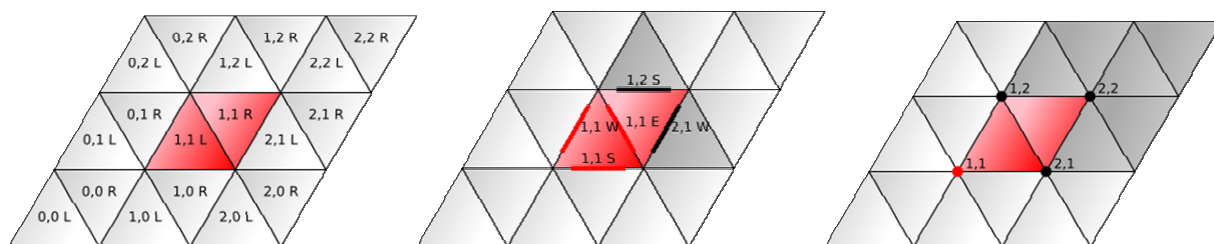
Σύστημα συντεταγμένων εξαγωνικού πλέγματος [E, A, K]

Σε κάθε επιφάνεια ανατίθενται αυθαίρετα τρεις ακμές οι οποίες μοιράζονται τις συντεταγμένες της. Στην προκειμένη περίπτωση N(North), E (East) και W (west). Η ανάθεση των ακμών στο παραπάνω σχήμα, υποδηλώνεται από το χρώμα με το οποίο σημειώνονται στο χάρτη καθώς αυτό είναι όμοιο με το χρώμα του κόμβου του οποίου τη συντεταγμένη υιοθετούν.

Σε κάθε επιφάνεια ανατίθενται αυθαίρετα δύο κορυφές η οποίες μοιράζονται τις συντεταγμένες της, στην προκειμένη περίπτωση ονομάζονται απλά αριστερή και δεξιά πλευρά L και R. Οι ακμές επίσης, σημειώνονται με το αντίστοιχο χρώμα του κόμβου στον οποίο ανατέθηκαν.

2.1.3. ΤΡΙΓΩΝΙΚΟ ΠΛΕΓΜΑ

Επειδή το τριγωνικό πλέγμα προκύπτει από το τετραγωνικό τοποθετώντας τους άξονες υπό γωνία, οι συντεταγμένες της τριγωνικής επιφάνειας μπορούν να είναι ίδιες με αυτές της τετραγωνικής επιφάνειας. Η καταμέτρηση των [E, A, K] είναι [2, 3, 1].



Σύστημα συντεταγμένων τριγωνικού πλέγματος [E, A, K]

Επειδή η τετραγωνική επιφάνεια διαιρείται σε δύο τριγωνικές, ονομάζουμε κάθε μία από τις τριγωνικές επιφάνειες αριστερή και δεξιά L και R.

Οι ακμές είναι ίδιες με αυτές του τετραγωνικού πλέγματος η νότια και η δυτική (S και W), με μία επιπλέον ακμή την ανατολική (E), η οποία είναι αυτή που διαίρεσε τον τετραγωνικό κόμβο σε δύο τριγωνικούς.

Οι κορυφές είναι ίδιες ακριβώς με αυτές στην περίπτωση του τετραγωνικού πλέγματος, και διατηρούν τις ίδιες συντεταγμένες.

Ο χρωματικός κώδικας διαχωρισμού χρησιμοποιείται και στο παραπάνω σχήμα.

2.5 ΣΧΕΣΕΙΣ ΜΕΤΑΞΥ ΜΕΡΩΝ ΠΛΕΓΜΑΤΟΣ

Συνενώνοντας ανά δύο τα μέρη δύο γειτονικών κόμβων του πλέγματος προκύπτουν 9 συνδυασμοί. Συνενώνοντας περισσότερα από δύο μέρη, μπορούν να προκύψουν ακόμη περισσότεροι συνδυασμοί.

Παραλλαγές των συνδυασμών αυτών χρησιμοποιούνται τακτικά στα Path Finder. Για παράδειγμα, οι συντεταγμένες της διαγωνίου κίνησης θα μπορούσαν να υπολογιστούν με τη βοήθεια του συνδυασμού δύο επιφανειών ή από το συνδυασμό κορυφών δύο γειτονικών κόμβων. Από το συνδυασμό δύο ακμών διαπιστώνεται η συνέχεια του πλέγματος η οποία θα μπορούσε να συμπεριλάβει μη συγγραμμικές με την αρχική ακμή ακμές.

Μέρος Β (κόκκινο)			
Μέρος Α (μαύρο)	ΕΠΙΦΑΝΕΙΑ	ΑΚΜΗ	ΚΟΡΥΦΗ
ΕΠΙΦΑΝΕΙΑ			
ΑΚΜΗ			
ΚΟΡΥΦΗ			

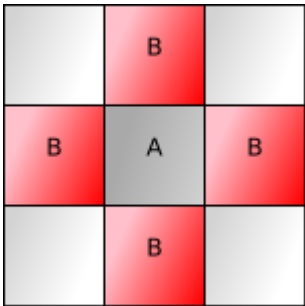
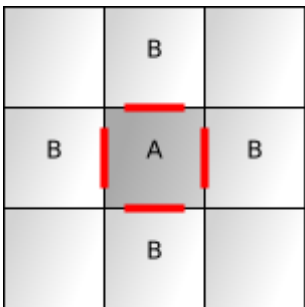
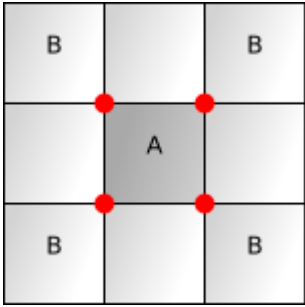
2.6 ΤΟ ΠΛΕΓΜΑ ΣΤΟΝ ΑΛΓΟΡΙΘΜΟ

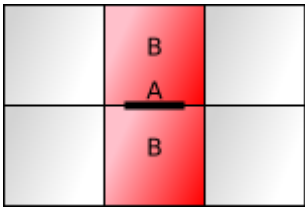
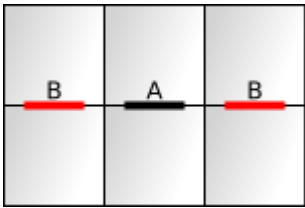
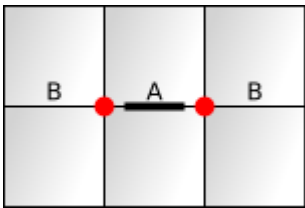
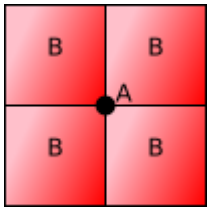
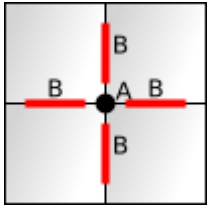
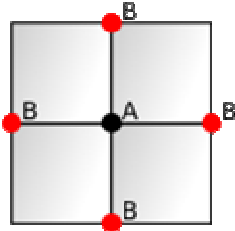
Κάθε ένας από τους παραπάνω συνδυασμούς αναπαριστά μία κίνηση στο πλαίσιο του μονοπατιού και μπορεί να εκφραστεί μέσα στον αλγόριθμο με μορφή συντεταγμένων. Στον επόμενο πίνακα θεωρούμε κίνηση με αφετηρία τον εκάστοτε κόμβο A και φορά προς οποιονδήποτε από τους κόμβους B, για κάθε έναν από τους παρακάτω συνδυασμούς.

Τα B στον παρακάτω πίνακα διαδέχονται το ένα το άλλο στις στήλες πλέγματος, με τη σύνταξη $A \rightarrow B_1 B_2 B_3$. Στις περιπτώσεις τις οποίες υπάρχουν περισσότερες από μία μεταβλητές A υπάρχει επιπλέον δείκτης ο οποίος προσδιορίζει την πλευρά.

Για την υλοποίηση του αλγορίθμου, αρχικά πρέπει να δημιουργηθεί μια δομή που να περιγράφει το επιλεγμένο είδος κόμβου στο πλαίσιο ενός συστήματος συντεταγμένων.

Στη συνέχεια, διαθέτοντας τη βασική μονάδα που συνιστά το πλέγμα, πρέπει να υλοποιηθούν οι συναρτήσεις οι οποίες είναι απαραίτητες για το σχηματισμό του πλέγματος.

ΣΥΝΔΥΑΣΜΟΣ ΜΕΡΩΝ	ΣΥΝΤΕΤΑΓΜΕΝΕΣ ΠΛΕΓΜΑΤΟΣ		
	Τετραγωνικό	Εξαγωνικό	Τριγωνικό
	$(u,v) \rightarrow$ $(u,v+1)$ $(u+1,v)$ $(u,v-1)$, $(u-1,v)$	$(u,v) \rightarrow$ $(u,v+1)$ $(u+1,v)$ $(u+1,v-1)$ $(u,v-1)$ $(u-1,v)$ $(u-1,v+1)$	$(u,v,L) \rightarrow$ (u,v,R) $(u,v-1,R)$ $(u-1,v,R)$ $(u,v,R) \rightarrow$ $(u,v+1,L)$ $(u+1,v,L)$ (u,v,L)
	$(u,v) \rightarrow$ $(u,v+1,S)$ $(u+1,v,W)$ (u,v,S) (u,v,W)	$(u,v) \rightarrow$ (u,v,N) (u,v,E) $(u+1,v-1,W)$ $(u,v-1,N)$ $(u-1,v,E)$ (u,v,W)	$(u,v,L) \rightarrow$ (u,v,E) (u,v,S) (u,v,W) $(u,v,R) \rightarrow$ $(u,v+1,S)$ $(u+1,v,W)$ (u,v,E)
	$(u,v) \rightarrow$ $(u+1,v+1)$ $(u+1,v)$ (u,v) $(u,v+1)$	$(u,v) \rightarrow$ $(u+1,v,L)$ (u,v,R) $(u+1,v-1,L)$ $(u-1,v,R)$ (u,v,L) $(u-1,v+1,R)$	$(u,v,L) \rightarrow$ $(u,v+1)$ $(u+1,v)$ (u,v) $(u,v,R) \rightarrow$ $(u+1,v+1)$ $(u+1,v)$ $(u,v+1)$

ΣΥΝΔΥΑΣΜΟΣ ΜΕΡΩΝ	ΣΥΝΤΕΤΑΓΜΕΝΕΣ ΠΛΕΓΜΑΤΟΣ		
	Τετραγωνικό	Εξαγωνικό	Τριγωνικό
	$(u,v,W) \rightarrow (u,v)$ $(u-1,v)$ $(u,v,S) \rightarrow (u,v)$ $(u,v-1)$	$(u,v,N) \rightarrow (u,v+1) (u,v)$ $(u,v,E) \rightarrow (u+1,v) (u,v)$ $(u,v,W) \rightarrow (u,v) (u-1,v+1)$	$(u,v,S) \rightarrow (u,v,L)$ $(u,v-1,R)$ $(u,v,E) \rightarrow (u,v,R)$ (u,v,L) $(u,v,W) \rightarrow (u,v,L)$ $(u-1,v,R)$
	$(u,v,W) \rightarrow (u,v+1,W)$ $(u,v-1,W)$ $(u,v,S) \rightarrow (u+1,v,S) (u-1,v,S)$	ΑΔΥΝΑΤΟΝ (Οι ακμές δε σχηματίζουν ευθεία στο εξαγωνικό πλέγμα)	$(u,v,W) \rightarrow (u,v+1,W) (u,v-1,W)$ $(u,v,E) \rightarrow (u+1,v-1,E) (u-1,v+1,E)$ $(u,v,S) \rightarrow (u+1,v,S) (u-1,v,S)$
	$(u,v,W) \rightarrow (u,v+1) (u,v)$ $(u,v,S) \rightarrow (u+1,v) (u,v)$	$(u,v,N) \rightarrow (u+1,v,L) (u-1,v+1,R)$ $(u,v,E) \rightarrow (u,v,R) (u+1,v,L)$ $(u,v,W) \rightarrow (u-1,v+1,R) (u,v,L)$	$(u,v,W) \rightarrow (u,v+1) (u,v)$ $(u,v,E) \rightarrow (u+1,v) (u,v+1)$ $(u,v,S) \rightarrow (u+1,v) (u,v)$
	$(u,v) \rightarrow (u,v)$ $(u,v-1) (u-1,v-1) (u-1,v)$	$(u,v,L) \rightarrow (u,v) (u-1,v) (u-1,v+1)$ $(u,v,R) \rightarrow (u+1,v) (u+1,v-1) (u,v)$	$(u,v) \rightarrow (u-1,v,R)$ $(u,v,L) (u,v-1,R)$ $(u,v-1,L) (u-1,v-1,R) (u-1,v,L)$
	$(u,v) \rightarrow (u,v,W)$ $(u,v,S) (u,v-1,W) (u-1,v,S)$	$(u,v,L) \rightarrow (u,v,W) (u-1,v,E)$ $(u-1,v,N)$ $(u,v,R) \rightarrow (u+1,v-1,N)$ $(u+1,v-1,W) (u,v,E)$	$(u,v) \rightarrow (u,v,W)$ $(u,v,S) (u,v-1,E)$ $(u,v-1,W) (u-1,v,S) (u-1,v,E)$
	$(u,v) \rightarrow (u,v+1)$ $(u+1,v) (u,v-1) (u-1,v)$	$(u,v,L) \rightarrow (u-1,v+1,R) (u-1,v,R) (u-2,v+1,R)$ $(u,v,R) \rightarrow (u+2,v-1,L)$ $(u+1,v-1,L) (u+1,v,L)$	$(u,v) \rightarrow (u,v+1)$ $(u+1,v) (u+1,v-1)$ $(u,v-1) (u-1,v) (u-1,v+1)$

2.7 ΜΕΤΑΣΧΗΜΑΤΙΣΜΟΣ ΣΥΝΤΕΤΑΓΜΕΝΩΝ

Για να μπορέσει να κινηθεί μια μονάδα στον φυσικό χώρο, θα πρέπει πρώτα σε συνθήκες προσομοίωσης να αναπαρασταθεί ο χώρος ως πλέγμα και να προσδιοριστεί η διαδρομή που θα ακολουθήσει, και έπειτα να μετατραπούν οι συντεταγμένες του πλέγματος σε συντεταγμένες φυσικού χώρου. Ο μετασχηματισμός αυτός λαμβάνει χώρα σε σημεία, και κάθε σημείο παριστάνει τη θέση της κινούμενης μονάδας.

Από συντεταγμένες πλέγματος σε φυσικού χώρου, μετασχηματίζονται οι κατακόρυφοι και τα κέντρα των επιφανειών των κόμβων στις περισσότερες περιπτώσεις. Από συντεταγμένες φυσικού χώρου προς πλέγματος, εντοπίζεται είτε η επιφάνεια που περιβάλλει το σημείο, η ακμή κοντύτερα στο σημείο, ή η κορυφή κοντύτερα στο σημείο. Ακολουθεί περιγραφή της διαδικασίας μετασχηματισμού κάθε κατηγορίας πλέγματος :

2.7.1. ΤΕΤΡΑΓΩΝΙΚΟ ΠΛΕΓΜΑ

Εάν μία πλευρά του τετράγωνου κόμβου έχει μήκος s και οι ακμές του είναι ευθυγραμμισμένες με τους άξονες x και y , το μόνο που χρειάζεται είναι να πολλαπλασιαστούν με s οι συντεταγμένες κάθε κορυφής για να προκύψουν οι συντεταγμένες του φυσικού χώρου.

Για να υπολογιστεί αντίστροφα, το ποια κορυφή βρίσκεται πλησιέστερα σε κάποιο σημείο του φυσικού χώρου, πρέπει να διαιρεθούν οι συντεταγμένες του φυσικού χώρου με s και να στρογγυλοποιηθεί το πηλίκο σε ακέραιο αριθμό.

0	1	2	3	4	5	6	7	8	i
1	0	1	2	3	4	5	6	7	
2	8	9	10	11	12	13	14	15	
3	16	17	18	19	20	21	22	23	
4	24	25	26	27	28	29	30	31	
5	32	33	34	35	36	37	38	39	
6	40	41	42	43	44	45	46	47	
7	48	49	50	51	52	53	54	55	
j									

Συσχετισμός συντεταγμένων φυσικού χώρου με την αρίθμηση του πλέγματος

Όπως εικονίζεται στο παραπάνω σχήμα, στην περίπτωση των επιφανειών ανατίθεται ένας αύξων ακέραιος αριθμός σε κάθε επιφάνεια, ξεκινώντας από το μηδέν. Ο

υπολογισμός των συντεταγμένων φυσικού χώρου (x, y) συνδέεται με την αρίθμηση της κάθε επιφάνειας του πλέγματος (i, j) διαμέσου της σχέσης:

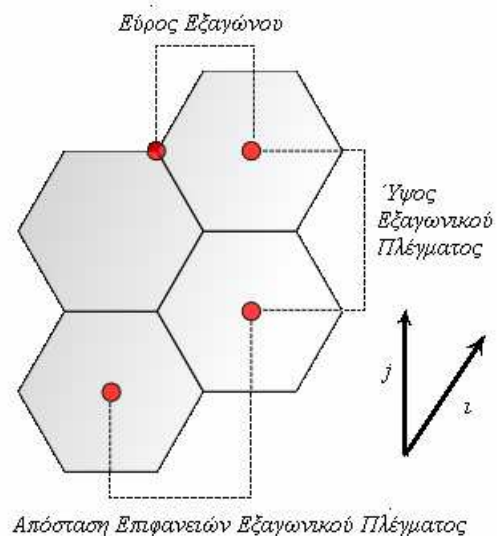
$$X_{\text{KOMBOY}} = i \cdot j - 1$$

$$Y_{\text{KOMBOY}} = i \cdot j - 1$$

όπου $i, j \geq 1$

2.7.2. ΕΞΑΓΩΝΙΚΟ ΠΛΕΓΜΑ

Στην περίπτωση αυτή, ο υπολογισμός είναι ελαφρώς περιπλοκότερος από τον αντίστοιχο στο τετραγωνικό πλέγμα. Ο υπολογισμός του κέντρου της επιφάνειας κάθε κυψελίδας είναι απλός και βασίζεται στον διανυσματικό λογισμό. Στο επόμενο σχήμα εικονίζονται τα διανύσματα i και j . Ο μετασχηματισμός από συντεταγμένες πλέγματος (u, v) σε συντεταγμένες φυσικού χώρου (x, y) γίνεται με τη βοήθεια των εξισώσεων:



$$u = \frac{x}{\text{Απόσταση} \cdot \text{Επιφανειών}} \Rightarrow x = (\text{Απόσταση} \cdot \text{Επιφανειών}) \cdot u$$

$$v = \frac{y}{\text{Υψος}} - u \cdot 0.5 \Rightarrow y = (\text{Υψος}) \cdot (u \cdot 0.5 + v)$$

Όπου: $x = i_x \cdot u + j_x \cdot v$ και $y = i_y \cdot u + j_y \cdot v$

2.7.3. ΤΡΙΓΩΝΙΚΟ ΠΛΕΓΜΑ

Όπως προαναφέρθηκε, οι τριγωνικοί κόμβοι δεν είναι παρά διχοτομημένα τετράγωνα. Ο μετασχηματισμός των συντεταγμένων των κορυφών ενός τριγώνου από συντεταγμένες πλέγματος σε συντεταγμένες φυσικού χώρου, λαμβάνει χώρα με τον πολλαπλασιασμό των συντεταγμένων πλέγματος με τα διανύσματα των αξόνων.

Κατά τον μετασχηματισμό των συντεταγμένων των επιφανειών ενός τριγώνου από συντεταγμένες πλέγματος (i, j) σε συντεταγμένες φυσικού χώρου (x, y), θα πρέπει να δοθεί προσοχή στη διαίρεση των επιφανειών σε L και R. Μέσω της άλγεβρας επιτυγχάνεται ο μετασχηματισμός των συντεταγμένων. Τα κέντρα των επιφανειών L και

R, απέχουν από την κάτω αριστερή κορυφή του υποτιθέμενου αρχικού τετράγωνου κόμβου :

$$\text{Επιφάνεια L: } X_L = 0.25 \cdot i \text{ και } Y_L = 0.75 \cdot j$$

$$\text{Επιφάνεια R: } X_R = 0.75 \cdot i \text{ και } Y_R = 0.75 \cdot j$$

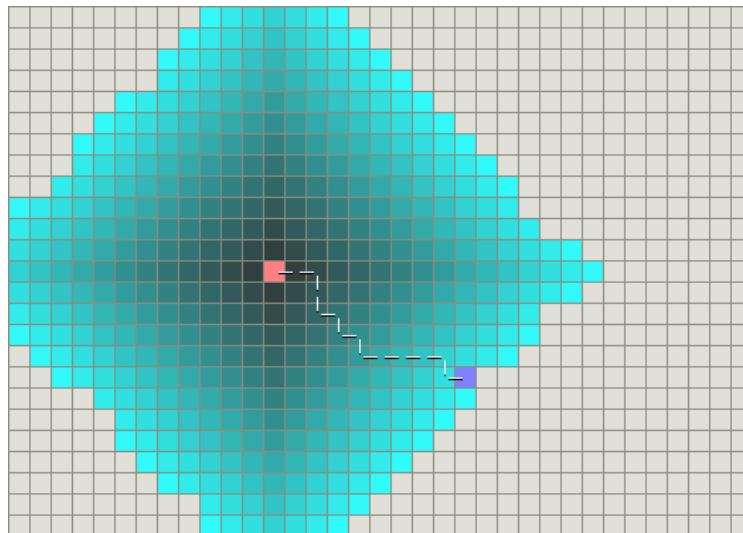
Στο τριγωνικό πλέγμα, πρωταρχικό ρόλο έχουν οι κορυφές και δευτερεύοντα οι επιφάνειες, δηλαδή το αντίθετο από ότι συμβαίνει στο εξαγωνικό πλέγμα. Οπότε γνωρίζοντας τις συντεταγμένες πλέγματος της κάθε κορυφής, προκύπτει εύκολα η θέση της κινούμενης μονάδας στον φυσικό χώρο.

3. ΕΙΔΗ ΑΛΓΟΡΙΘΜΩΝ PATH FINDING

3.1. Αλγόριθμος *Dijkstra's*

Ένα κοινό παράδειγμα αλγόριθμου εύρεσης βέλτιστης διαδρομής που βασίζεται σε γράφημα είναι ο αλγόριθμος *Dijkstra's*. Ο εν λόγω αλγόριθμος έχει ως εκκίνηση έναν κόμβο έναρξης, και ακολουθεί ένα ανοιχτό σύνολο κόμβων προς επιλογή, κοινό χαρακτηριστικό των οποίων είναι ότι απέχουν ένα βήμα από τη θέση στην οποία είναι τοποθετημένη η κινούμενη μονάδα. Σε κάθε βηματισμό μελετάται ο κόμβος που περιέχεται στο ανοιχτό (open) σύνολο και απέχει τη μικρότερη απόσταση από την αφετηρία. Τότε ο κόμβος αυτός χαρακτηρίζεται ως κλειστός (closed), και όλοι οι εφαπτόμενοι σε αυτόν κόμβοι εισέρχονται στο ανοιχτό σύνολο, εάν δεν έχουν ελεγχτεί ήδη. Η διαδικασία αυτή επαναλαμβάνεται έως ότου βρεθεί το μονοπάτι που οδηγεί στον προορισμό. Εφόσον μελετώνται πρώτα οι κόμβοι που απέχουν λιγότερο από την αφετηρία, την πρώτη φορά που προσεγγίζεται ο στόχος επιτυγχάνεται και το συντομότερο δυνατό μονοπάτι.

Ο Αλγόριθμος επεκτείνεται περιμετρικά της αφετηρίας έως ότου να εντοπίσει τον στόχο, όπως φαίνεται στην παρακάτω απεικόνιση. Ο αλγόριθμος του Dijkstra με βεβαιότητα προσδιορίζει τη συντομότερη διαδρομή από την αφετηρία προς τον στόχο, εφ' όσον δεν υπάρχουν βήματα με αρνητικό κόστος.



Περιοχή διερεύνησης Αλγόριθμου Dijkstra's

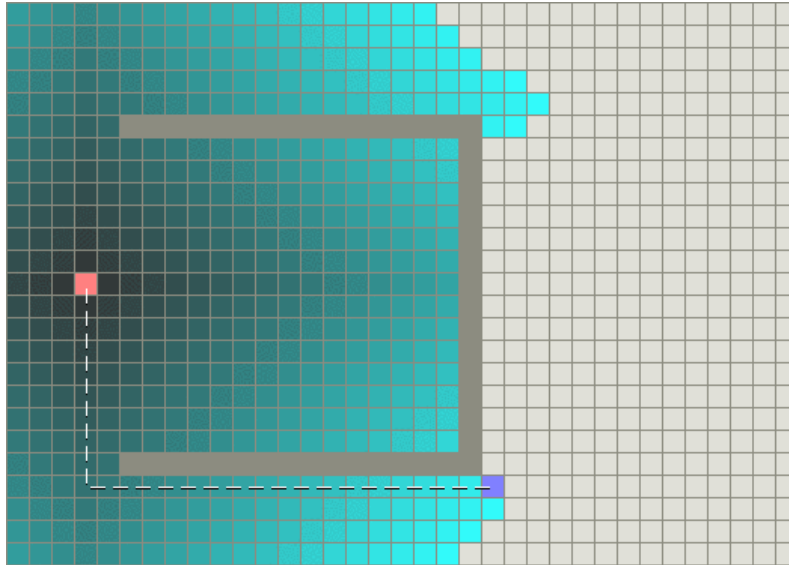
Στο παραπάνω διάγραμμα, ο κόκκινος κόμβος αποτελεί την αφετηρία, ο μωβ κόμβος είναι ο στόχος, και οι κυανές περιοχές υποδηλώνουν τους κόμβους τους οποίους έχει διερευνήσει ο αλγόριθμος. Στον χάρτη δεν επιτρέπεται η διαγώνιος κίνηση. Η προσδιοριζόμενη διαδρομή σημειώνεται ως διακεκομμένη γραμμή.

Ωστόσο η περίπτωση που εικονίζεται δεν είναι ιδανική για τη μελέτη της αποδοτικότητας του αλγορίθμου εφόσον στον χάρτη αναζήτησης δεν υπάρχουν εμπόδια.

3.1.1. Αλγόριθμος *Dijkstra's* μετ' Εμποδίων

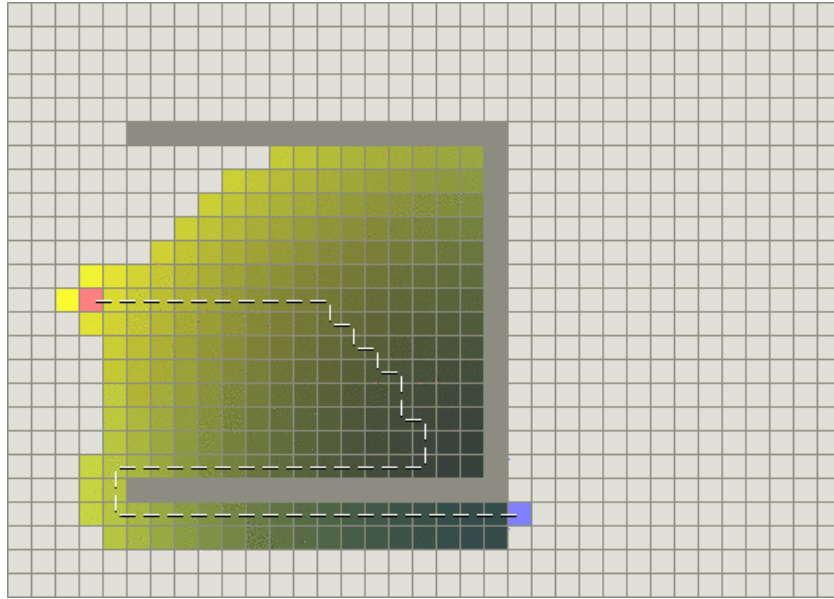
Η αποδοτικότητα ενός αλγορίθμου που αναζητά τη βέλτιστη διαδρομή, μελετάται με τον καλύτερο δυνατό τρόπο τοποθετώντας ένα κοίλο εμπόδιο στο χάρτη.

Στο επόμενο διάγραμμα εικονίζεται με χρήση του ίδιου χρωματικού κώδικα η συμπεριφορά του αλγορίθμου σε χάρτη ο οποίος διαθέτει κοίλο εμπόδιο.



Περιοχή διερεύνησης *ς* Αλγόριθμου *Dijkstra's* μετ' Εμποδίων

Η περιοχή η οποία διερευνάται από τον αλγόριθμο είναι μεγάλη και εκτείνεται σε όλες τις δυνατές κατευθύνσεις. Επομένως καταναλώθηκε χρόνος στη μελέτη περιοχών οι οποίες είναι απομακρυσμένες από το στόχο. Ωστόσο, το μονοπάτι το οποίο βρέθηκε είναι πραγματικά το βέλτιστο.



Περιοχή διερεύνησης Αλγόριθμου Greedy Best First Search και Εμπόδια

Αυτό οφείλεται στην «απληστία» του για την εύρεση του στόχου. Ο στόχος τον έλκει καθώς ο ίδιος λαμβάνει υπόψη του μόνο το κόστος του να καταλήξει σε αυτόν και αγνοεί το κόστος της μακρύτερης πορείας που ακολουθεί.

Ο συνδυασμός και των δύο αυτών προτεραιοτήτων, του κόστους εύρεσης του στόχου και του κόστους του μακρύτερου του μονοπατιού, θα ήταν ιδανικός για ένα αλγόριθμο pathfinder. Αυτό επιτυγχάνεται από τον αλγόριθμο $A^*(star)$.

3.3. Βελτιωτικός Αλγόριθμος *Contraction Hierarchies*

Στα Εφαρμοσμένα Μαθηματικά, η μέθοδος *Contraction Hierarchies* αποτελεί μια τεχνική για την επιτάχυνση του Path Finding, δημιουργώντας προϋπολογισμένες συμπιεσμένες εκδοχές του διαγράμματος που ενώνει σημεία/περιοχές του χώρου. Ο *Contraction Hierarchies* είναι πολύ πιο αποτελεσματικός από τον αλγόριθμο Dijkstra's και χρησιμοποιείται σε πολλές προηγμένες τεχνικές δρομολόγησης.

Σε γενικές γραμμές, οι αλγόριθμοι Path Finding που διερευνούν τον χώρο έχουν δύο διακριτές φάσεις:

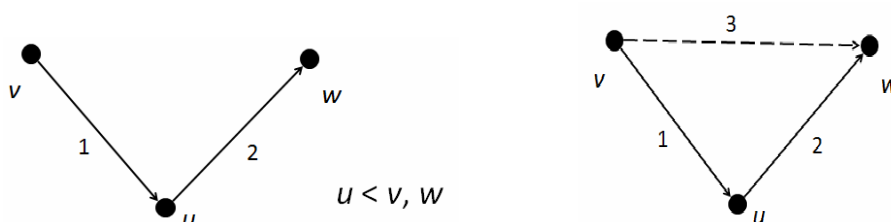
1. προ-επεξεργασία του αρχικού γραφήματος (χρονοβόρα διεργασία) και
2. υποβολή της διαδρομής (στιγμιαία διεργασία).

Ο *Contraction Hierarchies* δημιουργεί μια πολύ-επίπεδη ιεραρχία κόμβων στο στάδιο της προ-επεξεργασίας. Κάθε κόμβος του γραφήματος διαθέτει το δικό του επίπεδο ιεραρχίας. Αυτό επιτυγχάνεται με πολλούς τρόπους. Ένας απλός τρόπος είναι η απαρίθμηση κάθε κόμβου από 1 έως $|N|$. Πιο λεπτομερείς προσεγγίσεις εξετάζουν και λεπτομέρειες του χάρτη όπως, το είδος της οδού κίνησης (αυτοκινητόδρομος, στενό κλπ)

Προ-επεξεργασία: Το επίπεδο, που συνήθως εκφράζεται με ως απαρίθμηση των κόμβων, μπορεί να είναι αυθαίρετο. Κύριο σημείο είναι ότι, **συντομεύσεις** εισάγονται μόνο όταν είναι απαραίτητο και το τότε καθορίζεται μέσα από τον ίδιο τον αλγόριθμο αναζήτησης. Ο αλγόριθμος αναζήτησης περιορίζεται από την ιεραρχία, και καθιστά διαθέσιμες μόνο τις ακμές που συνδέονται με κόμβους που έχουν υψηλότερη ιεραρχία από το τρέχοντα κόμβο προς μία κατεύθυνση, και αντίστροφα. Με αυτόν τον περιορισμό, ο αλγόριθμος δεν θα εντοπίσει ορισμένα βέλτιστα μονοπάτια στο αδιερεύνητο τμήμα του πλέγματος και επομένως θα πρέπει να εισαχθούν νέες ακμές στο διάγραμμα για να αντιπροσωπεύσουν τα υπάρχοντα βέλτιστα μονοπάτια που δεν εντοπίστηκαν. Δεν χρειάζεται όλες οι βέλτιστες διαδρομές να μετατραπούν σε νέες ακμές συντόμευσης. Αρκεί να διερευνηθούν οι γειτονικοί κόμβοι υψηλότερης ιεραρχίας του τρέχοντος κόμβου. Στον αλγόριθμο ακολουθείται η διαδικασία:

- Προσθήκη ιεραρχίας (επίπεδων) στους κόμβους του γραφήματος
- Για κάθε κόμβο, λήψη με βάση την ιεραρχία γειτονικών κόμβων ανώτερης τάξης και εύρεση εάν η βέλτιστη διαδρομή ανάμεσα σε κάθε ζεύγος αυτών διέρχεται από τον τρέχοντα κόμβο. Αν ναι, πρόσθεση ακμής συντόμευσης

Έστω ότι λήφθηκαν υπόψη μόνο δύο γειτονικοί κόμβοι (από τους n του γραφήματος):

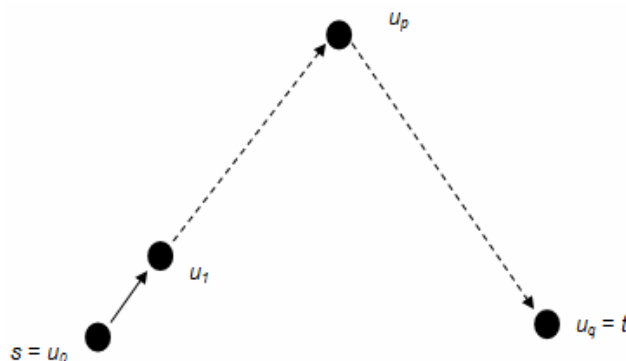


Με βάση το παραπάνω διάγραμμα, αν το βέλτιστο μονοπάτι από τον κόμβο v ως τον w διέρχεται από τον κόμβο u που βρίσκεται χαμηλότερα στην ιεραρχία, μια νέα ακμή πρέπει να προστεθεί το γράφημα ώστε τα συντομότερα μονοπάτια που ο αλγόριθμος αναζήτησης λαμβάνει υπόψη να διατηρηθούν. Η αξία της νέας ακμής ισούται με την απόσταση από v έως w .

Όταν ολοκληρωθεί η προ-επεξεργασία του αρχικού γραφήματος προκύπτει ένα νέο, που αποτελείται από το αρχικό γράφημα στο οποίο έχει προστεθεί ιεραρχία στους κόμβους και συντόμευση στις ακμές.

Διαδρομή: Για το Path Finding, χρησιμοποιείται αμφίδρομος αλγόριθμος του Dijkstra. Ο αλγόριθμος αρχίζει την αναζήτηση από την αφετηρία προς μια κατεύθυνση και καθιστά διαθέσιμες τις ακμές που κατευθύνονται προς υψηλότερης ιεραρχίας κόμβους και από τον στόχο (το πέρας της διαδρομής) προς μία άλλη κατεύθυνση, καθιστώντας διαθέσιμες τις ακμές που κατευθύνονται προς χαμηλότερης ιεραρχίας κόμβους. Οι δύο αναζητήσεις θα συναντηθούν σε κάποιο κόμβο v . Η βέλτιστη διαδρομή από το s στο t αποτελείται από δύο μονοπάτια, του s έως v και του v έως t .

Τα συντομότερα μονοπάτια που εντοπίζονται από αυτόν τον αλγόριθμο έχουν τη συγκεκριμένη μορφή:



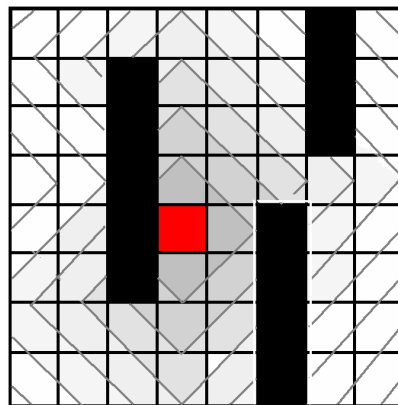
Η διαδρομή που βρέθηκε θα είναι η βέλτιστη, λόγω της προ-επεξεργασίας. Η προ-επεξεργασία μετασχηματίζει το γράφημα εισάγοντας ακμές συντόμευσης, η οποίες συμβολίζουν τα βέλτιστα μονοπάτια τα οποία δεν λήφθηκαν υπόψη από το Path Finding. Ωστόσο, αυτές οι ακμές συντόμευσης χρειάζεται να υποστούν ξανά επεξεργασία για να προκύψουν οι διαδρομές στο αρχικό γράφημα και κατ' επέκταση στον χάρτη.

4. *A*(STAR) Algorithm*

Ο A* (A STAR) είναι ο δημοφιλέστερος αλγόριθμος αναζήτησης διαδρομών και είναι μία παραλλαγή του αλγόριθμου Dijkstra's. Όπως προαναφέρθηκε, αποτελεί μέλος μίας οικογένειας αλγορίθμων αναζήτησης γραφήματος (graph search) οι οποίοι χαρακτηρίζονται από παρόμοια δομή. Οι αλγόριθμοι αυτοί αναπαριστούν τον χάρτη ως ένα πλέγμα κόμβων και προσδιορίζουν το μονοπάτι που πρόκειται να ακολουθηθεί μέσα σε αυτό το πλέγμα. Κάθε κόμβος του πλέγματος αποτελεί μια θέση στο χώρο.

4.1 *Μέθοδος Ανάπτυξης Μονοπατιού*

Κεντρική ιδέα για το είδος αυτό των αλγορίθμων είναι η καταγραφή ενός διευρυνόμενου δακτυλίου ο οποίος ονομάζεται μεθόριος (frontier). Η μεθόριος μπορεί να αναπαρασταθεί από ισοϋψείς γραμμές οι οποίες εκτείνονται σε όλη τη προσπελάσιμη περιοχή.



Αναπαράσταση μεθορίου

Ο σχεδιασμός των ισοϋψών αυτών γραμμών υλοποιείται μέσω της επανάληψης των ακόλουθων βημάτων:

- Επιλογή και αφαίρεση μιας τοποθεσίας (ενός κόμβου) από τη μεθόριο
- Σήμανση της θέσης ως διερευνημένη, ώστε να μην υποστεί επεξεργασία εκ νέου.
- Ανάπτυξη της μεθορίου με έλεγχο των παρακείμενων κόμβων. Όσοι από αυτούς δεν έχουν ελεγχθεί προστίθενται στη μεθόριο.

Η παραπάνω λογική δεν προσδιορίζει τη βέλτιστη διαδρομή, αποτελεί απλά μία οδηγία για τον τρόπο προσέγγισης κάθε σημείου στο χάρτη και αποτελεί την κεντρική λογική ανάπτυξης των αλγορίθμων graph search.

Για να χρησιμοποιηθεί η λογική αυτή για τον προσδιορισμό του μονοπατιού, θα πρέπει να καταγράφει την πορεία που ακολουθήθηκε έως τον τρέχοντα κόμβο (τον κόμβο στον οποίο είναι τοποθετημένη η κινούμενη μονάδα) για κάθε θέση που έχει διερευνηθεί, και να μετονομάσει αυτή η τρέχουσα θέση από διερευνημένη σε προέλευσης. Για κάθε τρέχουσα τοποθεσία η προέλευσή της παραπέμπει στο βήμα που προηγήθηκε για να βρεθεί η μονάδα στην τρέχουσα θέση. Η πληροφορία αυτή αρκεί για να ανακατασκευάσει το σύνολο της διαδρομής.

Η λογική αυτή αποτελεί τον απλούστερο αλγόριθμο Path Finder. Εφαρμόζεται τόσο σε πλέγματα, όσο και σε κάθε είδους γράφημα που αναπαριστά τον χώρο. Γνωρίζοντας τη μέθοδο σχηματισμού της διαδρομής από μία αφετηρία προς όλες τις άλλες θέσεις στο πλέγμα, απομένει ο προσδιορισμός του βέλτιστου μονοπατιού από την αφετηρία προς τον στόχο. Η διερεύνηση των κόμβων που έχει ως αποτέλεσμα την επέκταση της μεθορίου, παύει τη στιγμή που εντοπίζεται ο στόχος.

4.2 **Κόστος Κίνησης**

Γνωρίζοντας το σκεπτικό με το οποίο διευρύνεται η μεθόριος και επομένως αναπτύσσεται ένα μονοπάτι, πρέπει να καθοριστούν οι συνθήκες υπό τις οποίες λαμβάνει χώρα η ανάπτυξη αυτή.

Στην έως τώρα περιγραφή του αλγορίθμου, θεωρήθηκε ότι κάθε βήμα στο χάρτη έχει την ίδια αξία. Το ιδανικό όμως, θα ήταν το Path Finder να λαμβάνει υπόψη την μορφολογία του εδάφους και τον τύπο κίνησης κατά τον υπολογισμό του κόστους κάθε βήματος. Για παράδειγμα η κίνηση σε ασφαλισμένο δρόμο είναι ευκολότερη από την κίνηση σε χέρσα έκταση και η διαγώνιος κίνηση σε ένα πλέγμα προϋποθέτει μεγαλύτερο βηματισμό από ό, τι αξονική κίνηση

Για τον υπολογισμό αυτών των παραμέτρων, ο A* αναθέτει κάποια αξία σε κάθε κόμβο που δεν έχει αποτελέσει **βάση** στην έως τώρα πορεία του οχήματος, δηλαδή στους κόμβους από τους οποίους δεν έχει διέλθει η κινούμενη μονάδα (**ανοιχτοί κόμβοι**). Η αξία αυτή προκύπτει από το άθροισμα της σχετικής απόστασης από το σημείο εκκίνησης G, με την σχετική απόσταση του κόμβου από το στόχο H και περιγράφεται από την εξίσωση:

$$F(n) = G(n) + H(n)$$

δηλαδή

- **G** το κόστος της κίνησης από την αφετηρία A έως τον κόμβο n όπου βρίσκεται η μονάδα κίνησης (**τρέχοντας κόμβος**), ακολουθώντας τη πορεία της μονάδας.
- **H** το εκτιμώμενο κόστος της απόστασης από τον τρέχοντα κόμβο n έως τον στόχο B, που αποτελεί τον προορισμό. Επειδή δεν είναι γνωστή η ακριβής απόσταση μέχρι τον στόχο λόγω των εμποδίων που μπορεί να παρεμβάλλονται στις ανεξερεύνητες από τη μεθόριο περιοχές, το H αποτελεί μία υπόθεση για την εν λόγω απόσταση. Αυτή η υπόθεση βασίζεται σε μία συνάρτηση η οποία ονομάζεται ευρετική (heuristic).

Η νέα μεταβλητή F είναι αναγκαίο να εισαχθεί για τον υπολογισμό της δαπάνης της κίνησης, και καταγράφει το συνολικό κόστος της μετακίνησης από την αφετηρία έως τον στόχο. Προκειμένου όμως να συμπεριληφθεί αυτή η νέα πληροφορία πρακτικά στον υπολογισμό για την επιλογή του επόμενου βήματος, θα πρέπει δημιουργηθεί μία λίστα προτεραιότητας.

Ανοιχτή λίστα ονομάζεται η δομή η οποία περιέχει τους ανοιχτούς κόμβους μαζί με τις τιμές του F που αντιστοιχούν σε κάθε έναν από αυτούς. Το μονοπάτι αναπτύσσεται υπό την συνεχή εξέταση της ανοικτής λίστας ώστε να επιλεγθεί ο κόμβος που χαρακτηρίζεται από το μικρότερο F.

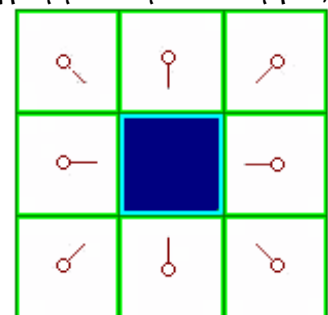
Η δημιουργία αυτής της λίστας όμως, δεν αποτρέπει την επαναλαμβανόμενη διέλευση από ένα κόμβο με διαφορετικό κόστος βηματισμού σε κάθε διέλευση. Για την αποφυγή αυτού του προβλήματος αναζήτησης, κάθε νέος κόμβος θα πρέπει να προστίθεται στο μονοπάτι μόνο εάν η νέα διαδρομή για τον κόμβο είναι καλύτερη από ό, τι η προηγούμενη.

4.2.1. Ανοιχτή Λίστα

Όπως προαναφέρθηκε, η ανοιχτή λίστα (Open List) αποτελεί τον σημαντικότερο παράγοντα για τον καθορισμό της βέλτιστης διαδρομής. Το ζητούμενο μονοπάτι αναπτύσσεται ελέγχοντας την τιμή του F που υπολογίστηκε για κάθε έναν από τους εφαπτόμενους κόμβους στον τρέχοντα κόμβο. Η διαδικασία της αναζήτησης περιλαμβάνει τα εξής στάδια:

1. Η αναζήτηση αρχίζει από την αφετηρία A. Εφόσον η κινούμενη μονάδα βρίσκεται στην αφετηρία, η αφετηρία στην αρχή της αναζήτησης αποτελεί και τον τρέχοντα κόμβο. Υπολογίζονται οι τιμές του F όλων των εφαπτόμενων κόμβων στον τρέχοντα και προστίθενται προς εξέταση στην ανοικτή λίστα, ενέργεια η οποία αποτελεί το πρώτο στάδιο της επέκτασης της μεθοδίου. Επομένως, η ανοικτή λίστα, περιέχει κόμβους, εκ των οποίων κάποιος θα αποτελέσει τμήμα του μονοπατιού που αναζητείται.
2. Ελέγχονται μόνο οι προσπελάσιμοι παρακείμενοι κόμβοι του τρέχοντος. Εάν ένας κόμβος είναι μη προσπελάσιμος, δεν προστίθεται στην ανοικτή λίστα.
3. Αφαιρείται ο τρέχοντας κόμβος από την ανοικτή λίστα και προστίθεται στην κλειστή λίστα. Άπαξ και ένας κόμβος εισέλθει στην κλειστή λίστα, καθίσταται τμήμα της διαδρομής. Συνεπώς, κάθε κόμβος που περιλαμβάνεται στην κλειστή λίστα, αποτελεί τμήμα του μονοπατιού και ένα από τα βήματα που πρέπει να εκτελέσει η κινούμενη μονάδα για να καταλήξει στον στόχο.
4. Τέλος, με εξαίρεση τον κόμβο της αφετηρίας ο οποίος εξαρχής αποτελεί τμήμα της κλειστής λίστας, ο τρέχοντας κόμβος καθίσταται **βάση**, δηλαδή ένας κόμβος γύρω από τον οποίο θα αναζητηθεί το επόμενο βήμα.

Σχηματικά η διαδικασία αυτή περιγράφεται από το διπλανό διάγραμμα. Ο μπλε κόμβος που βρίσκεται στο κέντρο αποτελεί τη βάση. Το γαλάζιο περίγραμμα υποδηλώνει ότι ο κόμβος αυτός αποτελεί ήδη τμήμα της κλειστής λίστας. Όλοι οι γειτονικοί κόμβοι που εφάπτονται στη βάση εισέρχονται στην ανοικτή λίστα και αυτό υποδηλώνεται από το ανοικτό πράσινο περίγραμμα. Κάθε κόμβος που βρίσκεται στην ανοικτή λίστα έχει ένα δείκτη ο οποίος παραπέμπει στην βάση από την οποία προέρχεται, δηλαδή τον αρχικό μπλε κόμβο.



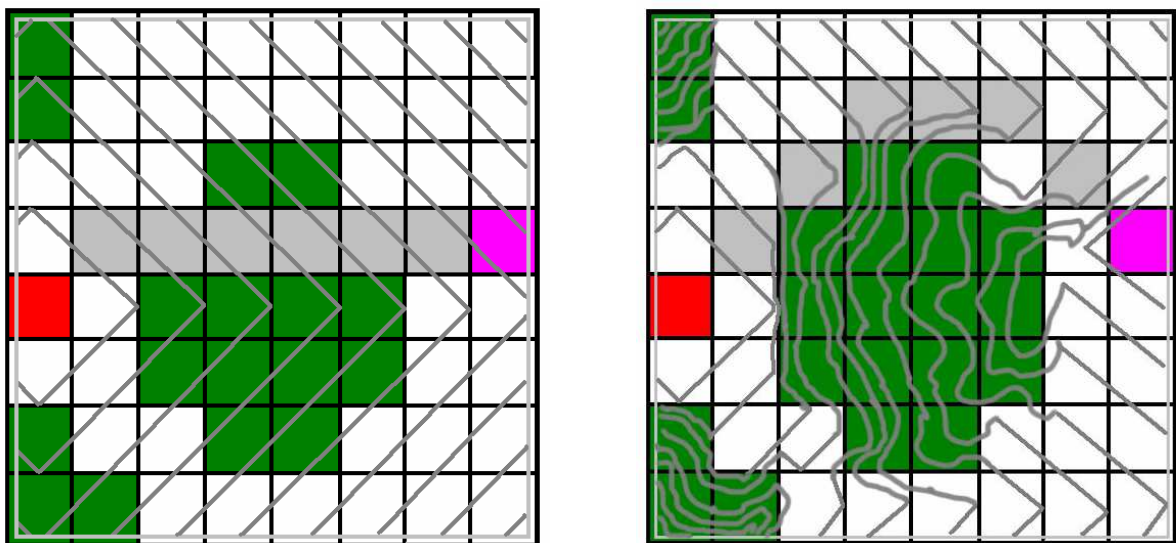
Σχηματική αναπαράσταση ανοικτής λίστας

4.2.2. Μεταβλητός Βαθμός Προσπέλασης

Πρακτικά στο φυσικό χώρο, μία περιοχή δε μπορεί να είναι απλά προσπελάσιμη ή μη προσπελάσιμη. Μπορεί μία περιοχή να είναι προσπελάσιμη, αλλά με μεγαλύτερο βαθμό δυσκολίας από ότι το επίπεδο έδαφος, όπως στην περίπτωση των βάλτων, των λόφων, των σκαλοπατιών κλπ. Ομοίως ένας δρόμος συνήθως παρουσιάζει μικρότερο βαθμό δυσκολίας στην κίνηση από ότι η περιοχή που τον περιβάλλει.

Αυτό το ζήτημα αντιμετωπίζεται εύκολα προσθέτοντας έναν βαθμό δυσκολίας στο G ενός δεδομένου κόμβου. Ο βαθμός αυτός αντιστοιχεί σε φυσικό σημείο με μεταβλητό βαθμό προσπέλασης. Ο αλγόριθμος A* επιλέγει μαθηματικά την πορεία διαμέσου της συνάρτησης F, πορεία η οποία θα ρυθμιστεί ανάλογα από το G. Λαμβάνοντας όμως, υπόψη τη παράμετρο της διαμόρφωσης του εδάφους, το βέλτιστο μονοπάτι δεν θα είναι πλέον αυτό που απέχει απλά την συντομότερη απόσταση από το στόχο, αλλά αυτό που προσεγγίζει ταχύτερα και πιο αβίαστα τον στόχο.

Είναι η χρήση αυτής της λίστας προτεραιότητας που αλλάζει τον τρόπο που η μεθοδολογία επεκτείνεται συγκριτικά με τη συνήθη προτεραιότητα,. Οι ισοϋψείς καμπύλες στο επόμενο σχήμα δηλώνουν την ταχύτητα διέλευσης του οχήματος από τον κάθε κόμβο. Οι πράσινοι κόμβοι αντιστοιχούν σε περιοχή δύσκολης διέλευσης.



Επέκταση μεθοδίου χωρίς και με υπολογισμό της διαμόρφωσης του εδάφους

Η πληροφορία του κόστους κίνησης μπορεί επίσης να χρησιμοποιηθεί για να αποφευχθούν ή να προτιμηθούν περιοχές με βάση την επικινδυνότητά τους.

Αυτός ο υπολογισμός της δαπάνης κίνησης επιτρέπει την εξερεύνηση πιο περίπλοκων γραφικών αναπαραστάσεων, όχι μόνο πλέγματα. Παράδειγμα αποτελεί ο υπολογισμός του κόστους μετακίνησης με βάση την απόσταση από δωμάτιο σε δωμάτιο ή από επαρχία σε επαρχία.

4.2.3. Διαμόρφωση Εδάφους

Αν ο χάρτης περιέχει περιοχές με διαφορετικό βαθμό προσπέλασης, τότε το βέλτιστο μονοπάτι υπολογίζεται δίνοντας στο ομαλότερο έδαφος μικρότερο κόστος κίνησης.

Παραδείγματος χάριν, εάν η κίνηση σε λιβάδι κοστίζει 1, στους λόφους 2, και στα ορεινά 3, τότε ο A^* θα προτιμήσει να κινηθεί μέσα από 3 λιβάδια για να αποφύγει μία ορεινή έκταση. Αν όμως το κόστος κίνησης τεθεί ως 1 σε έκταση καλυμμένη από λιβάδι, 1,1 στους λόφους, και 1,2 στα ορεινά, τότε ο A^* θα καταναλώσει λιγότερο χρόνο προσπαθώντας να αποφύγει τις ορεινές περιοχές, και θα προσδιορίσει το μονοπάτι ταχύτερα, προσεγγίζοντας τα οφέλη μίας ευρετικής συνάρτησης ακριβείας.

Επομένως το κόστος που θα ανατεθεί σε κάθε μορφολογία εδάφους, επηρεάζει τόσο το μονοπάτι, όσο και την ταχύτητα προσδιορισμού του. Ωστόσο, η επιλογή μεταξύ ταχύτητας εκτέλεσης και ακρίβειας δε χρειάζεται να είναι στατική (με την ανάθεση μίας σταθεράς τιμής ως κόστος κίνησης σε κάθε είδος εδάφους). Η απόφαση μπορεί να είναι δυναμική, βασιζόμενη δηλαδή σε μία πολυπλοκότερη συνάρτηση του $G(n)$. Υποθέτοντας ότι το κόστος βηματισμού είναι η μονάδα (1), είναι δυνατόν να σχεδιαστεί μια συνάρτηση κόστους με μεταβλητή τιμή.

$$G'(n) = 1 + \alpha \cdot (G(n) - 1)$$

Στην περίπτωση που το α είναι μηδέν, τότε το κόστος κίνησης θα είναι η μονάδα και η μορφολογία του εδάφους δεν λαμβάνεται υπόψη. Εάν το α είναι ίσο με τη μονάδα, η συνάρτηση κόστους της μορφολογίας του εδάφους χρησιμοποιείται, και ο A^* υπολογίζει την περιοχή που είναι δυσκολότερο να προσπελαστεί. Οι ενδιαμέσες τιμές, δηλαδή οι $0 < \alpha < 1$, θα πρέπει να αντιστοιχηθούν προσεκτικά σε κάθε είδος εδάφους.

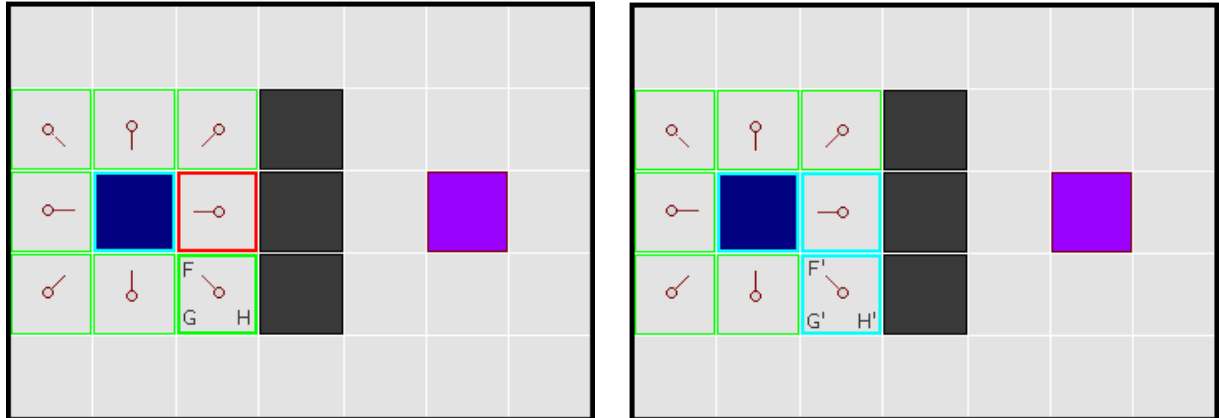
4.2.4. Διαγώνιος Βηματισμός

Ο διαγώνιος βηματισμός περιγράφεται από μαθηματική σκοπιά στο κεφάλαιο της ευρετικής. Ουσιαστικά για να επιτευχθεί αυτή η κίνηση, υπολογίζεται ο αριθμός των κινήσεων που μπορούν να γίνουν στην περίπτωση που οι διαγώνιες κινήσεις απαγορεύονται, και στη συνέχεια αφαιρούνται τα βήματα που εξοικονομούνται χρησιμοποιώντας τη διαγώνιο κίνηση.

Για να επιτευχθεί αυτό, σε κάθε στάδιο, πέρα από σύγκριση των τιμών του F , γίνεται ένας δευτερεύοντας έλεγχος του G των κόμβων του αναπτυσσόμενου μονοπατιού. Οι τιμές του G που συγκρίνονται, είναι η τρέχουσα τιμή G του κόμβου προς επιλογή που προέρχεται από την τρέχουσα βάση, με την τιμή του G που είχε ο κόμβος αυτός από τους υπολογισμούς της προηγούμενης από την τρέχουσα βάσης.

Με τη βοήθεια του ακόλουθου σχήματος αυτή διαδικασία γίνεται ευκολότερα κατανοητή. Ο μπλε κόμβος αποτελεί την αφετηρία και την αρχική βάση της κίνησης. Οι σκούροι κόμβοι παριστούν τα εμπόδια που θα πρέπει να παρακαμφθούν και μωβ είναι ο

στόχος της κίνησης. Με μία πρώτη διερεύνηση της ανοιχτής λίστας επιλέγεται ο σημειωμένος με κόκκινο περίγραμμα κόμβος ως νέα βάση και ακριβώς στον από κάτω κόμβο, τον διαγώνιο από την βάση, σημειώνονται οι τιμές των F, G και H που διέθετε ο εν λόγω κόμβος κατά την πρώτη αυτή επιλογή.



Διαδικασία εκτέλεσης διαγώνιου βήματος

Στη συνέχεια ο αλγόριθμος, έχοντας καθορίσει με κριτήριο το F τον πρότερο κόκκινο κόμβο ως νέα βάση, υπολογίζει τα μεγέθη F, G και H στους εφαπτόμενους κόμβους της νέας αυτής βάσης. Όπως γίνεται εμφανές και στο σχήμα, ο διαγώνιος κόμβος αποκτά νέες τιμές F', G' και H'. Συγκρίνοντας τα G και G' του κόμβου αυτού, ο αλγόριθμος θα μπορέσει να εκτελέσει την διαγώνιο κίνηση. Εάν το G' είναι μικρότερο του G, θα πρέπει να επέλθει αλλαγή της βάσης μεταξύ αυτών των δύο κόμβων. Δηλαδή να διαγραφεί ο κόκκινος κόμβος από την κλειστή λίστα και να προστεθεί ο διαγώνιος. Τέλος, αφού ολοκληρωθεί αυτή η διαδικασία, θα πρέπει να επαναυπολογιστούν τα F και G αυτού του κόμβου.

Ωστόσο, αυτή η τεχνική είναι πρακτικά υλοποιήσιμη μόνο στην περίπτωση που οι βάσεις της διαδρομής σχηματίζουν ορθή γωνία. Στην ευθεία πορεία μία τέτοια σύγκριση αποβαίνει καταστροφική.

5. ΕΥΡΕΤΙΚΗ (Heuristics)

Η ευρετική συνάρτηση $H(n)$ αναγκάζει τον αλγόριθμο να υπολογίσει κατά προσέγγιση το ελάχιστο κόστος που πρέπει να καταβληθεί ώστε να καταλήξει η μονάδα από οποιονδήποτε κόμβο n του πλέγματος, στον στόχο. Η επιλογή της ευρετικής συνάρτησης σχετίζεται άμεσα με τη μορφή της διαδρομής που θα προκύψει από την αναζήτηση.

Ο A^* υπολογίζει το $F(n) = G(n) + H(n)$. Για να προστεθούν αυτές οι δύο συναρτήσεις, θα πρέπει να βρίσκονται στην ίδια κλίμακα. Αν η $G(n)$ μετριέται σε ώρες και η $H(n)$ μετριέται σε μέτρα, τότε ο A^* θα υπολογίσει σε υψηλότερη τιμή ή το G ή το H στον υπολογισμό, με αποτέλεσμα είτε η διαδρομή που θα βρεθεί να μην είναι η βέλτιστη, είτε η εκτέλεση του A^* να γίνει χρονοβόρα.

5.1 Ευρετική σε Ιδανικές Συνθήκες

Όταν η ευρετική ισούται ακριβώς με το μήκος της βέλτιστης διαδρομής, διερευνώνται λιγостоί κόμβοι του χώρου. Ο αλγόριθμος υπολογίζει τη τιμή του $F(n)$ σε κάθε κόμβο που διερευνάται. Όταν οι τιμές των $H(n)$ και $G(n)$ είναι ίσες, η τιμή του $F(n)$ παραμένει σταθερή κατά μήκος της διαδρομής. Οι κόμβοι που δεν ανήκουν στη βέλτιστη διαδρομή, θα έχουν υψηλότερη τιμή F από τους κόμβους που ανήκουν.

Επειδή ο A^* δεν διερευνά υψηλής F αξίας κόμβους, δεν παρεκτρέπεται από την βέλτιστη διαδρομή.

5.1.1. Υπολογισμός Επακριβούς Ευρετικής

Προ-υπολογισμένη Ευρετική Ακριβείας: Μία μέθοδος δόμησης μίας ευρετικής ακριβείας, είναι μέσω του προϋπολογισμού του μήκους της βέλτιστης διαδρομής ανάμεσα σε κάθε ζεύγος σημείων, μέθοδος μη εφαρμόσιμη στους περισσότερους χάρτες. Ωστόσο, μπορεί να προσεγγιστεί αυτή η μέθοδος ξεκινώντας με τους ακόλουθους τρόπους:

- Τοποθετώντας ένα χονδροειδέστερο πλέγμα πάνω από το λεπτομερέστερο που απεικονίζει το χάρτη. Ακολουθεί προϋπολογισμός της συντομότερης διαδρομής μεταξύ οποιουδήποτε ζεύγους κόμβων w_1 & w_2 του χονδροειδούς πλέγματος, ή
- Προϋπολογισμός της συντομότερης διαδρομής μεταξύ δύο οποιονδήποτε σημείων πλοήγησης w_1 & w_2 (γενίκευση της προσέγγισης χονδροειδούς πλέγματος)

Στη συνέχεια, εφαρμόζεται σε μια ευρετική H' , που υπολογίζει το κόστος της μετάβασης από μία οποιαδήποτε θέση n στο κοντινό σημείο πλοήγησης w_1 . Η τελική ευρετική είναι:

$$H(n) = H'(n, w_1) + \text{απόσταση}(w_1, w_2) + H'(w_2, \text{goal})$$

Όπου w_1, w_2 τα ζεύγη των κόμβων, n ο τρέχων κόμβος και goal ο κόμβος προορισμού.

Γραμμική Ευρετική Ακριβείας: Στην ειδική περίπτωση όπου στο χάρτη δεν υπάρχουν εμπόδια και το έδαφος είναι ομαλό για κίνηση, τότε η βέλτιστη διαδρομή από την αφετηρία ως στο στόχο θα είναι ευθεία γραμμή. Υπό αυτές τις συνθήκες, η ευρετική μπορεί να είναι ακριβής χωρίς να χρειαστεί να προϋπολογιστεί κάτι.

5.2 Ευρετική σε Χάρτη Πλέγματος

Σε οποιασδήποτε μορφής πλέγμα, καθορίζοντας την κίνηση που επιτρέπεται να επιτελέσει η κινούμενη μονάδα, επιλέγεται η κατάλληλη ευρετική συνάρτηση. Η ευρετική της απόστασης ανήκει σε μία από τις ακόλουθες κατηγορίες ανάλογα με την επιτρεπόμενη κίνηση:

- Σε τετραγωνικό πλέγμα που επιτρέπει κίνηση σε 4 κατευθύνσεις, χρησιμοποιείται την **απόσταση Manhattan (L_1)**.
- Σε τετραγωνικό πλέγμα που επιτρέπει κίνηση σε 8 κατευθύνσεις, χρησιμοποιείται η **απόσταση Diagonal (L_∞)**.
- Σε τετραγωνικό πλέγμα που επιτρέπει κίνηση προς κάθε κατεύθυνση, μπορεί να επιλεγεί η **Ευκλείδεια απόσταση (L_2)**. Αν είναι επιτρεπτές κινήσεις και εκτός πλέγματος, μπορούν να εξεταστούν άλλες απεικονίσεις του χάρτη.
- Σε πλέγμα εξαγωνικό που επιτρέπει κίνηση σε 6 κατευθύνσεις, χρησιμοποιείται η **απόσταση Manhattan** προσαρμοσμένη στο εξαγωνικό πλέγμα.

5.2.1. Απόσταση Manhattan

Η κλασικότερη ευρετική συνάρτηση που χρησιμοποιείται στο τετραγωνικό πλέγμα είναι η απόσταση Manhattan. Το κύριο πλεονέκτημά αυτής της μεθόδου είναι ότι συγκριτικά με τις εναλλακτικές ευρετικές, καταλήγει ταχύτερα στο στόχο.

Αντιθέτως, κύριο μειονέκτημα της είναι ότι σε αυτό το πλέγμα, επιτρέπεται κίνηση παράλληλη με τους άξονες του πλέγματος μόνο (οριζόντιος και κάθετος κίνηση), οπότε το μονοπάτι που θα βρεθεί δε θα είναι το συντομότερο δυνατό, αν και θα πλησιάζει πολύ το βέλτιστο. Επιπλέον, επειδή στη συνάρτηση της προτεραιότητας F , το H έχει μεγαλύτερη βαρύτητα από το G , η απόσταση Manhattan αγνοεί τις μικρές ρυθμίσεις που μπορεί να χρειαστεί να προστεθούν στον υπολογισμό του G . Περιγραφή των συνθηκών που απαιτούν τη ρύθμιση του G , γίνεται στην επόμενη παράγραφο.

Μελετώντας τη συνάρτηση του κόστους πρέπει να δοθεί προσοχή στο ελάχιστο κόστος D της μετάβασης (του βηματισμού δηλαδή) από τον ένα κόμβο στον παρακείμενο. Στην απλούστερη περίπτωση τίθεται $D=1$. Η ευρετική θα πρέπει να είναι D φορές η απόσταση Manhattan:

Κίνηση στον άξονα x: $dx = |node.x - goal.x|$

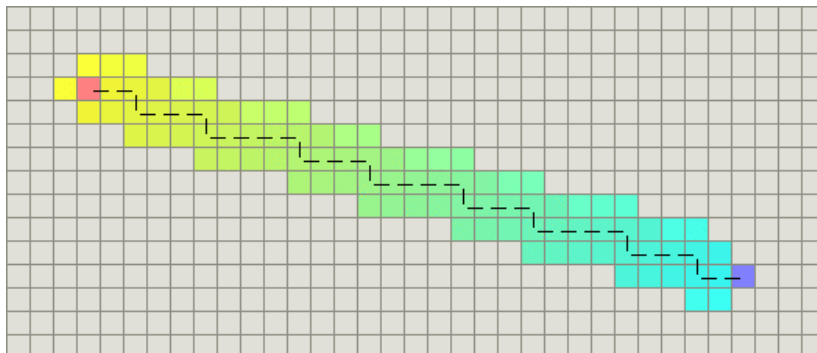
Κίνηση στον άξονα y: $dy = |node.y - goal.y|$

ΑΠΟΣΤΑΣΗ: $D \cdot (dx + dy)$

Μέσα στην απόλυτη τιμή αφαιρούνται οι συντεταγμένες του κόμβου προορισμού από τον τρέχοντα κόμβο.

Τεχνική επιλογής κόστους κίνησης : Έστω D το κόστος βηματισμού. Κατά την επιλογή αυτή, προσοχή πρέπει να δοθεί στο να είναι η χρησιμοποιούμενη κλίμακα ίδιας τάξης μεγέθους με την κλίμακα της συνάρτησης του κόστους. Για το βέλτιστο μονοπάτι, στη κίνηση μεταξύ γειτονικών κόμβων (D) θα πρέπει να δοθεί η χαμηλότερη δυνατή τιμή.

Σε περίπτωση απουσίας εμποδίων, και σε έδαφος ομαλό το οποίο χαρακτηρίζεται επίσης από ελάχιστο κόστος κίνησης D, η κίνηση προς τον στόχο θα πρέπει να αυξάνει το G κατά D και να μειώνει αντίστοιχα το H κατά D. Όταν προστίθενται αυτά τα δύο, το F το οποίο έχει οριστεί ως $F = G + H$, θα παραμένει σταθερό κατά μήκος της διαδρομής. Ακόμη, μπορεί το Pathing να καταλήξει ταχύτερα στο στόχο, διατηρώντας διαφορετική αναλογία κόστους D ανάμεσα στις χαμηλότερες και υψηλότερες ακμές ή κορυφές του κόμβου. Το αν θα αφορά ο υπολογισμός ακμές ή κορυφές εξαρτάται από τον σχεδιασμό του πλέγματος.



Κίνηση με ευρετική απόστασης Manhattan

Στο παραπάνω πλέγμα, η αφετηρία της διαδρομής σημειώνεται με κόκκινο και ο στόχος με μωβ. Οι χρωματιστοί κόμβοι του πλέγματος αντιστοιχούν στους κόμβους τους οποίους διερεύνησε κατά την αναζήτηση το Pathing (στην προκειμένη περίπτωση ο αλγόριθμος A*). Με κίτρινο συμβολίζεται το υψηλότερο κόστος της ευρετικής συνάρτησης H και με κυανό το χαμηλότερο.

5.2.2. Απόσταση Diagonal

Στην περίπτωση που επιτρέπεται η διαγώνιος κίνηση στον χάρτη, η απόσταση Manhattan δεν ενδείκνυται. Προστίθενται 4 επιπλέον δυνατές κατευθύνσεις κίνησης και συνολικά οι δυνατές κατευθύνσεις γίνονται 8.

Η μέθοδος αυτή εξισορροπεί τα G και H. Εξαιτίας αυτής της ισορροπίας, είναι δυνατός ο υπολογισμός μικρότερων ρυθμιστικών παραμέτρων, όπως το επιπλέον κόστος στροφής ή η διαμόρφωση του εδάφους. Μειονεκτεί ωστόσο συγκριτικά με την απόσταση Manhattan από άποψη ταχύτητας εύρεσης του προορισμού.

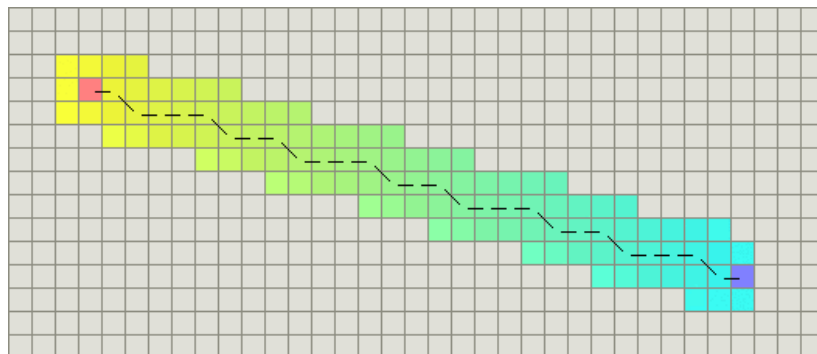
Κίνηση στον άξονα x: $dx = \text{abs}(\text{node.x} - \text{goal.x})$

Κίνηση στον άξονα y: $dy = \text{abs}(\text{node.y} - \text{goal.y})$

ΑΠΟΣΤΑΣΗ: $D \cdot (dx + dy) + (D_2 - 2 \cdot D) \cdot \min(dx, dy)$

Όπως περιγράφηκε στο κεφάλαιο του A*, στην περίπτωση diagonal υπολογίζεται ο αριθμός των κινήσεων που θα μπορούσαν να γίνουν εάν οι διαγώνιες κινήσεις απαγορευόταν, και έπειτα από αυτό τον αριθμό αφαιρείται ο αριθμός των βημάτων που εξοικονομούνται χρησιμοποιώντας τη διαγώνια κίνηση. Υπάρχουν $\min(dx, dy)$ διαγώνια βήματα, καθένα εκ των οποίων κοστίζει D_2 , αλλά αυτά τα διαγώνια βήματα εξοικονομούν $2 \cdot D$ μη διαγώνιες κινήσεις.

Αν το κόστος της κίνησης είναι D , τότε η κίνηση στη διαγώνιο του τετραγώνου θα είναι με βάση το Πυθαγόρειο Θεώρημα $D_2 = \sqrt{2} \cdot D$. Δηλαδή, στην περίπτωση όπου η πλευρά του τετραγωνικού πλέγματος είναι $D=1$, τότε η διαγώνιος του θα είναι $D_2 = \sqrt{2}$.



Κίνηση με ευρετική απόστασης Diagonal

Στο παραπάνω πλέγμα, χρησιμοποιείται ο ίδιος χρωματικός κώδικας με την προηγούμενη περίπτωση. Παρατηρούμε ότι η περιοχή που σαρώνει ο αλγόριθμος είναι ελάχιστα μεγαλύτερη, επομένως και ο χρόνος ολοκλήρωσης της αναζήτησης θα είναι μεγαλύτερος. Ωστόσο η τελική διαδρομή είναι συντομότερη.

Οι παραπάνω δύο περιπτώσεις απαντώνται στις περισσότερες εφαρμογές. Συνοπτικά, όσο περισσότερο ταυτίζεται η προσέγγιση H με την ακριβή απόσταση που απομένει να διανυθεί, τόσο γρηγορότερα καταλήγει ο αλγόριθμος στο στόχο. Ωστόσο, υπερεκτίμηση της τιμής του H σχετικά με την απομένουσα απόσταση, μπορεί να αποφέρει μη βέλτιστα μονοπάτια. Επομένως είναι απαραίτητη η προσέγγιση του H με όσο δυνατόν μεγαλύτερη ακρίβεια.

5.2.3. Ευκλείδεια Απόσταση

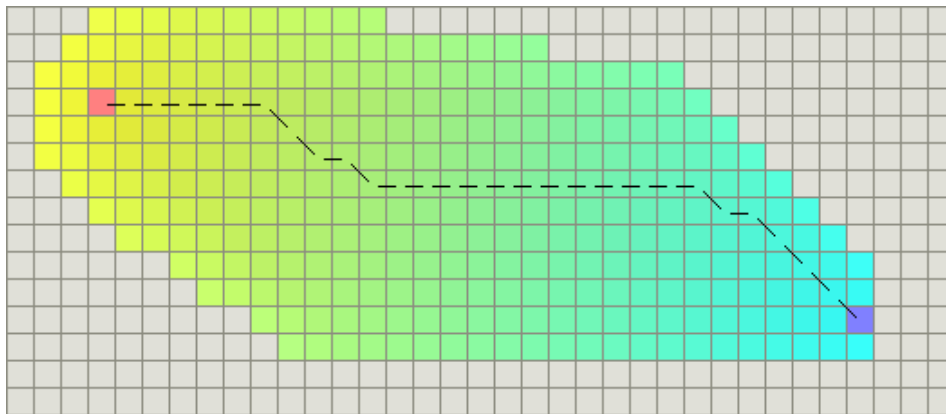
Εάν η κινούμενη μονάδα μπορεί να κινηθεί προς οποιαδήποτε γωνία (όχι μόνο αυτές των κατευθύνσεων του πλέγματος), τότε μάλλον ενδείκνυται μια ευθεία απόσταση:

Κίνηση στον άξονα x: $dx = \text{abs}(\text{node.x} - \text{goal.x})$

Κίνηση στον άξονα y: $dy = \text{abs}(\text{node.y} - \text{goal.y})$

ΑΠΟΣΤΑΣΗ: $D \cdot \sqrt{(dx \cdot dx + dy \cdot dy)}$

Ωστόσο, στην περίπτωση αυτή, η χρήση του αλγόριθμου A* δε θα είναι και τόσο αποδοτική, επειδή η συνάρτηση κόστους του G δεν είναι της ίδιας κλίμακας με την ευρετική συνάρτηση H. Εφόσον η Ευκλείδεια απόσταση είναι μικρότερη αυτής της Manhattan ή της Diagonal, πάλι θα προκύπτει το βέλτιστο μονοπάτι, όμως το Path Finder θα χρειάζεται περισσότερο χρόνο για να ολοκληρώσει την αναζήτηση.



Κίνηση με ευρετική Ευκλείδεια απόσταση

Στο παραπάνω πλέγμα, χρησιμοποιείται ο ίδιος χρωματικός κώδικας με τις παραπάνω περιπτώσεις. Παρατηρούμε ότι η περιοχή που σαρώνει ο αλγόριθμος είναι κατά πού μεγαλύτερη, επομένως και ο χρόνος ολοκλήρωσης της αναζήτησης θα είναι πολλαπλά μεγαλύτερος. Ωστόσο η τελική διαδρομή είναι η συντομότερη δυνατή και σε αυτή την περίπτωση.

Σε κάποιες περιπτώσεις συνιστάται η αποφυγή της τετραγωνικής ρίζας στην Ευκλείδεια απόσταση και ο υπολογισμός της χωρίς τετράγωνο με βάση τον επόμενο τύπο:

ΑΠΟΣΤΑΣΗ: $D \cdot (dx \cdot dx + dy \cdot dy)$

Ο υπολογισμός αυτός ωστόσο, συνοδεύεται πάντοτε από το πρόβλημα κλίμακας και εκφυλίζει την ακρίβεια της ευρετικής.

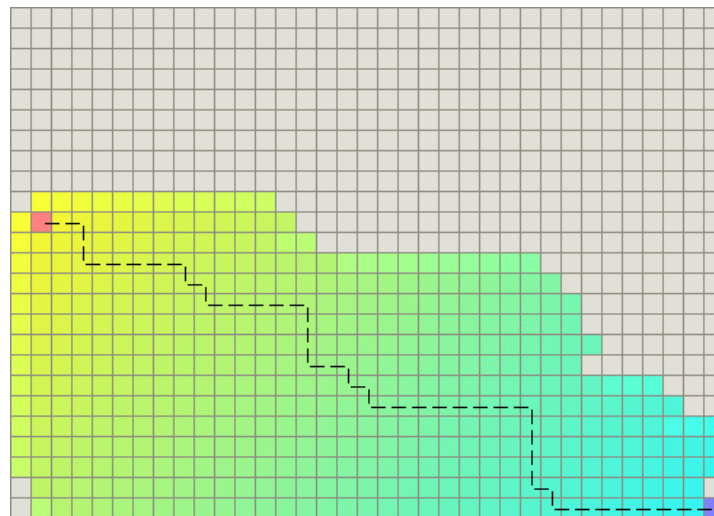
5.2.4. Ευρετική & Πολλαπλοί στόχοι

Στην περίπτωση όπου υπάρχουν πάνω από ένας προορισμοί, θα πρέπει να συνταχθεί μια ευρετική συνάρτηση $H'(x)$, που θα επιστρέφει τη μικρότερη των συναρτήσεων $H_1(x)$, $H_2(x)$, $H_3(x)$, ..., όπου H_1, H_2, H_3 η ευρετική κάθε ενός από τους στόχους.

Αν χρειαστεί να αναζητηθεί κάποιο σημείο του χάρτη που βρίσκεται κοντά σε έναν από τους στόχους και όχι ο ίδιος ο στόχος, θα πρέπει αρχικά να βρεθεί μια διαδρομή προς την κατεύθυνση που τοποθετούνται οι στόχοι. Η διαδικασία ολοκληρώνεται μέσα από την επεξεργασία των κόμβων της ανοιχτής λίστας, μόλις εντοπιστεί ένας επαρκώς κοντινός κόμβος στο σημείο αυτό.

5.3 Πολλαπλά και Ισότιμα H

Σε ορισμένους χάρτες πλέγματος υπάρχουν περισσότερες από μία διαδρομές με το ίδιο μήκος. Αυτό συνήθως συμβαίνει, σε επίπεδες περιοχές με σταθερή διαμόρφωση εδάφους, όπου απαντώνται πολλά ισομήκη μονοπάτια. Ένας αλγόριθμος, όποια ευρετική συνάρτηση και να διαθέτει, διερευνά όλες τις πιθανές διαδρομές με την ίδια αξία F , εφόσον δεν υπάρχει μία βέλτιστη.



Κίνηση σε χάρτη με πολλαπλά ισότιμα μονοπάτια

Στο παραπάνω πλέγμα, παρατηρούμε ότι η περιοχή που σαρώνει ο αλγόριθμος είναι τεράστια και επομένως η διαδικασία καθίσταται πολύ χρονοβόρα. Όποια διαδρομή και να βρεθεί ωστόσο, είναι βέλτιστη.

Για να λυθεί αυτό το πρόβλημα, θα πρέπει να ρυθμιστεί είτε η τιμή του G είτε του H . Η ισοτιμία θα πρέπει να λύνεται με χρήση της τιμής της επιλεγόμενης συνάρτησης (G ή H) ως προς τον κόμβο, και πρέπει να έχει ως αποτέλεσμα την παραγωγή διαφορετικών τιμών για το F .

Ακολουθεί περιγραφή μεθόδων για να επιτευχθεί η λύση της ισοτιμίας :

1^{ος} ΤΡΟΠΟΣ :

Ένας τρόπος είναι η ελαφριά μεταβολή της κλίμακας του H. Αν περιοριστεί προς τα κάτω, τότε η F θα αυξάνεται καθώς πλησιάζουμε προς τον στόχο. Δυστυχώς, αυτό σημαίνει ότι ακόμη και ο A* θα επεκτείνει την αναζήτηση γύρω από την αφετηρία, αντί του στόχου. Αν αντιθέτως, η H αυξηθεί ελαφρά (έστω και μόνο κατά 0,1%), τότε ο A* θα επεκτείνει την αναζήτηση γύρω από τον στόχο.

$$\text{heuristic}^* = (1.0 + p)$$

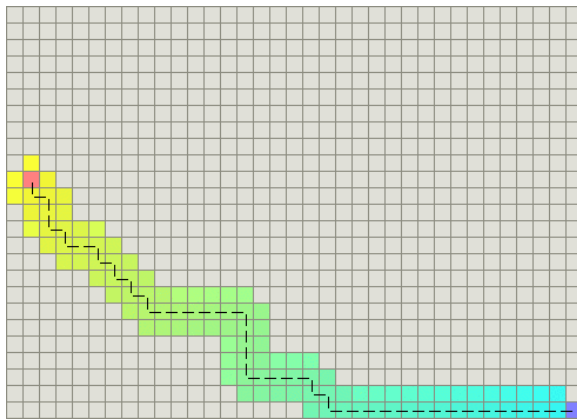
Ο συντελεστής p θα πρέπει να επιλέγεται ούτως ώστε να ισχύει

$$p < \frac{\text{ελάχιστο} \cdot \text{κόστος} \cdot \text{ανά} \cdot \text{βήμα}}{\text{αναμενόμενο} \cdot \text{μέγιστο} \cdot \text{μήκος} \cdot \text{διαδρομής}}$$

Υποθέτοντας ότι το μονοπάτι δεν είναι μακρύτερο από 1000 βήματα, επιλέγεται

$p = \frac{1}{1000}$. Αυτή βέβαια, δεν είναι ακριβώς αποδεκτή ευρετική, αλλά δεν έχει μεγάλη επίδραση στο τελικό αποτέλεσμα.

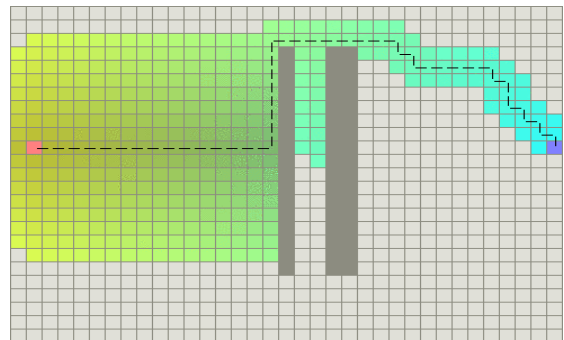
Το αποτέλεσμα είναι ότι ο A* διερευνά πολύ μικρότερη περιοχή όπως φαίνεται στο διπλανό σχήμα.



Προσθήκη σπασίματος ισοτιμίας στην Ευρετική

Η προσθήκη εμποδίων, φυσικά έχει ως αποτέλεσμα την μεγαλύτερη διερεύνηση του χάρτη προκειμένου να βρεθεί ένα μονοπάτι που να τα παρακάμπτει. Σημειωτέον, ότι άπαξ και περαστεί το εμπόδιο, ο A* διερευνά ελάχιστα την περιοχή.

Η περίπτωση του χάρτη με εμπόδια εικονίζεται στο διπλανό σχήμα:



Προσθήκη σπασίματος ισοτιμίας μετ'εμποδίων

2^{ος} ΤΡΟΠΟΣ :

Υπάρχει ένας απλούστερος τρόπος για επιτευχθούν τα παραπάνω, μέσω σύγκρισης των τιμών του H για κάθε F που ανήκει στο σύνολο της ισοτιμίας.

3^{ος} ΤΡΟΠΟΣ :

Ακόμη ένας τρόπος είναι η πρόσθεση ενός τυχαίου αριθμού που καθορίζεται από άλλες παραμέτρους στην ευρετική, ή στο κόστος της ακμής. Η επιλογή του αριθμού αυτού προέρχεται από τον κατακερματισμό των συντεταγμένων. Με αυτόν τον τρόπο δε ρυθμίζεται ουσιαστικά το H, αλλά πραγματικά σπάει η ισοτιμία.

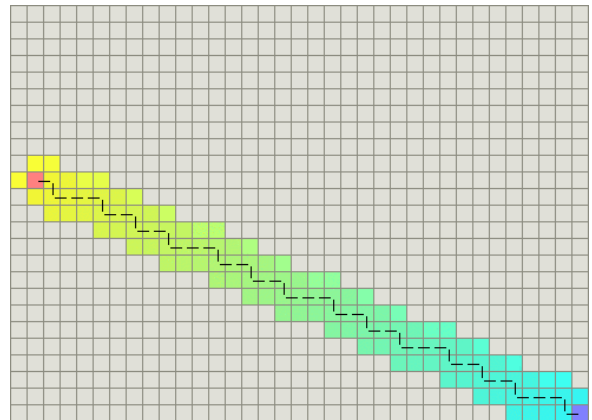
4^{ος} ΤΡΟΠΟΣ :

Επίσης, μπορεί να δοθεί μεγαλύτερη βαρύτητα στα μονοπάτια που βρίσκονται κατά μήκος της ευθείας γραμμής που ενώνει την αφετηρία και το στόχο:

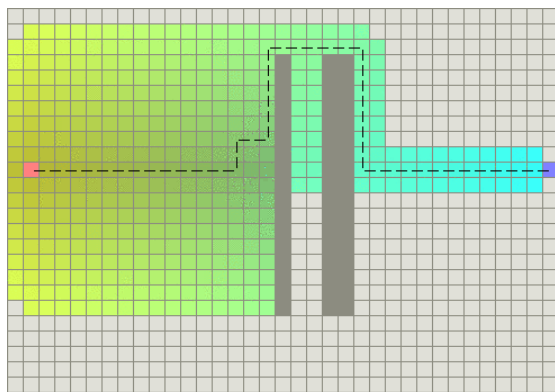
Συντεταγμένες απόστασης από στόχο	Μήκος ευθείας αφετηρίας-στόχου
$dx1 = \text{current.x} - \text{goal.x}$	$dx2 = \text{start.x} - \text{goal.x}$
$dy1 = \text{current.y} - \text{goal.y}$	$dy2 = \text{start.y} - \text{goal.y}$
$\text{cross} = dx1 \cdot dy2 - dx2 \cdot dy1 $	
$\text{heuristic}^+ = \text{cross} \cdot 0.001$	

Οι παραπάνω εξισώσεις υπολογίζουν το γινόμενο διανυσμάτων μεταξύ του διανύσματος αφετηρίας-προορισμού και του διανύσματος τρέχοντος σημείου-προορισμού. Όταν τα διανύσματα αυτά δεν είναι ευθυγραμμισμένα, το γινόμενό τους θα είναι μεγαλύτερο.

Ως αποτέλεσμα, ο αλγόριθμος θα εμφανίζει μία ελαφριά προτίμηση στη διαδρομή που βρίσκεται κατά μήκος της ευθείας που ενώνει την αφετηρία με το στόχο. Υπό συνθήκες έλλειψης εμποδίων, ο A* όχι μόνο διερευνά μικρή περιοχή, μα και η διαδρομή είναι συμμετρική και ομοιόμορφη. Το μονοπάτι εικονίζεται στον διπλανό χάρτη:



Σπάσιμο ισοτιμίας με γινόμενο διανυσμάτων



Σπάσιμο ισοτιμίας με γινόμενο διανυσμάτων μετ'εμποδίων

Ωστόσο, με την προσθήκη εμποδίων στο πλέγμα, παρατηρούμε ότι το μονοπάτι που προσδιορίζεται δεν είναι τόσο ομοιόμορφο ή συμμετρικό. Παρότι η διαδρομή που βρίσκεται είναι η βέλτιστη, η παράκαμψη του εμποδίου δε συμβαίνει τόσο αβίαστα, όπως και φαίνεται στο διπλανό σχήμα. Αυτό οφείλεται σε αυτή ακριβώς την προτίμηση της διεύθυνσης της ευθείας που ενώνει την αφετηρία και τον στόχο.

5^{ος} ΤΡΟΠΟΣ :

Ακόμη, με προσεκτική σχεδίαση της λίστας προτεραιότητας του A*, μπορεί να αποφευχθεί η εμφάνιση ισοτιμίας. Ο σχεδιασμός αυτός περιστρέφεται γύρω από την προσπάθεια οι νεότερες προσθήκες στη λίστα που χαρακτηρίζονται από μία

συγκεκριμένη τιμή F να κατατάσσονται με υψηλότερη τιμή προτεραιότητας από παλαιότερες εισαγωγές με την ίδια τιμή F .

6^{ος} ΤΡΟΠΟΣ :

Άλλη μέθοδος είναι η ελαχιστοποίηση των στροφών, η οποία προϋποθέτει την χρήση των ακμών του κόμβου. Η μεταβολή των x, y κατά τη μετακίνηση από τη βάση στον τρέχοντα κόμβο, υποδηλώνει την κατεύθυνση της κίνησης. Για τις ακμές που εξετάζονται, από τον τρέχοντα στον γείτονα κόμβο, αν η αλλαγή στα x, y είναι διαφορετική από εκείνη της βάσης προς τον τρέχοντα, θα πρέπει να προστίθεται μια μικρή ποινή στο κόστος μετακίνησης.

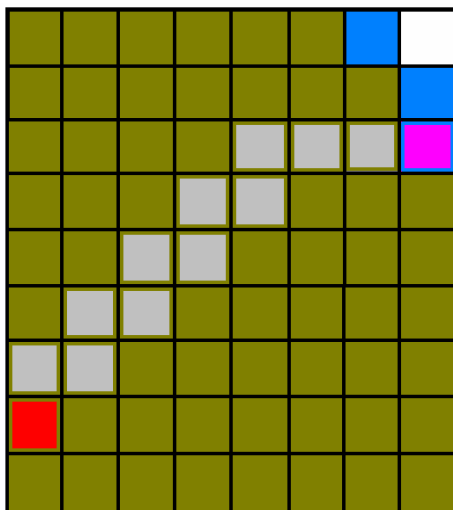
6. ΕΥΡΕΤΙΚΗ ΑΝΑΖΗΤΗΣΗ ΚΑΙ ΑΠΟΔΟΣΗ ΑΛΓΟΡΙΘΜΩΝ

Κατά την προσπάθεια προσδιορισμού μιας διαδρομής είτε προς όλες τις δυνατές θέσεις/κόμβους είτε προς πολλαπλούς στόχους, η επέκταση της μεθόδου προς όλες τις κατευθύνσεις με την ίδια δυναμική αποτελεί την λογικότερη προσέγγιση (αλγόριθμος Breadth First Search). Αυτό επιτυγχάνεται συνδυάζοντας την απλή αναζήτηση που δεν περιλαμβάνει το κόστος κίνησης και τον αλγόριθμο του Dijkstra.

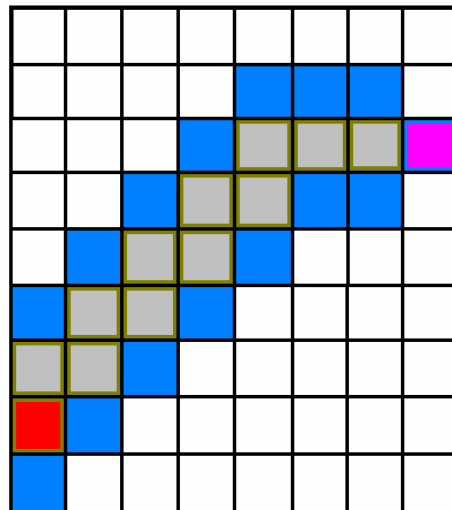
Ωστόσο, στις περισσότερες περιπτώσεις ο προσορισμός της μετατόπισης, ο κόμβος προσορισμού είναι ένας. Το ιδανικότερο σε αυτή την περίπτωση θα ήταν η μέθοδος να επεκτείνεται περισσότερο προς την κατεύθυνση του στόχου από ότι προς τις άλλες κατευθύνσεις (αλγόριθμος Greedy Best First Search). Για να επιτευχθεί αυτό ορίζεται μια ευρετική συνάρτηση που υπολογίζει την απόσταση από το στόχο.

Στον αλγόριθμο του Dijkstra χρησιμοποιείται μόνο η ακριβής απόσταση από την αφετηρία για την ταξινόμηση στη λίστα προτεραιότητας. Αντίθετα, στην περίπτωση Greedy Best First Search, εκτιμάται μόνο η απόσταση από το στόχο για την κατάταξη των κόμβων στη λίστα προτεραιότητας, άρα, πρώτα θα διερευνάται η πλησιέστερη προς το στόχο περιοχή.

Τα ακόλουθα σχήματα αναπαριστούν τους αλγόριθμους Dijkstra's και Greedy Best First Search για της περιπτώσεις που στο χάρτη δεν υπάρχουν και υπάρχουν εμπόδια. Με χακί συμβολίζεται η περιοχή που διερευνήθηκε από τον αλγόριθμο, και με γαλάζιο οριοθετείται η μέθοδος.



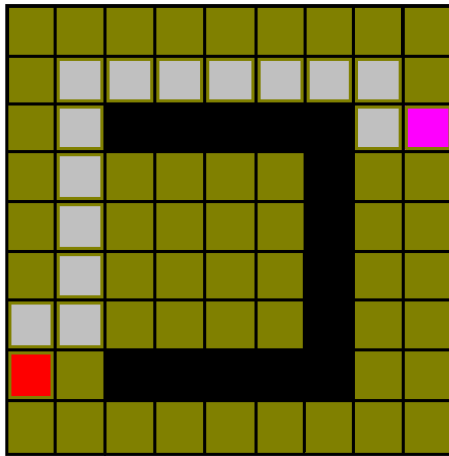
Dijkstra's



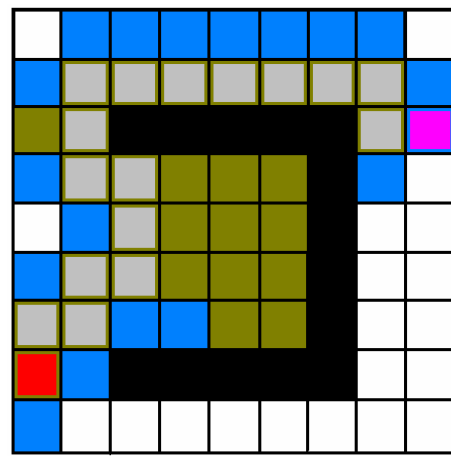
Greedy Best First Search

Παρατηρούμε ότι ο αλγόριθμος Dijkstra's διερευνά μεγαλύτερη περιοχή και επομένως δεν καταλήγει τόσο γρήγορα στον στόχο συγκριτικά με τον αλγόριθμο Greedy Best First Search. Όμως το μονοπάτι το οποίο βρίσκεται και από τους δύο αλγόριθμους είναι

πανομοίωτο. Προσθέτοντας κοίλα εμπόδια στο χώρο, προκύπτουν τα ακόλουθα αποτελέσματα:



Dijkstra's



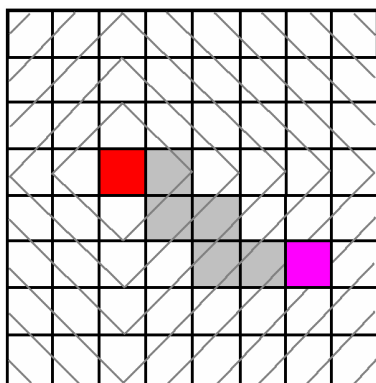
Greedy Best First Search

Όπως είναι αναμενόμενο, ο αλγόριθμος Breadth First Search μελετάει και πάλι μεγαλύτερη περιοχή και καταλήγει και πάλι αργότερα στον στόχο, όμως ο Greedy Best First Search δε βρίσκει το βέλτιστο μονοπάτι σε αυτή την περίπτωση.

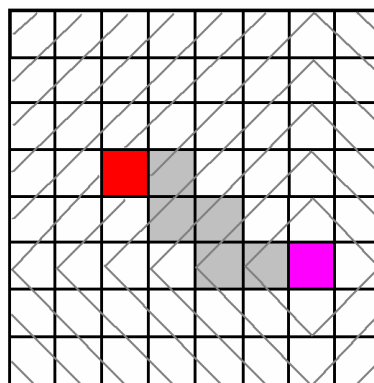
6.1 A*STAR Algorithm

Ο αλγόριθμος του Dijkstra ανταποκρίνεται ικανοποιητικά στην εύρεση της βέλτιστης διαδρομής, όμως καταναλώνει χρόνο στην διερεύνηση μη πιθανών κατευθύνσεων. Ο Greedy Best First Search διερευνά τις πιθανές κατευθύνσεις, αλλά δε βρίσκει τη βέλτιστη διαδρομή. Ο αλγόριθμος A * χρησιμοποιεί και την ακριβή απόσταση από την αφετηρία, απόσταση η οποία έχει διανυθεί ήδη, και την εκτιμώμενη απόσταση από το στόχο, και με αυτόν τον τρόπο να αποκλείει μη βέλιστα μονοπάτια άπαξ και βρεθεί ένα αρχικό.

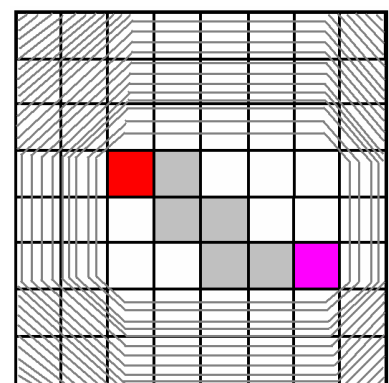
Συγκρίνοντας τον A * με τους Dijkstra και Greedy Best First Search παρατηρούμε αρχικά τα διαγράμματα των ισοϋψών γραμμών:



Dijkstra's



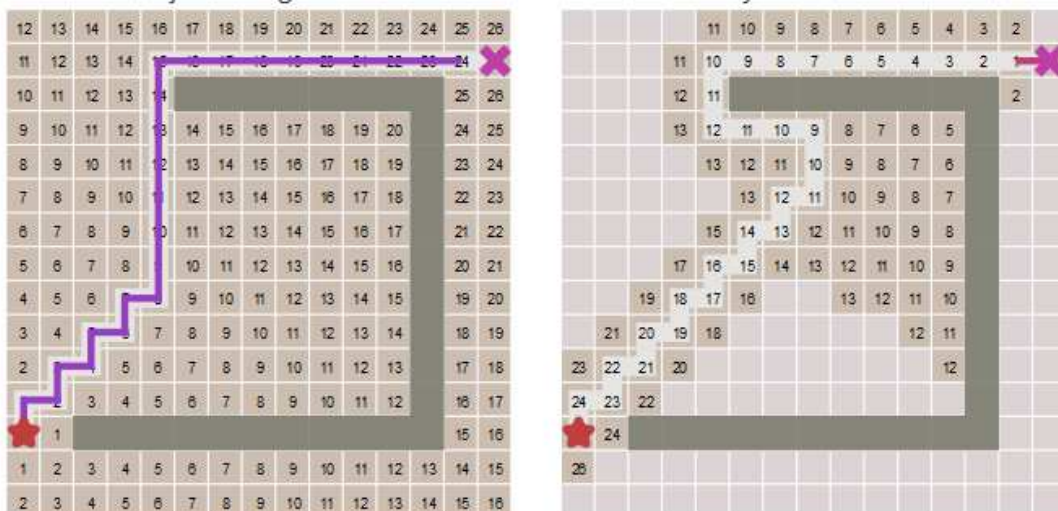
Greedy Best First Search



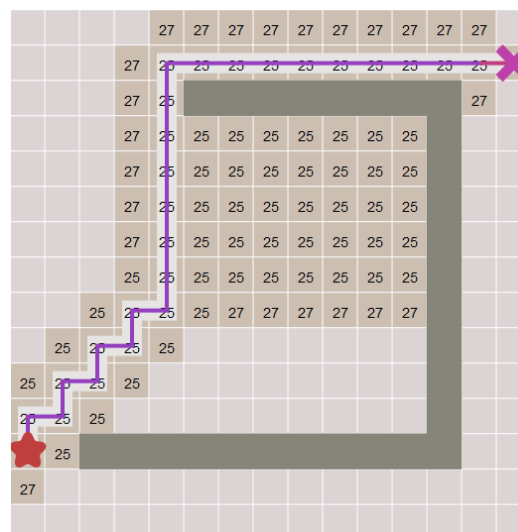
A*STAR

- Ο Αλγόριθμος Dijkstra's, αναζητά το στόχο έχοντας ως κέντρο την αφετηρία. Καθώς δεν είναι βέβαιη η περιοχή στην οποία βρίσκεται ο στόχος, ελέγχει όλες τις κατευθύνσεις περιμετρικά της αφετηρίας.
- Ο Greedy Best First Search επίσης, αναζητά το στόχο έχοντας ως κέντρο την αφετηρία, γνωρίζοντας όμως τη κατεύθυνση στην οποία βρίσκεται ο στόχος, οπότε και κινείται ανάλογα.
- Ο A* όντας ο συνδυασμός των δύο προηγούμενων, διαθέτει ισοϋψείς καμπύλες οι οποίες αντί να αποτελούν ένδειξη της απόστασης της αφετηρίας από τον στόχο, υποδεικνύουν απλά το μήκος του μονοπατιού. Οι μικρότερες διαδρομές εντοπίζονται στην εσωτερική περιοχή των ισοϋψών.

Ο A* λοιπόν, διερεύνα πρώτα την εσωτερική περιοχή, και κινείται προς τα έξω μόνο αν δεν μπορεί να εντοπίσει μια διαδρομή εκεί. Ο κώδικας του είναι συντακτικά παρόμοιος με του Dijkstra, όμως η προσθήκη της ευρετικής συνάρτησης βελτιώνει δραματικά την ταχύτητα αναζήτησης. Ακολουθεί παράδειγμα σε πλέγμα που δεν επιτρέπει διαγώνιο κίνηση και με κόστος κίνησης τη μονάδα στους δύο πρώτους αλγορίθμους:



Υπολογισμοί κόστους κίνησης αλγορίθμων Α) Greedy Best First Search και Β) Dijkstra's



Υπολογισμοί κόστους κίνησης αλγορίθμου A* (ευρετική Manhattan)

Είναι εμφανές ότι ο A^* , εκτελείται γρηγορότερα από ό, τι ο αλγόριθμος του Dijkstra, ακόμη και αν εφαρμόστηκε ένας επιπλέον αλγόριθμος βελτίωσης της διαδρομής ο οποίος βασίζεται στη μεθόριο. Συγκρίνοντας τον A^* με τον Greedy Best First Search, είναι εμφανές ότι οι δύο αλγόριθμοι διερευνούν το ίδιο σύνολο κόμβων, ωστόσο, η ευρετική συνάρτηση αλλάζει τη σειρά με την οποία επισκέπτεται τις τοποθεσίες ο A^* , γεγονός το οποίο καθιστά πιθανότερη την εύρεση της βέλτιστης διαδρομής.

Ο A^* το επιτυγχάνει αυτό βασιζόμενος στην παραπάνω περίπτωση σε μία θεμελιώδη λογική. Εάν υπάρχει ένα μονοπάτι μήκους x κόμβων μεταξύ αφετηρίας και στόχου, και η ελάχιστη απόσταση μεταξύ ενός κόμβου και του στόχου είναι μεγαλύτερη του x , ο κόμβος δεν είναι απαραίτητο να εξεταστεί ως πιθανή βάση.

7. ΑΝΟΙΧΤΗ & ΚΛΕΙΣΤΗ ΛΙΣΤΑ

Υπάρχουν πολλές διαφορετικές δομές δεδομένων που μπορούν να χρησιμοποιηθούν για την υλοποίηση της ανοιχτής και της κλειστής λίστας, όπως ο πίνακας (array) και η linked list. Η επιλογή του είδους της δομής πρέπει να εξαρτάται άμεσα από τις ενέργειες που χρειάζεται να εκτελεστούν.

Γενικά, η εισαγωγή νέων στοιχείων και η αφαίρεση του στοιχείου υψηλότερης προτεραιότητας αποτελούν τυπικές διεργασίες μίας λίστας προτεραιότητας. Υπάρχουν τρεις κύριες λειτουργίες που επιτελούνται στην ανοιχτή λίστα:

1. ο κύριος βρόγχος του αλγορίθμου βρίσκει κατ' επανάληψη τον καλύτερο κόμβο και τον αφαιρεί
2. η εκάστοτε βάση θα ελέγξει εάν ένας παρακείμενός της κόμβος βρίσκεται στην ανοιχτή λίστα
3. η εκάστοτε βάση θα εισαγάγει νέους κόμβους.

Η επιλογή της δομής δεδομένων δεν εξαρτάται μόνο από το είδος των διεργασιών, αλλά και από το πόσο συχνά μία διεργασία πρέπει να εκτελεστεί. Εάν επιτρέπεται η διαγώνιος κίνηση, κάθε βάση γειτονεύει άμεσα με 8 κόμβους, ειδώλλως με 4. Σε γενικές γραμμές :

1. Το κριτήριο εισαγωγής κάποιου κόμβου στην λίστα προτεραιότητας, ελέγχεται μία φορά για κάθε εφαπτόμενο με τη βάση κόμβο (8 φορές).
2. Εισαγωγή κόμβων επιτελείται 1 φορά για κάθε παρακείμενο στη βάση κόμβο που διερευνάται.
3. Η αφαίρεση του κόμβου με την καλύτερη προτεραιότητα εκτελείται 1 φορά για κάθε βάση.

Οι περισσότεροι κόμβοι που εξετάζονται θα διερευνηθούν, εκτός και αν είναι μη προσπελάσιμοι, ή βρίσκονται στη μεθόριο του χώρου αναζήτησης. Κατά την αξιολόγηση του κόστους των διεργασιών σε αυτές τις δομές δεδομένων, θα πρέπει να εξεταστεί το μέγιστο μέγεθος της μεθορίου (F).

Επιπλέον, υπάρχει μία τέταρτη λειτουργία, η οποία χρησιμοποιείται σχετικά σπάνια, αλλά χρειάζεται να υλοποιηθεί. Αν ο κόμβος που διερευνάται υπάρχει ήδη στην ανοιχτή λίστα, και η προτεραιότητά του που προκύπτει από τη δεύτερη αυτή διερεύνηση είναι διαφορετική (είτε χαμηλότερη είτε υψηλότερη) από την προτεραιότητα που έχει ήδη στο ανοικτό σύνολο, τότε η τιμή του στην ανοιχτή λίστα θα πρέπει να αλλάξει. Αυτό συμβαίνει με την:

4. Αλλαγή προτεραιότητας: αφαίρεση του κόμβου από την ανοικτή λίστα και την εκ νέου εισαγωγή του με τη νέα προτεραιότητα.

7.1 ΔΟΜΕΣ ΥΛΟΠΟΙΗΣΗΣ ΛΙΣΤΑΣ

7.1.1 Unsorted Arrays & Linked Lists

Η απλούστερες δομές δεδομένων είναι η αταξινόμητη διάταξη στοιχείων (unsorted array) ή οι συνδεδεμένες λίστες (linked list).

Το **unsorted array** είναι μία δομή δεδομένων που αποτελείται από μία τυχαία συλλογή στοιχείων (μεταβλητών). Κάθε στοιχείο προσδιορίζεται από τουλάχιστον μία τιμή και δομικά μπορεί να παρασταθεί ως πίνακας μίας σειράς.

Οι **linked lists** είναι δομές δεδομένων που αποτελούνται από ένα σύνολο κόμβων οι οποίοι σχηματίζουν μία ακολουθία. Στην απλούστερη της μορφή η linked list συντίθεται από κόμβους που περιέχουν δεδομένα και έναν δείκτη προς τον επόμενο κόμβο της ακολουθίας.



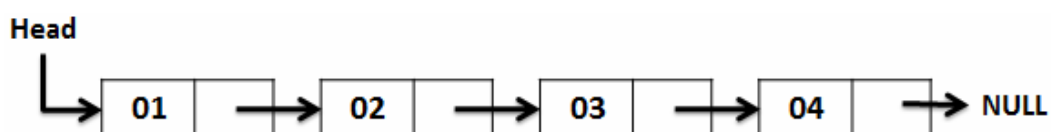
Κύριο πλεονέκτημα της linked list έναντι ενός array είναι ότι οι κόμβοι της λίστας εισάγονται ή εξάγονται εύκολα εάν ο δείκτης της ένωσης με τον προηγούμενο κόμβο διατηρηθεί κατά τη διαδικασία, πράγμα που δεν συμβαίνει στην περίπτωση των arrays. Μειονέκτημά τους είναι το ότι καταλαμβάνουν περισσότερο χώρο στη μνήμη λόγω της χρήσης δεικτών.

7.1.2 Sorted Arrays

Η ταξινομημένη διάταξη στοιχείων (sorted array) είναι μία δομή δεδομένων array, δηλαδή πίνακα μίας σειράς, στην οποία κάθε στοιχείο ταξινομείται αριθμητικά, αλφαβητικά, ή με κάποιου άλλου είδους λογική. Τα στοιχεία τοποθετούνται σε διευθύνσεις που καταναλώνουν ίδιο χώρο στη μνήμη.

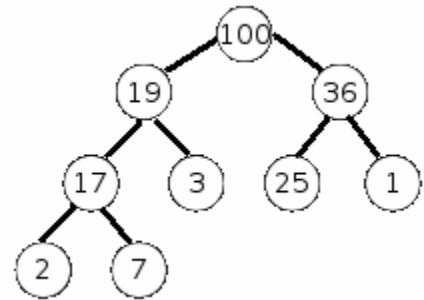
7.1.3 Sorted Linked Lists

Οι ταξινομημένες συνδεδεμένες λίστες (sorted linked lists) είναι linked lists οι οποίες διαθέτουν ένα μηχανισμό εσωτερικής ανακατάταξης των στοιχείων που αποθηκεύονται στους κόμβους τους με βάση κάποιο κριτήριο. Λόγω της ταξινόμησης, η εύρεση κάποιου στοιχείου στο πλαίσιο μίας αναζήτησης επιτυγχάνεται ταχύτερα.



7.1.4 Binary Heap

Η δυαδική στοίβα (binary heap), είναι μία δομή δεδομένων που βασίζεται στη μορφή των διακλαδώσεων, δηλαδή με αρχή έναν κόμβο, που ονομάζεται ρίζα της στοίβας, και με συνεχείς διακλαδώσεις που οδηγούν σε άλλους κόμβους, δημιουργείται μία δομή όμοια με δέντρο, όπως αυτή που εικονίζεται στο διπλανό διάγραμμα. Κάθε κόμβος διαθέτει κάποια δεδομένα και τον δείκτη που τον συνδέει με τον προηγούμενο του κόμβο, όπως συμβαίνει και στην linked list.

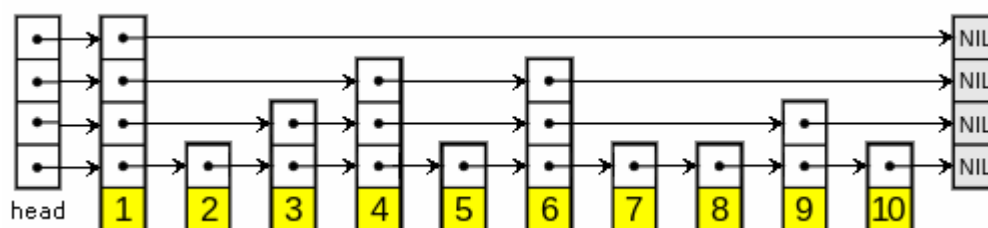


Η ιδιαιτερότητα της binary heap είναι ότι κάθε κόμβος συνδέεται με άλλους δύο στο βαθύτερο επίπεδο, τα παιδιά του, και ότι εάν δε συμπληρωθούν πλήρως όλες οι θέσεις ενός επιπέδου, δεν αρχίζει η συμπλήρωση του αμέσως βαθύτερου επιπέδου. Επιπλέον, κάθε στοιχείο ενός κόμβου είναι είτε μεγαλύτερο ίσο είτε μικρότερο ίσο από τα παιδιά του. Το αν η εξέλιξη της στοίβας είναι αύξουσα ή φθίνουσα εξαρτάται από το τι επιθυμεί ο προγραμματιστής.

Μια παραλλαγή της binary heap είναι η d-ary heap, κάθε κόμβος της οποίας έχει περισσότερα από δύο παιδιά. Στη στοίβα αυτή, η εισαγωγή στοιχείων είναι ταχύτερη, όμως η εξαγωγή περισσότερο χρονοβόρα.

7.1.5 Sorted Skip Lists

Οι sorted skip lists είναι linked lists δομές δεδομένων οι οποίες επιτρέπουν την γρήγορη αναζήτηση σε μία ταξινομημένη ακολουθία στοιχείων. Η ταχεία αναζήτηση επιτυγχάνεται από τη διατήρηση μίας διασυνδεδεμένης ιεραρχίας διάφορων υπορουτίνων, κάθε μία εκ των οποίων παραβλέπει μερικά στοιχεία.



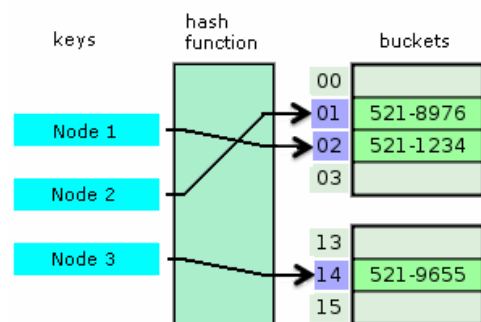
Η skip list δομείται σε στρώματα. Το χαμηλότερο στρώμα είναι μια συνήθης linked list. Κάθε ανώτερο στρώμα δρα ταχύτερα από τις λίστες που βρίσκονται στα κατώτερα του στρώματα. Η αναζήτηση ξεκινά στο κορυφαίο στρώμα με την πιο αραιή υπορουτίνα, και ολοκληρώνεται μόλις βρεθούν δύο διαδοχικά στοιχεία, ένα μικρότερο και ένα μεγαλύτερο από το αναζητούμενο στοιχείο. Μέσω της διασυνδεδεμένης ιεραρχίας, αυτά τα δύο στοιχεία συνδέονται με την επόμενη αραιότερη υπορουτίνα οπότε και η αναζήτηση συνεχίζεται με όμοιο τρόπο, ώσπου τελικά και να έλθει η σειρά της πλήρους ακολουθίας.

7.1.6 Indexed Arrays

Αν το σύνολο των κόμβων είναι πεπερασμένο και όχι υπερμέγεθες, μπορεί να χρησιμοποιηθεί μια δομή «ευρετηρίου», όπου ένας δείκτης $i(n)$ συσχετίζει κάθε κόμβο n σε ένα δείκτη ενός array. Σε αντίθεση με τα απλά και τα ταξινομημένα arrays, τα οποία έχουν μέγεθος ανάλογο με το μέγιστο μέγεθος της ανοιχτής λίστας, το μέγεθος του indexed array είναι πάντοτε $\max(i(n))$. Αν το πλέγμα είναι πυκνό, δηλαδή, δεν υπάρχουν δείκτες αχρησιμοποίητοι, τότε ο αριθμός των κόμβων του πλέγματος θα είναι $\max(i(n))$.

7.1.7 Hash Tables

Τα arrays που διαθέτουν δείκτες καταλαμβάνουν αρκετό χώρο στη μνήμη για να αποθηκεύσουν όλους τους κόμβους που δεν ανήκουν στην ανοιχτή λίστα. Εναλλακτική λύση είναι το hash table, μία δομή δεδομένων η οποία μέσω μιας συνάρτησης κατακερματισμού της $H(n)$ χαρτογραφεί κάθε κόμβο n , και τον συνδέει στη τιμή που του αντιστοιχεί. Προκειμένου να διατηρηθεί η πιθανότητα συγκρούσεων χαμηλή, το hash table πρέπει να είναι διπλάσιο από τον αριθμό των κόμβων του πλέγματος.



7.1.8 Splay Trees

Τα splay trees είναι αυτορυθμιζόμενα binary trees που έχουν την ιδιότητα να προσεγγίζουν ταχύτερα τα στοιχεία που χρησιμοποιήθηκαν πιο πρόσφατα. Αυτό επιτυγχάνεται εφαρμόζοντας μια διεργασία αναδιοργάνωσης του διαγράμματος μετά από κάθε ενέργεια που επιτελείται (εισαγωγή, εξαγωγή κλπ), ώστε το στοιχείο που χρησιμοποιήθηκε τελευταίο να τοποθετείται στη ρίζα της διακλάδωσης. Με τον τρόπο αυτό, οι κόμβοι που δεν χρησιμοποιούνται τακτικά καταλήγουν στο κατώτερο επίπεδο του διαγράμματος διακλάδωσης και δεν επιβραδύνουν τις όποιες διεργασίες.

Ωστόσο, όταν η ευρετική συνάρτηση των αλγορίθμων A^* και Dijkstra's δεν είναι ικανοποιητική υπολογίζοντας μικρότερη την απόσταση ως το στόχο, δεν εκμεταλλεύεται τις δυνατότητες του splay tree. Συγκεκριμένα, όταν $f(n') \geq f(n)$, όπου n' οι γειτονικοί κόμβοι της εκάστοτε βάσης n , το splay tree βγαίνει εκτός ισορροπίας επειδή όλες οι εισαγωγές λαμβάνουν χώρα στη μία πλευρά των διακλαδώσεων.

7.1.9 HOT Queues

Προκειμένου ο αλγόριθμος να εκμεταλλευτεί την ανισότητα $f(n') \geq f(n)$, μπορούν να χρησιμοποιηθούν οι HOT queues, οι οποίοι περιορίζουν το εύρος των τιμών που μπορούν να εισαχθούν στη λίστα προτεραιότητας. Η ταχύτητα της επιλογής βελτιώνεται με αυτόν τον αλγόριθμο.

7.1.10 Pairing Heaps

Θεωρητικά οι σωροί Fibonacci λειτουργούν ικανοποιητικά ως λίστες προτεραιότητας για τον A^* . Ωστόσο, πρακτικά δεν χρησιμοποιούνται. Ο pairing heap μπορεί να θεωρηθεί ως ένας απλοποιημένος σωρός Fibonacci.

7.2 Σύγκριση Δομών Δεδομένων

Προκειμένου να καταλήξει κάποιος στο είδος της δομής δεδομένων που θα χρησιμοποιήσει για την υλοποίηση της λίστας προτεραιότητας, θα πρέπει να συγκρίνει τις εφαρμογές ενός αλγορίθμου για όλες τις δυνατές περιπτώσεις υλοποίησης. Για να συμβεί αυτό πρέπει να είναι γνωστό το μέγεθος των δεδομένων. Συνήθως μια δομή που θεωρείται η καλύτερη για έναν συγκεκριμένο αριθμό κόμβων, δεν είναι συγκριτικά καλή με μία άλλη δομή όταν οι κόμβοι πολλαπλασιάζονται.

Σε γενικές γραμμές, καμία από τις υπάρχουσες δομές δεδομένων δεν είναι απόλυτα ικανοποιητική. Οι Unsorted arrays ή linked lists εισάγουν κόμβους πολύ γρήγορα, αλλά τα κριτήρια εισαγωγής και η εξαγωγή είναι πολύ χρονοβόρα. Οι Sorted arrays ή sorted linked lists καθιστούν τον έλεγχο των κριτηρίων εισαγωγής κάπως γρήγορα, την εξαγωγή πολύ γρήγορη, αλλά την εισαγωγή πολύ χρονοβόρα. Τα Binary heaps εισάγουν και εξάγουν κόμβους κάπως γρήγορα, αλλά τα κριτήρια εισαγωγής είναι πολύ χρονοβόρα. Τα Splay trees επιτελούν όλες τις διεργασίες κάπως γρήγορα. Οι HOT queues εισάγουν κόμβους γρήγορα, εξάγουν αρκετά γοργά, και τα κριτήρια εισαγωγής ελέγχονται κάπως γρήγορα. Οι Indexed arrays καθιστούν τον έλεγχο των κριτηρίων εισαγωγής και την εισαγωγή πολύ γρήγορα, αλλά η εξαγωγή είναι απίστευτα χρονοβόρα, και καταλαμβάνουν πολύ χώρο στη μνήμη. Τα Hash tables έχουν τις ίδιες επιδόσεις με τους indexed arrays, μα καταλαμβάνουν πολύ λιγότερη μνήμη, και οι εξαγωγή κόμβων είναι απλώς χρονοβόρα (αντί για εξαιρετικά χρονοβόρα).

Συνεπώς, για να αυξηθεί η αποδοτικότητα, ιδανικότερη θα ήταν μια υβριδική δομή δεδομένων. Ο παρακάτω πίνακας συνοψίζει την ταχύτητα επιτέλεσης διεργασιών κάθε δομής δεδομένων :

	linked lists	unsorted arrays	sorted arrays	sorted linked lists	Binary heap	Sorted Skip Lists	Indexed Arrays	Hash Tables	Splay Trees
Κριτήριο εισαγωγής	$O(F)$	$O(F)$	N/A	$O(F)$	$O(F)$	$O(\log F)$	$O(1)$	$O(1)$	N/A
Εισαγωγή (εισαγωγή-ταξινόμηση)	$O(1)$	$O(1)$	$O(F)$	$O(1)$	$O(\log F)$	$O(1)$	$O(1)$	$O(1)$	N/A
Αφαίρεση	$O(1)$	$O(F)$	$O(\log F)$	$O(1)$	$O(\log F)$	$O(1)$	$O(F)$	$O(F)$	$O(\log F)$
Αλλαγή Προτεραιότητας (εύρεση-αλλαγή)	$O(F)$ - $\Omega(1)$	$\Omega(1)$ - $O(F)$	$O(\log F)$ - $O(F)$	$O(F)$ - $O(1)$	$O(F)$ - $O(\log F)$	$O(F)$	$O(F)$ - $O(F)$	N/A	N/A
Υπολογισμός βέλτιστου	$O(F)$	$O(F)$	$O(1)$	$O(F)$	$O(F)$	$O(1)$	N/A	N/A	N/A

							$O(1)$	$O(1)$	$O(\log F)$
--	--	--	--	--	--	--	--------	--------	-------------

8. ΠΑΡΑΛΛΑΓΕΣ ΤΟΥ A*

Υπάρχουν κάποιες τεχνικές οι οποίες βελτιώνουν την αποδοτικότητα του A*. Ενδεικτικά αναφέρονται:

8.1 Dynamic weighting

Ο αλγόριθμος αυτός θεωρεί κατά την έναρξη της αναζήτησης ότι είναι επιτακτική ανάγκη η ταχεία κίνηση προς κάποια κατεύθυνση και προς το τέλος της αναζήτησης μόνο, θεωρεί σημαντικότερη την εύρεση του στόχου.

Μία νέα μεταβλητή ενσωματώνεται στην ευρετική. Πλησιέστερα προς το στόχο παίρνει μικρές τιμές, με αποτέλεσμα τη μείωση της βαρύτητας της ευρετικής.

$$f(p) = g(p) + w(p) * h(p)$$

8.2 Bandwidth search

Σε αυτή την παραλλαγή θεωρείται δεδομένο ότι η συνάρτηση H επιστρέφει υψηλότερη τιμή από την ακριβή απόσταση της διαδρομής, μα όχι περισσότερο από μία σταθερά e. Οπότε η διαδρομή που θα προκύψει δεν ξεπερνά το κόστος της βέλτιστης διαδρομής περισσότερο από e. Φυσικά όσο ακριβέστερη η ευρετική, τόσο καλύτερο το μονοπάτι.

8.3 Bidirectional search

Σε αυτή την παραλλαγή αντί για αναζήτηση της διαδρομής από την αφετηρία μέχρι το στόχο, μπορεί η αναζήτηση να εξελιχθεί παράλληλα προς δύο κατευθύνσεις, από την αφετηρία μέχρι το στόχο, και τον στόχο έως την αφετηρία. Η τελική συνάντηση αποφέρει ικανοποιητικά αποτελέσματα. Οι δύο αναζητήσεις συνδέονται ως εξής:

Αναζήτηση από Αφετηρία:	$G(\text{start}, x) + H(x, \text{end})$
Αναζήτηση από Στόχο:	$G(y, \text{end}) + H(\text{start}, y)$
Παράλληλη Αναζήτηση:	$G(\text{start}, x) + H(x, y) + G(y, \text{end})$

Η αναζήτηση αυτή μπορεί να επιτευχθεί και διαφορετικά. Μπορεί να εκτελεστεί μια αναζήτηση προς τον στόχο για ένα μικρό διάστημα, να επιλεγεί ο βέλτιστος κόμβος και στη συνέχεια να εκτελεστεί η αναζήτηση αντίστροφα, από το στόχο, μα όχι προς την αφετηρία, αλλά προς τον κόμβο αυτό που προσδιορίστηκε στο πρώτο σκέλος της αναζήτησης. Έτσι, επιλέγεται ένας δεύτερος βέλτιστος κόμβος από το στόχο προς την αφετηρία. Έπειτα εκτελείται αναζήτηση από τον νέο βέλτιστο κόμβο προς τον παλιό. Η διαδικασία συνεχίζεται έως ότου οι δύο κόμβοι, μπροστά και πίσω να γίνουν ένας.

8.4 Dynamic A* and Lifelong Planning A*

Υπάρχουν παραλλαγές του A* που επιτρέπουν την πραγματοποίηση αλλαγών στον χώρο μετά τον εντοπισμό της αρχικής διαδρομής. Όταν οι απαραίτητες πληροφορίες για το περιβάλλον, όπως η αφετηρία ή ο στόχος, απουσιάζουν, είναι προτιμότερη η χρήση του D* ή του LPA*. Ο D* εστιάζει στη διόρθωση των λαθών που μπορεί να κάνει ο A* λόγω αυτής της έλλειψης βασικών πληροφοριών, και μάλιστα ταχύτατα.

Ο LPA* χρησιμοποιείται όταν τα κόστη κίνησης μεταβάλλονται. Στην περίπτωση του μεταβλητού περιβάλλοντος, η διαδρομή που εντοπίζει ο A*, μπορεί να ακυρωθεί από το ίδιο το περιβάλλον. Ο LPA* χρησιμοποιεί τους ενυπάρχοντες υπολογισμούς του A* για να εντοπίσει μια νέα πορεία.

Ωστόσο, τόσο ο D* όσο και ο LPA* καταλαμβάνουν πολύ χώρο στη μνήμη, ιδιαίτερα όταν εκτελείται ο A* και διατηρεί τις εσωτερικές πληροφορίες (ανοικτή/κλειστή λίστα, κόστη κίνησης). Οι πληροφορίες αυτές χρησιμοποιούνται όταν μεταβάλλεται ο χάρτης, για να αποφασίσουν (οι D* και LPA*) εάν χρειάζεται να ρυθμιστεί εκ νέου η πορεία προκειμένου να παραμείνει η πορεία βέλτιστη.

Στην περίπτωση των πολλαπλών κινούμενων μονάδων οι D* και LPA* είναι μη εφαρμόσιμοι, καθώς το μέγεθος των πληροφοριών που πρέπει να διατηρηθεί είναι πολύ μεγάλο. Ενδείκνυνται όμως, για ρομποτικές εφαρμογές μίας κινούμενης μονάδας σε μεταβλητό περιβάλλον.

8.5 Jump Point Search

Προκειμένου να βελτιωθεί η ταχύτητα του A*, πολλές τεχνικές εστιάζουν στη μείωση του αριθμού των κόμβων. Σε τετραγωνικό πλέγμα με ομοιόμορφο κόστος κίνησης προς όλες τις κατευθύνσεις μπορεί να οικοδομηθεί ένα γράφημα πιθανών σημείων επισκεψιμότητας, για παράδειγμα κορυφών, προκειμένου να εκτελεστεί ο αλγόριθμος αναζήτησης. Ωστόσο, ο προϋπολογισμός ενός γραφήματος μέσω σημείων πλοήγησης είναι μη επιθυμητός, καθώς παραλείπει αρκετούς κόμβους από τον υπολογισμό στο τετραγωνικό πλέγμα. Κατά την διερεύνηση της εφαπτόμενης περιοχής του τρέχοντος κόμβου για πιθανή εισαγωγή στην ανοικτή λίστα, ο Jump Point Search μεταβαίνει σε μακρινούς κόμβους, ορατούς από τον τρέχοντα κόμβο. Το κόστος κάθε βήματος είναι υψηλότερο, αλλά υπάρχουν λιγότεροι κόμβοι συνολικά στο πλέγμα και επομένως και στην ανοικτή λίστα.

8.6 Theta*

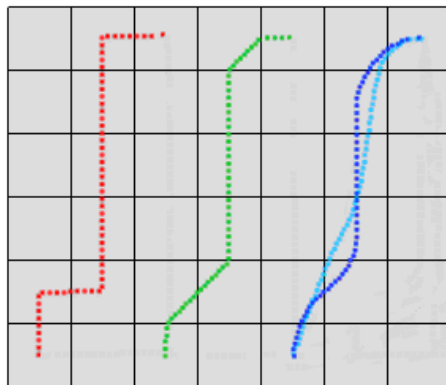
Ο Theta*, είναι μια παραλλαγή του A* η οποία εκτελείται σε τετραγωνικό πλέγμα, και η διαδρομή που προσδιορίζει δεν ακολουθεί αυστηρά το πλέγμα, αλλά, εξομαλύνει τη διαδρομή που έχει εντοπίζει ο A* ενώ ο καθορισμός του μονοπατιού βρίσκεται σε εξέλιξη. Αυτό επιτυγχάνεται με τη χρήση ενός δείκτη για κάθε βάση, ο οποίος παραλείπει τους

ενδιάμεσους κόμβους που παρεμβάλλονται, έως τον πρώτο κόμβο που βρίσκεται σε οπτική επαφή μαζί του.

Με αυτόν τον τρόπο, τα μονοπάτια που ανακαλύπτονται, είναι συντομότερα και το πλέγμα μετατρέπεται από πλέγμα κίνησης με βάση τις βασικές κατευθύνσεις, σε πλέγμα όπου η κίνηση γίνεται με βάση την ιδανική γωνία.

9. ΚΙΝΗΣΗ

Το πρόβλημα της κίνησης αφορά την μετακίνηση της κινούμενης μονάδας από μία περιοχή σε μία άλλη. Στο ακόλουθο σχήμα εικονίζονται οι πιθανές μέθοδοι μετακίνησης μίας μονάδας.

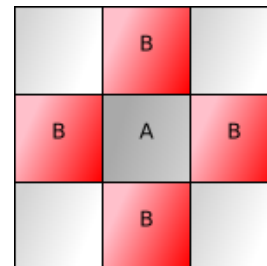


Η κόκκινη διαδρομή αποτελεί τη συνήθη προσέγγιση και αφορά την κίνηση από το κέντρο ενός κόμβου, στο κέντρο του επόμενου. Η πράσινη διαδρομή, ελαφρώς ομαλότερη, αφορά την ευθεία κίνηση, από το κέντρο μίας ακμής στο κέντρο της άλλης. Για κίνηση σε ευθεία γραμμή μπορούν να χρησιμοποιηθούν και οι γειτονικές κορυφές. Τέλος, το μπλε μονοπάτι, ενώνει διαφορετικής ιεραρχικής τάξης κόμβους (με γαλάζιο συμβολίζονται τα υψηλής ιεραρχικής τάξης τόξα κίνησης, ενώ με μπλε τα χαμηλής).

Σε γενικές γραμμές, όταν το κόστος κίνησης είναι διαφορετικό σε κάθε βήμα, η χρήση πλέγματος συνίσταται (δηλαδή όταν η διαμόρφωση του εδάφους λαμβάνεται υπόψη). Εάν το κόστος κίνησης είναι ομοιόμορφο σε όλες τις περιοχές του χώρου, η χρήση πλέγματος αποδεικνύεται ασύμφορη. Δεν υπάρχει λόγος η μονάδα κίνησης να μετακινείται ένα βήμα τη φορά, όταν μπορεί απλώς να διανύσει μονομιάς μία μεγάλη περιοχή.

9.1. Κίνηση Μεταξύ Επιφανειών

Η κίνηση από επιφάνεια σε επιφάνεια, είναι η κυριότερη από τις διαθέσιμες επιλογές, ιδιαίτερα όταν η μονάδα κίνησης διέρχεται από το κέντρο της επιφάνειας του κόμβου. Στο ακόλουθο σχήμα, η μονάδα κίνησης τοποθετημένη αρχικά στην επιφάνεια Α, μπορεί να κινηθεί σε οποιαδήποτε από τις επιφάνειες που σημειώνονται με Β. Επίσης είναι εφικτή η διαγώνιος κίνηση, όμως, με υψηλότερο κόστος μετακίνησης.



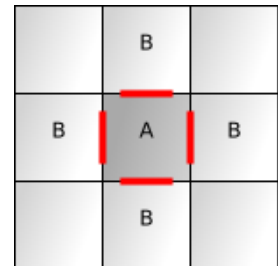
Στην περίπτωση όπου ο χώρος αναπαρίσταται με πλέγμα, η μονάδα κίνησης δεν δεσμεύεται στο πλέγμα, και το κόστος μετακίνησης είναι ισότιμο, είναι προτιμότερο το να ευθυγραμμιστούν οι πιθανές διαδρομές ακολουθώντας κίνηση σε ευθεία γραμμή από έναν κόμβο σε κάποιον

μακρύτερο κόμβο, με την προϋπόθεση ότι δεν υπάρχουν εμπόδια μεταξύ αυτών των δύο.

9.1.1. Κίνηση σε Ακμές

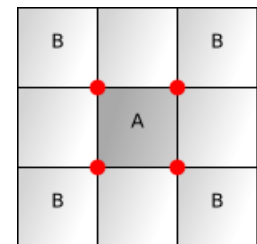
Στην περίπτωση που η μονάδα κίνησης μπορεί να κατευθυνθεί προς οποιοδήποτε σημείο του χώρου, ή η επιφάνεια των κόμβων είναι μεγάλη, τότε είναι πολύ πιθανόν οι ακμές και οι κορυφές να είναι προτιμότερη επιλογή για τον σχεδιασμό της κίνησης μεταξύ των κόμβων.

Η μονάδα εισέρχεται συνήθως στο κέντρο μίας επιφάνειας διαμέσου μιας ακμής και εξέρχεται από την επιφάνεια αυτή διαμέσου της απέναντι ακμής. Εάν το Path Finder σχεδιαστεί για επιφάνειες, η μονάδα κινείται από κέντρο προς κέντρο επιφανειών, όμως, με Path Finder σχεδιασμένο για κίνηση σε ακμές, η μονάδα κινείται απευθείας από τη μία ακμή του κόμβου προς την άλλη.



9.1.2. Κίνηση σε Κορυφές

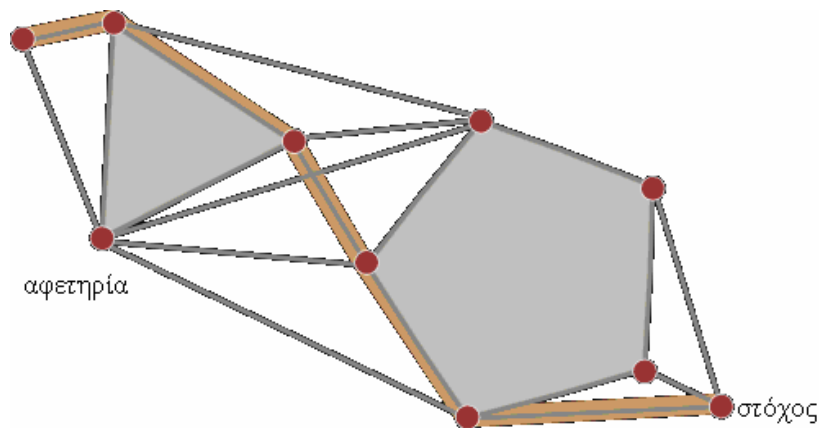
Ακόμη και τα εμπόδια σε ένα πλέγμα διαθέτουν μέρη όπως οι γωνίες στις κορυφές. Η συντομότερη μέθοδος παράκαμψης ενός εμποδίου είναι η περιστροφή γύρω από τη γωνία. Με Path Finder σχεδιασμένο για κίνηση στις κορυφές, η μονάδα κινείται από κορυφή σε κορυφή. Με τον τρόπο αυτό η κίνηση της μονάδας είναι βέλτιστη, μα θα πρέπει η προσδιοριζόμενη διαδρομή να μπορεί να λαμβάνει υπόψη της το μέγεθος της κινούμενης μονάδας ώστε να μην προσκρούει στα εμπόδια.



9.1.3. Πολυγωνικοί Χάρτες

Ίσως η κοινότερη εναλλακτική λύση στα πλέγματα, είναι η χρήση πολυγωνικής αναπαράστασης. Εάν το κόστος μετακίνησης σε μεγάλες περιοχές είναι ομοιόμορφο, και οι μονάδες έχουν τη δυνατότητα να κινηθούν στην ευθεία, είναι προτιμότερη η αποφυγή της αναπαράστασης του χώρου ως πλέγμα.

Ακολουθεί αναπαράσταση του χώρου με πολυγωνικό χάρτη όπου τοποθετούνται δύο εμπόδια. Η κινούμενη μονάδα θα πρέπει να τα παρακάμψει. Το βέλτιστο μονοπάτι σημειώνεται με πορτοκαλί χρώμα και τα εμπόδια με γκρίζο:



Για να σχεδιαστεί η διαδρομή, ο αλγόριθμος κίνησης θα πρέπει να διαθέτει κάποιες πληροφορίες. Πρώτον, η βέλτιστη διαδρομή που θα μπορούσε να ακολουθήσει η μονάδα κίνησης σε αυτό το χάρτη, ορίζεται από τις κορυφές των εμποδίων. Οπότε οι κόκκινες κορυφές αποτελούν σημεία πλοήγησης.

Εκτός από τα σημεία πλοήγησης, το Path Finder θα πρέπει να γνωρίζει ποια σημεία συνδέονται μεταξύ τους. Ο απλούστερος αλγόριθμος οικοδομεί ένα γράφημα ορατότητας, δηλαδή ένα γράφημα θέσεων στον Ευκλείδειο χώρο όπου κάθε ακμή του κόμβου συμβολίζει μια ορατή διασύνδεση ανάμεσα στα σημεία πλοήγησης. Εάν ο χάρτης δεν μεταβάλλεται, αυτή η μέθοδος πλοήγησης είναι ικανοποιητική, εκτός ίσως κάποιων περιπτώσεων όπου αποδεικνύεται κάπως αργή.

Ακόμη, το Path Finder χρειάζεται να γνωρίζει τους χρόνους μετάβασης μεταξύ των σημείων πλοήγησης. Αυτοί οι χρόνοι είναι η απόσταση Manhattan ή η απόσταση diagonal εάν ο χώρος αναπαρίσταται ως πλέγμα. Αν η μονάδα χρησιμοποιεί σημεία πλοήγησης, η ευθεία απόσταση υπολογίζει τον χρόνο.

Η κίνηση μεταξύ σημείων πλοήγησης είναι πολύ ταχύτερη από την αναζήτηση μονοπατιών ανάμεσα σε σημεία του πλέγματος, κυρίως επειδή τα σημεία πλοήγησης είναι αριθμητικά λιγότερα. Ιδίως σε περιοχές χωρίς εμπόδια, ο αλγόριθμος θα αποδίδει βέλτιστα καθώς η αφετηρία και ο στόχος θα συνδέονται με μια ακμή και η λύση θα εντοπίζεται ακαριαία.

9.2. Σημεία Πλοήγησης

Το σημείο πλοήγησης αποτελεί τμήμα της διαδρομής. Σκοπός ενός τέτοιου σημείου, είναι η παράβλεψη κόμβων στο χάρτη, προκειμένου να υπολογιστεί ταχύτερα και με περισσότερο εξομαλυσμένη μορφή μια διαδρομή. Η skip link ενδείκνυται για την διαχείριση αυτών των σημείων.

Εάν τα κόστη κίνησης που υπολογίζονται είναι ανακριβή, χρήση των σημείων πλοήγησης έχει σαν αποτέλεσμα τον εντοπισμό όχι και τόσο βέλτιστων μονοπατιών. Ακόμη, είναι δυνατή η εξομάλυνση μη βέλτιστων μονοπατιών, στον αλγόριθμο movement.

9.3. *Skip Link*

Η skip link, είναι η ακμή που ενώνει δύο σημεία πλοήγησης και δρα ως συντόμευση στην διεργασία αναζήτησης της βέλτιστης διαδρομής. Η πρόσθεση της στον αλγόριθμο, αποτελεί μία προσέγγιση του ιεραρχικού χάρτη, δηλαδή του χάρτη στον οποίο κάθε κόμβος έχει τη δική του αξία.

9.4. *Εξομάλυνση Μονοπατιού*

Η εξομάλυνση ενός μονοπατιού είναι σχετικά εύκολη στις ήδη εντοπισμένες διαδρομές, υπό την προϋπόθεση ότι τα κόστη κίνησης είναι σταθερά. Η λογική του αλγορίθμου είναι η εξής:

- a. Εάν υπάρχει οπτική επαφή από το σημείο πλοήγησης n στο σημείο πλοήγησης $n+2$, παραλείπεται το σημείο $n+1$. Επανάληψη της διαδικασίας έως ότου να μην βρίσκεται σύνδεση μεταξύ δύο σημείων της διαδρομής.
- b. Αφού εφαρμοστεί η παραπάνω διαλογή, απομένουν μόνο οι κόμβοι που παρακάμπτουν τις γωνίες των εμποδίων.

Με χρήση της εξομάλυνσης στην διαδικασία αναζήτησης, είναι απαραίτητες μόνο οι κορυφές και όχι οι ακμές ή οι επιφάνειες, για χρήση ως σημεία πλοήγησης. Ωστόσο είναι ο αλγόριθμος που εντοπίζει το βέλτιστο μονοπάτι. Η εξομάλυνση απλά καθιστά την τροχιά της κινούμενης μονάδας ομαλότερη και είναι συμπληρωματική.

10. ΚΙΝΟΥΜΕΝΑ ΕΜΠΟΔΙΑ

Ένα Path Finding υπολογίζει μια διαδρομή παρακάμπτοντας τα στατικά εμπόδια. Θα μπορούσαν όμως τα εμπόδια να μετακινηθούν, δηλαδή, όταν η κινούμενη μονάδα προσεγγίσει κάποιο σημείο είτε το εμπόδιο που θα έπρεπε να υπάρχει εκεί να μην υπάρχει πια, είτε το αντίθετο, ένα νέο εμπόδιο να εμφανιστεί.

Στην περίπτωση αυτή θα πρέπει να εφαρμοστεί ένας ξεχωριστός αλγόριθμος αποφυγής εμποδίων, **μαζί** με το Path Finder. Το Path Finder εντοπίζει τη βέλτιστη διαδρομή, και ενώ η κινούμενη μονάδα την ακολουθεί, θα παρακάμπτει τα εμπόδια. Αν όμως τα εμπόδια μεταβάλλονται σημαντικά, είναι καλύτερο να εφαρμοστεί Path Finder για αποφυγή εμποδίων.

10.1. Επανεκτίμηση Διαδρομής

Με την πάροδο του χρόνου, το φυσικό περιβάλλον μπορεί να μεταβάλλεται. Προκειμένου να προσδιοριστεί πότε χρειάζεται να υπολογιστεί ξανά μία διαδρομή χρησιμοποιούνται τα εξής κριτήρια:

- Όποτε διανυθούν N βήματα: αυτή η αντιστοίχιση του χρόνου με την απόσταση, εγγυάται ότι οι πληροφορίες που χρησιμοποιούνται για τον υπολογισμό της διαδρομής δεν υπερβαίνουν χρονικά τα N βήματα.
- Όποτε η CPU είναι διαθέσιμη: επιτρέπει τη δυναμική προσαρμογή της ποιότητας του μονοπατιού
- Κάθε φορά που η μονάδα περιστρέφεται γύρω από μια γωνία ή διέρχεται από ένα σημείο πλοήγησης.
- Όποτε ο περιβάλλοντας χώρος στην περιοχή γύρω από τη μονάδα κίνησης μεταβάλλεται.

Κύριο μειονέκτημα της επανεκτίμησης της διαδρομής είναι ότι παραβλέπονται πολλές από τις πληροφορίες της διαδρομής. Εάν για παράδειγμα, η διαδρομή έχει μάκρος 100 βήματα και υπολογίζεται εκ νέου κάθε 10 βήματα, ο συνολικός αριθμός των βημάτων διαδρομής θα είναι $100 + 90 + 80 + 70 + 60 + 50 + 40 + 30 + 20 + 10 = 550$. Εάν η διαδρομή έχει M βήματα, υπολογίζονται M^2 βήματα καθώς ο χρόνος κυλά. Συνεπώς, στις μακρινές διαδρομές η επανεκτίμηση δεν είναι αποδοτική. Η επαναχρησιμοποίηση των πληροφοριών της διαδρομής είναι αποδοτικότερη.

10.2. Συναρμογή Διαδρομών

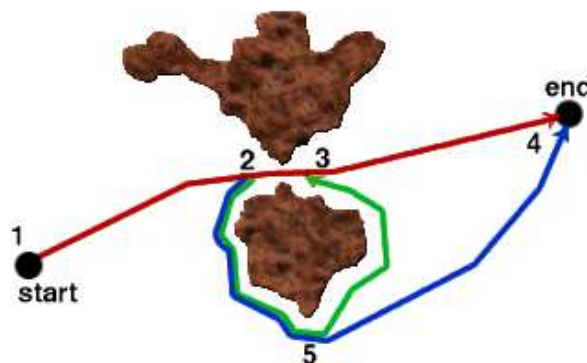
Όταν το περιβάλλον μεταβάλλεται, είναι απαραίτητος ο εκ νέου υπολογισμός της διαδρομής. Λογικά, τα σημεία του χάρτη που βρίσκονται σε κοντινή απόσταση είναι πιο γνωστά από αυτά στις μακρινές περιοχές. Μία μέθοδος αντιμετώπισης του προβλήματος

βασίζεται στην τοπική διόρθωση της διαδρομής: αναπτύσσει μία βέλτιστη διαδρομή στην κοντινή περιοχή, και δεν ασχολείται με τη διαδρομή σε μακρινή απόσταση, έως ότου να προσεγγίσει την περιοχή εκείνη. Οπότε με τον υπολογισμό των M πρώτων βημάτων, αποφεύγεται ο υπολογισμός εκ νέου ολόκληρης τη διαδρομής, ως εξής:

1. Έστω $p[1] \dots p[N]$ είναι το υπόλοιπο της διαδρομής (όπου: N τα βήματα)
2. Υπολογισμός ενός μονοπατιού από το σημείο $p[1]$ έως το $p[M]$
3. Ένωση του μονοπατιού αυτού με το παλαιό αντικαθιστώντας τα σημεία $p[1]$ έως $p[M]$ και εισάγοντας το νέο μονοπάτι στη θέση τους

Εφόσον τα $p[1]$ και $p[M]$ απέχουν περισσότερα από M βήματα, είναι απίθανο ότι η νέα διαδρομή θα είναι μεγάλη. Παρόλα αυτά, μπορεί να προκύψει αυτή η περίπτωση οπότε και η νέα διαδρομή θα είναι μεγάλη και όχι και πολύ καλή.

Το ακόλουθο σχήμα αναπαριστά αυτή την περίπτωση. Το αρχικό κόκκινο μονοπάτι διέρχεται από τις θέσεις 1-2-3-4. Τα εμπόδια σημειώνονται με καφέ χρώμα. Αν η μονάδα προσεγγίσει το σημείο 2, και ανακαλύψει ότι η διαδρομή 2-3 είναι αποκλεισμένη, με συναρμογή μονοπατιών αντικαθίσταται η απόσταση μεταξύ των σημείων 2-3 με την πράσινη διαδρομή 2-3-5, με αποτέλεσμα η μονάδα να κινείται κατά μήκος της διαδρομής 1-2-5-3-4. Είναι προφανές ότι η διαδρομή αυτή δεν είναι και πολύ καλή. Η μπλε διαδρομή 1-2-5-4 είναι καλύτερη.



Τα αν θα προκύψει μη βέλτιστο μονοπάτι μπορεί κανείς να το συμπεράνει μελετώντας το μήκος της νέας διαδρομής. Εάν αυτή είναι σημαντικά μεγαλύτερη από M , θα μπορούσε να μην είναι βέλτιστη. Αυτή η πληροφορία υλοποιείται στον αλγόριθμο με την προσθήκη ενός ορίου μέγιστου μήκους διαδρομής στο Path Finder. Εάν δεν βρεθεί βέλτιστη διαδρομή, ο αλγόριθμος επιστρέφει σφάλμα, οπότε και μπορεί να ξανά-αναζητηθεί η διαδρομή αντί να συναρμολογηθούν δύο διαδρομές εκ των οποίων θα προκύψει ένα μη βέλτιστο μονοπάτι όπως το 1-2-5-4.

Σε μια διαδρομή που συνίσταται από N βήματα, η συναρμογή θα υπολογίσει από $2N$ έως $3N$ βήματα, ανάλογα με το πόσο συχνά μια νέα διαδρομή συναρμολογείται στο μονοπάτι.

Αυτό το κόστος αντιπροσωπεύει την ικανότητα να ανταπεξέλθει σε ένα μεταβλητό περιβάλλον, και είναι αρκετά χαμηλό.

Παραδόξως, το κόστος είναι ανεξάρτητο του αριθμού των βημάτων συναρμογής (M). Οπότε αν το M είναι υψηλό, η κίνηση της μονάδας δεν θα ανταποκριθεί γρήγορα σε αλλαγές στο χάρτη. Εάν το M είναι πολύ χαμηλό, τα μονοπάτια που συναρμολογούνται μπορεί να είναι πολύ μικρά για να μπορέσει η νέα πορεία να παρακάμψει το εμπόδιο. Το πιο πιθανό είναι να βρεθούν όχι και τόσο βέλτιστες διαδρομές (όπως η 1-2-5-3-4 στο παραπάνω παράδειγμα). Το M θα πρέπει να προσαρμόζεται σε κάθε χάρτη εκ νέου.

Η συναρμογή διαδρομών είναι πολύ ταχύτερη από τον επαναυπολογισμό ολόκληρης της διαδρομής, αλλά δεν αποδίδει ικανοποιητικά σε μεγάλες αλλαγές της πορείας. Ωστόσο, είναι δυνατή η ανίχνευση των περιπτώσεων που θα συμβεί αυτό, ώστε να χρησιμοποιηθεί η εκ νέου αναζήτηση στη περίπτωση εκείνη. Επίσης, η τεχνική αυτή δεν μπορεί να χειριστεί τις περιπτώσεις όπου απαιτείται συντονισμός πολλαπλών μονάδων κίνησης, προκειμένου να υπάρξει προσπέραση.

10.3. **Επαγρύπνηση για Αλλαγές στο Χάρτη**

Εναλλακτική λύση έναντι του εκ νέου υπολογισμού του συνόλου ή μέρους της διαδρομής, είναι η αναμονή και παρακολούθηση του χάρτη, περιμένοντας την επερχόμενη αλλαγή, ώστε μόλις συμβεί αυτή να επανεκτιμηθεί η διαδρομή.

Ο χάρτης χωρίζεται σε περιοχές, και κάθε μονάδα κίνησης -εάν υπάρχουν περισσότερες από μία μονάδες- εκδηλώνει ενδιαφέρον για ορισμένες περιοχές. Για μια μονάδα παρουσιάζουν ενδιαφέρον όσες περιοχές περιέχουν τμήμα της διαδρομής, ή μόνο οι κοντινές της περιοχές. Κάθε φορά που ένα αντικείμενο εισέρχεται ή εξέρχεται από μία περιοχή ενδιαφέροντος, η περιοχή χαρακτηρίζεται ως **αλλαγμένη**, και αυτό γνωστοποιείται σε όσες μονάδες ενδιαφέρονται για την περιοχή αυτή, έτσι ώστε το βέλτιστο μονοπάτι να υπολογιστεί εκ νέου.

Η επαγρύπνηση για αλλαγές στο χάρτη επιτρέπει στις μονάδες να αποφύγουν τον επαναυπολογισμό της διαδρομής όταν τα εμπόδια παραμένουν σταθερά, οπότε συνιστάται ως μέθοδος, στις περιπτώσεις στις οποίες υπάρχουν πολλές περιοχές που δεν μεταβάλλονται συχνά.

10.4. **Πρόβλεψη Κίνησης Εμποδίων**

Εάν μπορεί να προβλεφθεί η κίνηση των εμποδίων, το Path Finder μπορεί να υπολογίσει τη μελλοντική θέση τους. Αλγόριθμοι, όπως ο A^* , που διαθέτει την ευρετική συνάρτηση η οποία καθορίζει τη δυσκολία διέλευσης από κάθε σημείο του χάρτη, μπορούν να τροποποιηθούν για να παρακολουθούν το χρόνο που απαιτείται για να προσεγγίσει η μονάδα κίνησης ένα σημείο (στο πλαίσιο της πρώτης υπολογισμένης βέλτιστης διαδρομής). Η ευρετική συνάρτηση μπορεί, μέσω αυτής της τροποποίησης, να

χρησιμοποιήσει την προβλεπόμενη θέση του εμποδίου εκείνη τη χρονική στιγμή για να καθορίσει αν η περιοχή του χάρτη είναι προσβάσιμη ή όχι.

Ωστόσο, αυτή η τροποποίηση, δε θα λάβει υπόψη την πιθανότητα η κινούμενη μονάδα να χρειαστεί να σταθμεύσει σε ένα σημείο περιμένοντας το εμπόδιο να μετακινηθεί, και ο A^* δεν έχει σχεδιαστεί για να διακρίνει διαδρομές κατά μήκος της ίδιας πορείας, αλλά να διακρίνει διαδρομές σε διαφορετικές χρονικές στιγμές.

11. ΧΩΡΟΣ ΣΤΗ ΜΝΗΜΗ

Η απόδοση του αλγορίθμου Path Finder, δεν εξαρτάται μόνο από το χρόνο που απαιτείται για να υπολογιστεί η διαδρομή, αλλά και από το χώρο που χρησιμοποιείται για να αποθηκευτεί η διαδρομή. Σε γενικές γραμμές, ο προσωρινός χώρος που απαιτείται για την εκτέλεση του αλγορίθμου είναι μεγαλύτερος από το χώρο που απαιτείται για την αποθήκευση της διαδρομής που θα προκύψει.

Για να ελαχιστοποιηθεί ο χώρος που απαιτείται επιβάλλεται ο υπολογισμός μίας μόνο βέλτιστης διαδρομής τη φορά. Επιπλέον, ο προσωρινός χώρος μνήμης που καταλαμβάνεται, εξαρτάται και από την δομή δεδομένων που υλοποιεί την ανοιχτή και την κλειστή λίστα και αναλύεται στο κεφάλαιο της υλοποίησης.

11.1. Θέσεις & Κατευθύνσεις

Μια διαδρομή μπορεί να συνίσταται είτε από θέσεις είτε από κατευθύνσεις. Οι θέσεις καταλαμβάνουν περισσότερο χώρο στη μνήμη, αλλά είναι ευκολότερο να προσδιοριστεί η κατεύθυνση της διαδρομής χωρίς να διανυθεί το μονοπάτι. Σε ένα τυπικό πλέγμα, οι θέσεις μπορούν να αποθηκευτούν με δύο 16-bitους ακέραιους, με κάθε βήμα να καταλαμβάνει 32-bit.

Όταν αποθηκεύονται κατευθύνσεις, η τοποθεσία μπορεί να προσδιοριστεί μόνο μετά την ολοκλήρωση της μετάβασης στον στόχο. Οι κατευθύνσεις καταλαμβάνουν λιγότερο χώρο επειδή είναι πολύ λιγότερες. Εάν η κινούμενη μονάδα μπορεί να κινηθεί σε τέσσερις κατευθύνσεις, κάθε βήμα καταλαμβάνει 2 bits, ενώ αν μπορεί να κινηθεί σε έξι ή οκτώ κατευθύνσεις, κάθε βήμα καταλαμβάνει 3 bits. Η εξοικονόμηση χώρου λοιπόν, είναι μεγάλη.

Επιπλέον, ο χώρος που καταλαμβάνεται στη μνήμη μπορεί να μειωθεί ακόμη περισσότερο, όταν αποθηκεύονται οι σχετικές κατευθύνσεις (υπολογιζόμενες σε μοίρες) και όχι οι απόλυτες (σημεία ορίζοντα). Άλλωστε σε ορισμένους τύπους μονάδων κίνησης, κάποιες εντολές δεν είναι λογικές. Για παράδειγμα, αν η μονάδα κινείται βόρεια, είναι απίθανο το επόμενο βήμα να κατευθύνεται νότια. Δηλαδή, εάν το πλέγμα είναι εξαγωνικό, οι ουσιαστικές κατευθύνσεις είναι μόνο πέντε, εφόσον είναι απίθανο η μονάδα να στραφεί ξαφνικά προς τα πίσω.

11.2. Συμπίεση Διαδρομής

Άπαξ και προσδιοριστεί η διαδρομή, μπορεί έπειτα να υποστεί συμπίεση. Ένας αλγόριθμος συμπίεσης σχεδιασμένος για τροχιές θα μπορούσε να χρησιμοποιηθεί για να συντομεύσει διαδρομές υλοποιημένες είτε με θέσεις είτε με κατευθύνσεις.

11.2.1. Συμπίεση Θέσης

Σε χάρτες όπου τα εμπόδια και όχι η διαμόρφωση του εδάφους καθορίζουν τη μορφή της διαδρομής, μπορεί να υπάρχουν πολλά ευθύγραμμα τμήματα στο τελικό μονοπάτι. Σε μονοπάτια αυτής της μορφής, θα πρέπει να περιέχεται μόνο το πέρας των ευθύγραμμων τμημάτων ως πληροφορία σχηματισμού της διαδρομής, καθώς η κίνηση βασίζεται στην εξέταση του επόμενου σημείου και την προσέγγισή του ευθύγραμμο στη συνέχεια.

11.2.2. Συμπίεση Κατεύθυνσης

Στην περίπτωση των κατευθύνσεων, μπορεί μια συγκεκριμένη κατεύθυνση να επιλέγεται επαναλαμβανόμενα.

Για να αποθηκευτεί ένα μονοπάτι, μπορεί να αποθηκευτεί η κατεύθυνση της κίνησης μαζί με έναν αριθμό που υποδηλώνει πόσοι είναι οι βηματισμοί που πρέπει να γίνουν προς τη συγκεκριμένη κατεύθυνση. Ωστόσο, αν μια κατεύθυνση δεν ακολουθείται συνεχόμενα και αρκετές φορές, ενδέχεται να προκύψουν προβλήματα. Επίσης, για πολλαπλά ευθύγραμμα τμήματα όπου συνιστάται η συμπίεση τοποθεσίας, η συμπίεση κατεύθυνσης δεν ενδείκνυται αν το ευθύγραμμο τμήμα δεν είναι παράλληλο με μία από τις κατευθύνσεις κίνησης.

Πρακτικά, για την αποθήκευση του μονοπατιού μπορεί να χρησιμοποιηθεί μεταβλητού μήκους κωδικοποίηση. Αντιστοιχώντας ένα bit(0) στο θεμελιώδες βήμα, την ευθύγραμμη κίνηση, και ένα bit(1) στη στροφή, μπορεί να αναπαρασταθεί η διαδρομή. Σε πλέγμα τεσσάρων κατευθύνσεων, επιτρέπονται μόνο δύο στροφές, αριστερή και δεξιά. Μπορεί να αντιστοιχηθεί 10 για αριστερόστροφη κίνηση και 11 για δεξιόστροφη. Η μεταβλητού μήκους κωδικοποίηση μπορεί να συμπίεσει καλύτερα από ό, τι να εκτελέσει μακρά κωδικοποίηση για μικτές τροχιές, αλλά όχι τόσο αποτελεσματικά όσο για μακρά ευθύγραμμο μονοπάτια.

11.3. Περιορισμός Μήκους Διαδρομής

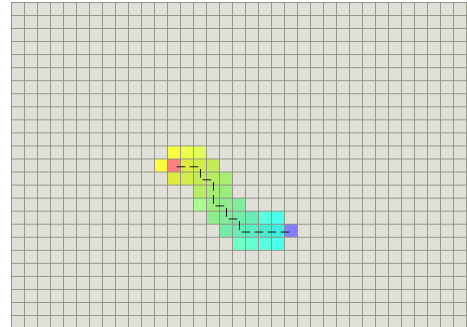
Δεδομένου ότι το περιβάλλον είναι μεταβλητό, η αποθήκευση μεγάλων διαδρομών δεν έχει καμία χρησιμότητα. Η μονάδα κίνησης μπορεί να αποθηκεύσει ένα συγκεκριμένο αριθμό βημάτων κατά την έναρξη της κίνησης, και στη συνέχεια να επανεκτιμήσει τη διαδρομή όταν έχει διανυθεί η αρχική απόσταση. Έτσι μειώνεται η ποσότητα των δεδομένων που αποθηκεύονται.

Τα μονοπάτια μπορούν δυνητικά να καταλαμβάνουν πολύ χώρο μνήμης, ειδικά όταν οι τροχιές είναι μακριές και υπάρχουν πολλές μονάδες κίνησης. Μπορεί να ελαττωθεί ο καταλαμβανόμενος χώρος, αποθηκεύοντας πολλαπλά βήματα σε μικρή ποσότητα δεδομένων.

12. ΣΥΝΟΨΗ

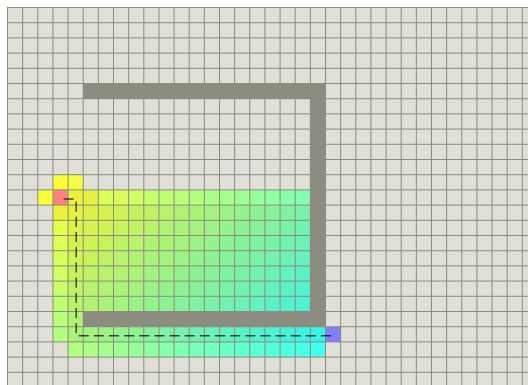
Ο A^* είναι ο πιο δημοφιλής αλγόριθμος Pathing, επειδή είναι αρκετά ευέλικτος και μπορεί να χρησιμοποιηθεί σε ένα ευρύ φάσμα περιπτώσεων.

Η ομοιότητα του A^* με τον αλγόριθμο του Dijkstra είναι το ότι μπορεί να χρησιμοποιηθεί για την εύρεση της βέλτιστης διαδρομής. Η ομοιότητα του A^* με τον αλγόριθμο Greedy Best-First-Search έγκειται στο ότι χρησιμοποιεί ευρετική συνάρτηση για να τον καθοδηγεί. Στην απλούστερη περίπτωση, είναι τόσο ταχύς όσο και ο Greedy Best-First-Search:



Περιοχή διερεύνησης Αλγόριθμου A^*

Στην περίπτωση του κοίλου εμποδίου, ο A^* εντοπίζει τη βέλτιστη διαδρομή όπως και ο αλγόριθμος του Dijkstra, διερευνώντας πολύ μικρότερη περιοχή:



Περιοχή διερεύνησης Αλγόριθμου A^* σε περιοχή με Εμπόδια

Η αποδοτικότητα του οφείλεται στο ότι συνδυάζει τα ψήγματα πληροφοριών που χρησιμοποιεί και ο αλγόριθμος του Dijkstra (προτίμηση περιοχών κοντινών στην αφετηρία) και ψήγματα πληροφοριών που χρησιμοποιεί ο Greedy Best-First-Search (προτίμηση περιοχών κοντινών στο στόχο).

Κατά τον υπολογισμό του κόστους κίνησης, η $G(n)$ αντιπροσωπεύει το ακριβές κόστος της διαδρομής από την αφετηρία προς οποιονδήποτε κόμβο n , και η $H(n)$ αντιπροσωπεύει την ευρετική εκτίμηση του κόστους κίνησης από τον κόμβο n ως τον στόχο. Στα παραπάνω διαγράμματα, το κίτρινο χρώμα (H) αντιπροσωπεύει κορυφές μακριά από το στόχο και το κυανό (G) παριστάνει κορυφές μακριά από την αφετηρία.

Ο A^* ισορροπεί ανάμεσα στις δύο συναρτήσεις καθώς κινείται από την αφετηρία προς τον στόχο. Σε κάθε βήμα, ο κύριος βρόχος του αλγορίθμου A^* , εξετάζει τον κόμβο n που χαρακτηρίζεται από το χαμηλότερο $F(n)$, όπου $F(n) = G(n) + H(n)$, προκειμένου να προσδιορίσει την επόμενη κίνηση.

Οι ευρετική προσέγγιση συνήθως δίνει μια κατά προσέγγιση μέθοδο επίλυσης ενός προβλήματος, χωρίς να εγγυάται ότι θα προκύψει η καλύτερη απάντηση. Ωστόσο, ο A^* είναι χτισμένος στην κορυφή της ευρετικής λογικής, και παρόλο που η ίδια η ευρετική δεν το εγγυάται, ο A^* μπορεί να εγγυηθεί τη βέλτιστη διαδρομή.

Συνοψίζοντας, η ευρετική χρησιμοποιείται για τον έλεγχο της συμπεριφοράς του A^* .

- Στο ένα άκρο, αν η $H(n)$ είναι ίση με μηδέν, τότε μόνο η $G(n)$ επηρεάζει την επιλογή της διαδρομής, ο A^* εκφυλίζεται σε αλγόριθμο του Dijkstra, και η βέλτιστη διαδρομή θα εντοπιστεί με βεβαιότητα.
- Αν η $H(n)$ είναι μικρότερη ή ίση του κόστους μετάβασης από τον κόμβο n στον στόχο, τότε το A^* εντοπίζει τη βέλτιστη διαδρομή εγγυημένα. Όσο χαμηλότερο είναι το $H(n)$, τόσο περισσότερο αναδιπλώνεται η περιοχή διερεύνησης, και ο A^* καθίσταται πιο χρονοβόρος.
- Αν η $H(n)$ ισούται με το κόστος της μετάβασης από τον κόμβο n στον στόχο, τότε ο A^* θα διερευνήσει μόνο την βέλτιστη διαδρομή χωρίς να επεκταθεί αλλού, οπότε η αναζήτηση θα είναι ταχύτατη.
- Αν η $H(n)$ είναι λίγο μεγαλύτερη από το κόστος της μετάβασης από τον κόμβο n στον στόχο, τότε ο A^* δεν είναι βέβαιο ότι η διαδρομή που θα βρεθεί θα είναι η βέλτιστη, αλλά η αναζήτηση θα είναι γρηγορότερη.
- Στο άλλο άκρο, εάν η $H(n)$ είναι πολύ υψηλή σε σχέση με την $G(n)$, τότε μόνο η $H(n)$ επηρεάζει την επιλογή της διαδρομής, και ο A^* μετατρέπεται σε Greedy Best-First-Search.

Οπότε, η με ακρίβεια υπολογισμένη ευρετική εντοπίζει βέλτιστα μονοπάτια ταχύτατα. Αν είναι πολύ χαμηλή η τιμή της, τότε θα εντοπίζονται και πάλι βέλτιστα μονοπάτια, αλλά η διαδικασία θα είναι χρονοβόρα. Αν είναι πολύ υψηλή, θα προκύπτουν απλά σύντομα μονοπάτια, αλλά ο A^* θα εκτελείται ταχύτερα.

Ακόμη με την προσθήκη μίας μεταβλητής στην ευρετική συνάρτηση λαμβάνεται υπόψη του αλγορίθμου η διαμόρφωση του εδάφους στο οποίο λαμβάνει χώρα η κίνηση.

ΕΦΑΡΜΟΓΕΣ

Το Path Finding συνδέεται συνήθως με την τεχνητή νοημοσύνη (AI) καθώς πολλοί αλγόριθμοι που ανήκουν σε αυτή την κατηγορία αναπτύχθηκαν από ερευνητές AI. Ο A^* μπορεί να εφαρμοστεί σε πολλούς τομείς πέρα από την ρομποτική, όπως η βιολογία, τα νευρωνικά δίκτυα, τους γενετικούς αλγόριθμους και την ενισχυμένη εκμάθηση.

Ακόμη, ο A^* εφαρμόζεται σε συστήματα τηλεπικοινωνιών και σε εφαρμογές hardware για σύντομη διασύνδεση ακροδεκτών pin, σε μικρο-ελεγκτές με πολλούς ακροδέκτες εξόδου.

13. ΥΛΟΠΟΙΗΣΗ

Η συγκεκριμένη εργασία πραγματεύεται την ανάπτυξη ενός αλγορίθμου προσδιορισμού διαδρομής με σκοπό την εφαρμογή του σε ρομποτικό όχημα. Στο σύστημα θα εισάγεται ένας χάρτης του περιβάλλοντα χώρου, η αφετηρία της τροχιάς που θα αναζητηθεί και στην οποία είναι τοποθετημένο αρχικά το όχημα και τέλος, ο προορισμός της κινούμενης μονάδας. Ο αλγόριθμος θα πρέπει να είναι σε θέση να παρακάμπτει εμπόδια και να υπολογίζει τη βέλτιστη διαδρομή. Η εργασία λοιπόν, ανήκει στους τομείς :

1. Του Path Finding
2. Του αντικειμενοστραφή προγραμματισμού και
3. Της Ρομποτικής

13.1. *PATH FINDING*

Στην περίπτωση αυτή που είναι γνωστό το πέρας της κίνησης, ο ιδανικότερος αλγόριθμος Path Finding είναι ο A*. Για την υλοποίηση του A* είναι απαραίτητες οι εξής δομές:

Χάρτης πλέγματος

μια δομή δεδομένων που περιέχει πληροφορίες για τη γειτονική περιοχή κάθε θέσης.

Τοποθεσίες

μια αριθμητική τιμή που προσδιορίζει τη θέση στο πλέγμα και κατ' επέκταση στο φυσικό χώρο

Αναζήτηση

ο αλγόριθμος A* που διαθέτει ένα γράφημα, μια αφετηρία, και προαιρετικά έναν στόχο, και υπολογίζει κάποιες χρήσιμες πληροφορίες (διερευνημένος, δείκτης βάσης, απόσταση) για ορισμένες ή όλες τις θέσεις στο χάρτη

Προτεραιότητα

μια δομή δεδομένων που χρησιμοποιείται από τον αλγόριθμο αναζήτησης για να αποφασίσει τη σειρά με την οποία θα διερευνηθούν οι θέσεις

13.1.1. *Ψευδοκώδικας*

Ο αλγόριθμος A*, καθαρά ως λογική και απογυμνωμένος κάθε κώδικα, είναι αρκετά απλός. Υπάρχουν δύο σύνολα, ή λίστες κόμβων, η Ανοιχτή και η Κλειστή. Η ανοιχτή λίστα περιέχει τους υποψήφιους προς εξέταση κόμβους και αρχικά περιέχει μόνο έναν κόμβο, την αφετηρία. Η κλειστή λίστα περιέχει τους κόμβους που έχουν ήδη εξεταστεί και είναι κενή αρχικά. Πρακτικά στο πλέγμα, η ανοιχτή λίστα είναι η μεθόριος και η κλειστή λίστα είναι οι κόμβοι διέλευσης της κινούμενης μονάδας (βάσεις), κάθε κόμβος εκ των οποίων διατηρεί επίσης ένα δείκτη προς τον κόμβο της βάσης από την οποία προήλθε.

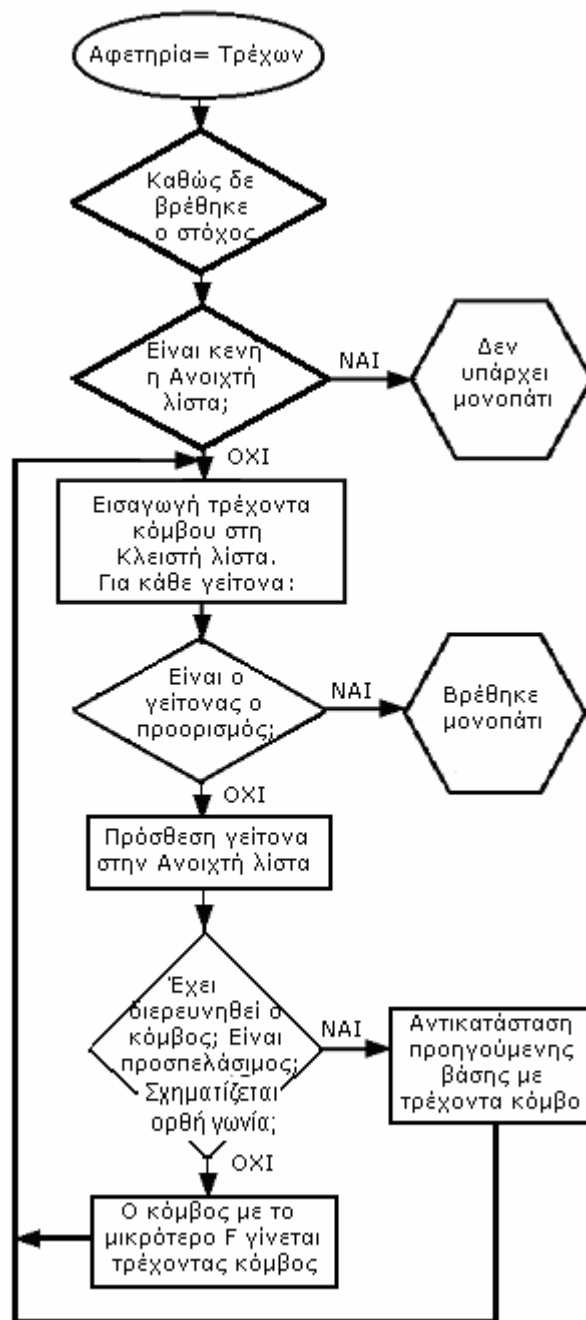
Ο κεντρικός βρόχος του αλγορίθμου, επανειλημμένα επιλέγει τον βέλτιστο κόμβο n από την ανοιχτή λίστα (ο κόμβος με τη μικρότερη τιμή F) και τον εξετάζει. Εάν ο n κόμβος είναι ο στόχος, τότε η αναζήτηση ολοκληρώνεται. Διαφορετικά, ο κόμβος n αφαιρείται από την ανοιχτή λίστα και προστίθεται στην κλειστή. Έπειτα, εξετάζονται οι γειτονικοί κόμβοι αυτού, οι n' . Εάν ένας γειτονικός κόμβος βρίσκεται στην κλειστή λίστα έχει ήδη διερευνηθεί, και δεν χρειάζεται να ξαναελεγχθεί, ενώ εάν βρίσκεται στην ανοιχτή λίστα πρόκειται να διερευνηθεί (ως προς F και ως προς G). Στην αντίθετη περίπτωση, προστίθεται στην ανοιχτή λίστα, με τον κόμβο βάσης του, να είναι ο n . Το κόστος διαδρομής του n' (δηλαδή το $G(n')$), ορίζεται στην τιμή $G(n') = G(n) + \text{ΚόστοςΚίνησης}(n, n')$.

Σε μορφή εντολών ο A^* δρα ως εξής:

- 1) Τίθεται αφετηρία = τρέχων κόμβος
- 2) Καθώς δε βρέθηκε ο στόχος
 - A. Εισαγωγή του τρέχοντα κόμβου στη Κλειστή λίστα (C)
 - B. Υπολογισμός για κάθε παρακείμενο κόμβο:
 - a. Αν ο εφαπτόμενος κόμβος είναι ο προορισμός, ολοκληρώθηκε η αναζήτηση
 - b. Πρόσθεση γείτονα στην Ανοιχτή λίστα (F)
 - c. Αν έχει διερευνηθεί ο κόμβος και είναι προσπελάσιμος
 - I. Αν σχηματίζεται ορθή γωνία στο μονοπάτι, αφαίρεση τελευταίου κόμβου από κλειστή λίστα και προσθήκη τρέχοντα κόμβου
 - II. Αλλιώς ο κόμβος με το μικρότερο F γίνεται ο τρέχοντας κόμβος
 - C. Αν η Ανοιχτή λίστα είναι άδεια, ο στόχος δεν είναι προσβάσιμος
 - D. Διαγραφή του περιεχομένου της λίστας προτεραιότητας
- 3) Εξαγωγή κλειστής λίστας προς ανάγνωση μονοπατιού
- 4) Εξαγωγή κατευθύνσεων
- 5) Εξομάλυνση διαδρομής

13.1.2. Διάγραμμα Ροής

Ακολουθεί το διάγραμμα ροής του αλγορίθμου:



Διάγραμμα Ροής Υλοποιημένου Αλγορίθμου (κύριος βρόχος)

13.2. ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Έπειτα από το σχεδιασμό της αρχιτεκτονικής δομής του Path Finder, πρέπει να υλοποιηθεί ο αλγόριθμος. Ανάλογα με το είδος της κινούμενης μονάδας, επιλέγεται η καταλληλότερη γλώσσα προγραμματισμού. Ωστόσο, είναι αναγκαίο η προσομοίωση η οποία προηγείται του ελέγχου της κίνησης της μονάδας, να δίνει τη δυνατότητα μελέτης της συμπεριφοράς του αλγορίθμου σε ποικίλες μορφολογίες της περιοχής κίνησης.

13.2.1. C++

Η γλώσσα που χρησιμοποιείται για τον προγραμματισμό του αλγορίθμου είναι η C++. Υποστηρίζει δομημένο αντικειμενοστραφή προγραμματισμό, ο οποίος αποτελεί βασικό κριτήριο για την υλοποίηση του Path Finder, προκειμένου το πρόγραμμα να είναι εφαρμόσιμο σε διαφορετικούς τύπους οχημάτων. Το περιβάλλον προσομοίωσης επιτρέπει την παραγωγή μίας καλής και ευανάγνωστης εικόνας για την συμπεριφορά του αλγορίθμου στο χώρο. Επιπλέον, η ύπαρξη των κλάσεων στη C++ επιτρέπει την ταχύτερη εκτέλεση του προγράμματος, καθώς οι συναρτήσεις που απαρτίζουν διάφορες κλάσεις εκτελούνται παράλληλα. Τέλος, η χρήση των destructors αποτρέπει την συσσώρευση πολλών δεδομένων στη μνήμη.

Το Path Finder καθορίζει τις δομές δεδομένων που θα πρέπει να υλοποιηθούν.

a. Χάρτης πλέγματος

Προκειμένου να αναπαρασταθεί ο χώρος, το πρώτο στάδιο είναι η υλοποίηση του πλέγματος. Επιλέγεται το τετραγωνικό πλέγμα και ως βασικό μέρος του κόμβου επιλέγεται η επιφάνεια. Στην περίπτωση επιλογής ακμών ή κορυφών για τον προσδιορισμό της κίνησης, θα ήταν αναπόφευκτες κάποιες συγκρούσεις κατά την κίνηση του οχήματος στις περιοχές που εφάπτονται με εμπόδια.

Η υλοποίηση του χάρτη στο πρόγραμμα επιτυγχάνεται με τη βοήθεια dynamic 2D arrays. Κάθε θέση του πίνακα, κάθε κόμβος δηλαδή, αποτελεί μία θέση στη μνήμη όπου αποθηκεύεται η τιμή των B (βάση), ή G και ή H που είναι ακέραιος αριθμός. Η διάσταση του πίνακα καθορίζεται στην αρχή της εκτέλεσης του προγράμματος.

Εμπόδια: Η εισαγωγή των εμποδίων επιτυγχάνεται με τη δημιουργία ενός πανομοιότυπου πίνακα με αυτόν του χάρτη, του Walkable, κάθε θέση του οποίου μπορεί να πάρει μόνο δύο τιμές. Αν ο κόμβος έχει την τιμή 0(true), τότε στον κόμβο δεν υπάρχει εμπόδιο. Ειδάλλως, η τιμή 1(false), υποδηλώνει ότι η συγκεκριμένη θέση στο χώρο, δεν μπορεί να προσπελαστεί.

b. Τοποθεσίες

Από άποψη συντεταγμένων, σε κάθε κόμβο ανατίθεται ένας αύξων αριθμός μέσω της συνάρτησης Key(), καθώς και δύο συντεταγμένες X και Y, μέσω των συναρτήσεων KeyX() και KeyY(). Η διάσταση του πίνακα που καθορίζεται στην αρχή της εκτέλεσης του προγράμματος, είναι η σταθερά με βάση την οποία προσδιορίζονται οι υπόλοιπες συντεταγμένες.

```

int Key(int &KeyX, int &KeyY){
    int r;
    r=KeyY*fieldWidth+KeyX;
    return r;
}
int KeyX(int &Key){
    int r;
    r=Key%fieldWidth;
    return r;
}
int KeyY(int &Key){
    int r;
    r=Key/fieldHeight;
    return r;
}

```

c. Αναζήτηση

Κατά την εκτέλεση του αλγορίθμου, δημιουργούνται δύο χάρτες, ο ένας εκ των οποίων υπολογίζει τη συνάρτηση G και ο άλλος την ευρετική συνάρτηση H για κάθε θέση του χώρου. Οι χάρτες αυτοί, αθροίζουν τις τιμές τους στην εκάστοτε θέση και προκύπτει το F. Το F των κόμβων που διερευνώνται εισάγεται στην Ανοιχτή λίστα.

Κόστος κίνησης : Το ελάχιστο κόστος D της μετάβασης από τον ένα κόμβο στον παρακείμενο ορίζεται σε 10 μονάδες για οριζόντια και κάθετη κίνηση, και στις 14 μονάδες για τη διαγώνια κίνηση. Η επιλογή συνδέεται με το μήκος του ανθρώπινου βήματος, το οποίο κατά μέσο όρο υπολογίζεται στο ένα μέτρο. Αν ο βηματισμός της κινούμενης μονάδας, δηλαδή η απόσταση μεταξύ δύο κόμβων, είναι 1, το βήμα κατά μήκος της διαγωνίου του τετραγώνου υπολογίζεται με βάση το Πυθαγόρειο Θεώρημα περίπου ίσο με 1,4. Προκειμένου να αποφευχθούν πράξεις με δεκαδικούς αριθμούς οι οποίοι θα καταστούσαν τον πίνακα δυσανάγνωστο στην προσομοίωση, μετατρέπουμε τις τιμές αυτές σε ακέραιες.

Αφού ολοκληρωθεί η διαδικασία της αναζήτησης της διαδρομής, εξάγονται οι κόμβοι που συνιστούν το μονοπάτι από την κλειστή λίστα, και εισάγονται σε έναν νέο χάρτη, τον B, που δημιουργείται με σκοπό να απεικονίσει τη βέλτιστη διαδρομή. Οι θέσεις από τις οποίες διέρχεται το όχημα αναγράφονται ως μονάδα (1). Η αφετηρία και ο στόχος της κίνησης, σημειώνονται ως 11 και οι μη προσπελάσιμοι κόμβοι με 111.

d. Προτεραιότητα

Ίσως η σημαντικότερη από όλες τις διεργασίες που επιτελεί ο αλγόριθμος, είναι ο υπολογισμός της προτεραιότητας, που μεταφράζεται πρακτικά σε επιλογή του επόμενου βήματος. Η προτεραιότητα προσδιορίζεται διαμέσου της Ανοιχτής λίστας, η υλοποίηση της οποίας επιτυγχάνεται με τη βοήθεια μίας Linked List η οποία υλοποιείται ως μία αλυσίδα κόμβων. Κάθε κόμβος της λίστας αυτής αποτελεί μία δομή η οποία περιλαμβάνει:

- 1) την αρίθμηση (συντεταγμένη) της θέσης του χάρτη που μελετάται για το αν θα γίνει η επόμενη βάση,
- 2) την προτεραιότητα της θέσης αυτής που είναι η τιμή F όπως αυτή υπολογίζεται από την πρόσθεση των πλεγμάτων,
- 3) και ένα δείκτη που πληροφορεί για τον ποιος είναι ο επόμενος κόμβος της αλυσίδας αυτής.

Με την προσθήκη ενός νέου κόμβου και της προτεραιότητάς του στη λίστα, αρχίζει να επιτελείται απευθείας και εσωτερικά στην συνάρτηση εισαγωγής, μία ταξινόμηση των κόμβων με βάση την τιμή της προτεραιότητάς τους (F). Η ταξινόμηση αρχίζει από την κεφαλή της αλυσίδας, συγκρίνονται ανά δύο οι προτεραιότητες του νεοεισαχθέντος κόμβου, με την προτεραιότητα της κεφαλής. Αν ο νεοεισαχθείς κόμβος έχει μικρότερη τιμή F από την κεφαλή της λίστας, τοποθετείται στην κεφαλή της λίστας εκτοπίζοντας μία θέση τους υπόλοιπους κόμβους. Αν όχι, τοποθετείται στο τέλος της λίστας (την ουρά). Με αυτόν τον τρόπο η διαδικασία της επιλογής καθίσταται άμεση.

Ακόμη, το πρόβλημα του χρόνου κατά την επιλογή του επόμενου βήματος βελτιώνεται εάν φροντίσουμε η λίστα να διαθέτει όσο το δυνατόν λιγότερα στοιχεία προς σύγκριση ανά πάσα στιγμή. Σε τετραγωνικό πλέγμα με δυνατότητα διαγωνίου κίνησης, οι γειτονικοί κόμβοι, οκτώ στον αριθμό, καθορίζουν τις δυνατές κατευθύνσεις της κίνησης. Λαμβάνονται υπόψη μόνο αυτοί οι οκτώ κόμβοι, καθώς ο ένατος, ο τρέχων κόμβος, αποτελεί ήδη βάση και δεν υπάρχει λόγος να συμπεριληφθεί στην απόφαση για το επόμενο βήμα. Επίσης, αποτρέπεται η προσθήκη των μη προσπελάσιμων κόμβων στη λίστα. Τέλος, μετά την ολοκλήρωση της σύγκρισης σε κάθε κύκλο του αλγορίθμου, φροντίζουμε να αδειάζει η λίστα, προκειμένου να μην καταναλώνει πολύ χώρο μνήμης. Όλες αυτές οι ρυθμίσεις εκτελούνται μέσα από τον κύριο βρόχο του αλγορίθμου.

Προκειμένου να βελτιωθεί η ταχύτητά της Linked List, το χρονοβόρο κριτήριο εισαγωγής μίας θέσης σε αυτή ελέγχεται εκτός της κλάσης της λίστας, μέσα από τον κύριο αλγόριθμο και με τη βοήθεια του χάρτη αληθείας. Ακόμη η συνάρτηση που αφορά την αλλαγή προτεραιότητας η `Priority_change`, δεν ακολουθεί τη διαδικασία της αφαίρεσης της παλιάς τιμής και πρόσθεσης της νέας, διαδικασία που είναι επίσης πολύ χρονοβόρα, αλλά προσθέτει απευθείας την νέα οντότητα βασιζόμενη σε εσωτερική ταξινόμηση η οποία θα αναδείξει το F με τη μικρότερη τιμή.

e. Κλειστή Λίστα

Μόλις επιλεγεί ποιο θα είναι το επόμενο βήμα στο χάρτη από την Ανοιχτή λίστα, το βήμα αυτό αποθηκεύεται για να σχηματίσει τη διαδρομή μαζί με όλα τα αποθηκευμένα βήματα στο τέλος της αναζήτησης. Είναι υψίστης σημασίας η αποθήκευση των βημάτων, ακριβώς με τη σειρά που θα γίνουν. Η ιδανικότερη δομή δεδομένων για αυτό, είναι η απλή Linked List, με τους νέους κόμβους να μην εισάγονται στην κεφαλή της λίστας όπως στην παραπάνω περίπτωση, αλλά στην ουρά.

Εφόσον η λίστα δεν είναι ταξινομημένη, η ενέργεια της εισαγωγής εκτελείται ταχύτατα. Ο αλγόριθμος όμως απαιτεί να γίνεται έλεγχος για το εάν έχει εισαχθεί ο κόμβος

προορισμού, και μάλιστα, σε κάθε βήμα της αναζήτησης, προκειμένου να τερματιστεί η αναζήτηση. Η απαίτηση αυτή, προϋποθέτει τη διερεύνηση όλης της λίστας, και καθώς ο χάρτης μεγαλώνει και μαζί με αυτόν και η Κλειστή λίστα, η όλη διαδικασία καθίσταται χρονοβόρα. Ωστόσο, είναι δυνατόν να απλουστευτεί απόλυτα αυτή η διαδικασία, ελέγχοντας μόνο το τελευταίο στοιχείο, την ουρά της λίστας. Άλλωστε στην ουρά θα τοποθετηθεί ο στόχος.

Ακόμη μέσω της κλειστής λίστας υλοποιείται ο έλεγχος σχηματισμού ορθής γωνίας. Η τεχνική σύγκρισης παλαιών και νέων τιμών του G , ενώ λειτουργεί ικανοποιητικά στις γωνίες, προκαλεί προβλήματα στην ευθεία, απωθώντας το μονοπάτι από τον στόχο. Για τον λόγο αυτό υλοποιείται ένα πακέτο συναρτήσεων μέσα στην κλειστή λίστα το οποίο ελέγχει για ακολουθίες κόμβων οι οποίες σχηματίζουν ορθή γωνία. Άπαξ και εντοπιστεί μία τέτοια ασυνέπεια στο μονοπάτι, διορθώνεται στον κύριο βρόχο.

Τέλος, θα πρέπει να δοθεί προσοχή στο γεγονός ότι το μονοπάτι αναπτύσσεται από την κεφαλή της λίστας προς την ουρά και με αυτή την σειρά πρέπει να τελεστούν τα βήματα από την κινούμενη μονάδα.

Ένας επιπλέον λόγος για την επιλογή *arrays*, οποιασδήποτε διάστασης (ισχύει και στον χάρτη δηλαδή) είναι το ότι, παρόλο που οι κόμβοι μπορούν να δημιουργηθούν και να διατηρηθούν με δυναμικό αντικειμενοστραφή τρόπο, ο χρόνος που χρειάζεται για να εκτελεστούν ο *constructor* και ο *destructor* είναι μεγάλος και επιβραδύνει τα πράγματα. Και για τους πίνακες, είναι απαραίτητη αυτή η διαγραφή.

Η διαδικασία αυτή μπορεί να αποφευχθεί, με το μηδενισμό μόνο της Ανοιχτή λίστας στο τέλος κάθε κύκλου διαλογής (μετά τον προσδιορισμό του βήματος δηλαδή). Παρόλα αυτά η αποθήκευση δεδομένων σε πολλαπλούς πίνακες καταναλώνει αρκετή μνήμη.

f. Προσβασιμότητα

Υπάρχει περίπτωση ο στόχος να μην είναι προσβάσιμος από την αφετηρία λόγω παρεμβολής εμποδίων στο χάρτη. Ο A^* θα καταναλώσει πολύ χρόνο ώσπου να συνειδητοποιήσει ότι δεν υπάρχει προσπελάσιμο μονοπάτι, δεδομένου ότι έχει να εξερευνήσει κάθε πιθανό συνδυασμό κόμβων που μπορεί να οδηγή στον προορισμό.

Το πρόγραμμα έχει την ικανότητα να αντιλαμβάνεται την κατάσταση αυτή και να τερματίζει την αναζήτηση. Ακόμη, η ίδια αυτή συνάρτηση, μπορεί να αντιληφθεί την περίπτωση στην οποία δεν υπάρχει στόχος κίνησης, οπότε και συντελείται και πάλι τερματισμός της αναζήτησης. Ο κώδικας που επιτυγχάνει αυτόν τον έλεγχο είναι ο ακόλουθος:

```

if (F.empty()){
    searching=false;
    found=false;
    cout<<"Destination is unreachable"<<endl;
}

```

g. **Επιδόσεις**

Πυρήνας του αλγορίθμου A* είναι η επιλογή που επιτελείται μέσα στη λίστα προτεραιότητας. Ο A* πρέπει να είναι σε θέση να μελετά τη λίστα προτεραιότητας, να την αναλύει και να εισάγει κόμβους. Επιπροσθέτως, εντοπίζει τους κόμβους που έχουν ήδη διερευνηθεί. Μέθοδοι βελτίωσης της απόδοσης του A* αφορούν :

- **Τη μείωση του μεγέθους του γραφήματος.** Η ενέργεια αυτή μειώνει τον αριθμό των κόμβων που υποβάλλονται σε επεξεργασία, τόσο αυτών που αποτελούν τμήμα του μονοπατιού όσο και εκείνων που δεν καταλήγουν στη τελική διαδρομή. Οι ιεραρχικές αναπαραστάσεις χάρτη συμβάλουν στη βελτίωση αυτή.
- **Τη βελτίωση της ακρίβεια της ευρετικής.** Η ενέργεια αυτή μειώνει τον αριθμό των κόμβων που ενώ διερευνούνται δεν γίνονται τμήμα της τελικής διαδρομής. Όσο πιο κοντά βρίσκεται η ευρετική (H) στο πραγματικό μήκος της διαδρομής- όχι την απόσταση ως τον στόχο-, τόσο λιγότερους κόμβους θα διερευνήσει ο A*.
- **Τη ταχύτερη λίστα προτεραιότητας.** Εναλλακτικές δομές δεδομένων σύμφωνα με τις απαιτήσεις της κάθε περίπτωσης βελτιώνουν την απόδοση. Η επεξεργασία κόμβων σε πακέτα και η εσωτερική κατάταξη των κόμβων επίσης βελτιώνει την χρονική απόδοση.
- **Τη ταχύτερη ευρετική.** Η ευρετική συνάρτηση χρησιμοποιείται σε κάθε ανοιχτό κόμβο. Η ενσωμάτωση της κλήσης της σε κάθε κόμβο βελτιώνει την χρονική απόδοση.

13.2.2. **VHDL**

Επιπλέον μελετήθηκε η υλοποίηση σε γλώσσα περιγραφής υλικού, και συγκεκριμένα την VHDL. Οι δομές δεδομένων που θα πρέπει να υλοποιηθούν με βάση το Path Finding, είναι διαφορετικές συγκριτικά με την προηγούμενη περίπτωση και σύμφωνες με τις απαιτήσεις και τις λύσεις που παρέχει η VHDL.

a. **Χάρτης πλέγματος**

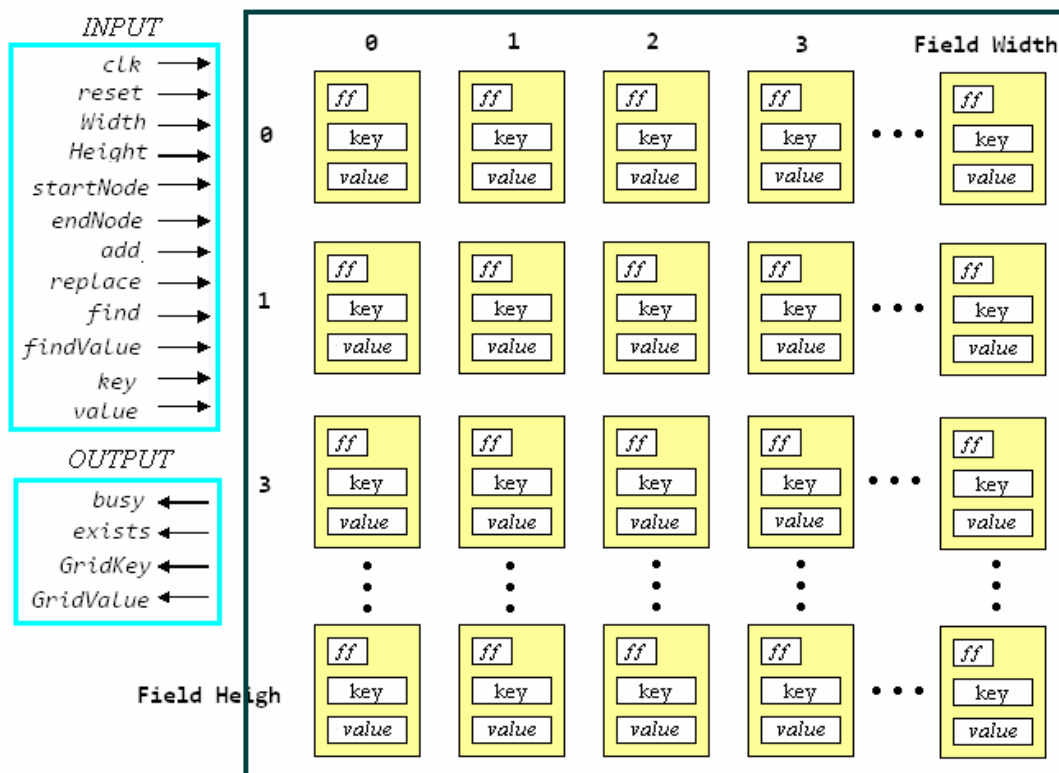
Προκειμένου να υλοποιηθεί το πλέγμα, χρησιμοποιείται δομικά, ένας διδιάστατος πίνακας 2Δ. Ο πίνακας στη VHDL είναι μία συλλογή αντικειμένων ίδιου τύπου δεδομένων (κόμβοι). Κάθε κόμβος, που συνιστά και θέση του πίνακα, αποτελεί μία

δομή η οποία αποτελείται από δύο καταχωρητές και ένα flip-flop. Στον κόμβο καταχωρούνται η **συντεταγμένη** της θέσης του κόμβου αυτού, και η **τιμή** που αντιστοιχεί σε αυτή τη θέση από τους υπολογισμούς της ευρετικής συνάρτησης. Το flip-flop υποδεικνύει το αν **υπάρχει αποθηκευμένη πληροφορία** στην εν λόγω θέση ή όχι. Η διάσταση του πίνακα καθορίζεται στον κύριο βρόγχο του προγράμματος.

Εμπόδια: Ο πίνακας με τη βοήθεια σημάτων εισόδου δέχεται τις ίδιες αλλαγές στις πληροφορίες που περιέχει, με την υλοποίηση σε C++ όπως και είναι αναμενόμενο.

Η εισαγωγή των εμποδίων επιτυγχάνεται, και σε αυτή την περίπτωση, με τη δημιουργία ενός πανομοιότυπου πίνακα με αυτόν, του Walkable, κάθε θέση του οποίου μπορεί να πάρει μόνο δύο τιμές. Αν ο κόμβος έχει την τιμή 0 (true), τότε στον κόμβο δεν υπάρχει εμπόδιο, ενώ αν η τιμή είναι 1 (false), η συγκεκριμένη θέση στο χώρο, είναι μη προσπελάσιμη.

Στο επόμενο σχήμα εικονίζεται η δομή του χάρτη πλέγματος:



Διάγραμμα Βαθμίδων Χάρτη

b. Αναζήτηση

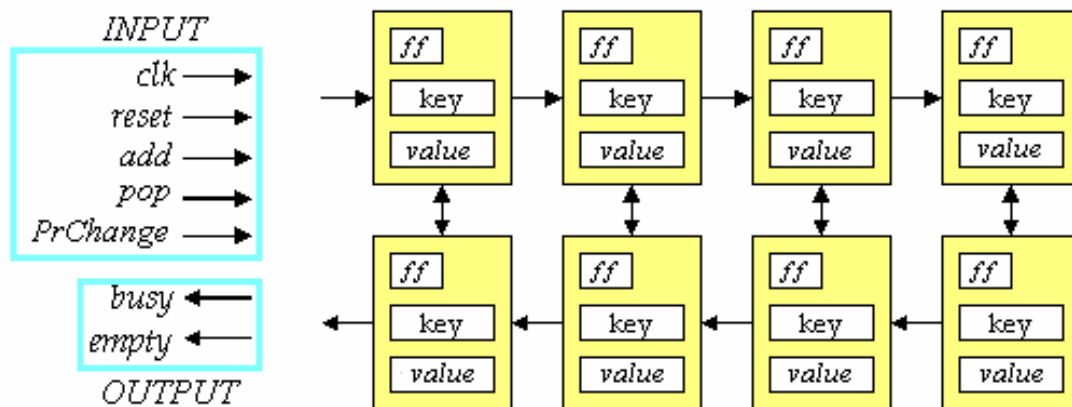
Και σε αυτή τη περίπτωση, κατά την εκτέλεση του αλγορίθμου, δημιουργούνται δύο χάρτες, ο ένας περιέχει τη συνάρτηση G και ο άλλος την ευρετική συνάρτηση H για κάθε θέση του χώρου. Οι χάρτες αυτοί, αθροίζουν τις τιμές τους στην εκάστοτε θέση μέσω ενός αθροιστή και προκύπτει το F. Το F των κόμβων που έχουν διερευνηθεί εισάγεται στην ανοιχτή λίστα. Το διάγραμμα βαθμίδων είναι το ακόλουθο:

Το κόστος κίνησης ορίζεται ξανά, στις 10 μονάδες για οριζόντια και κάθετη κίνηση, και

στις 14 μονάδες για τη διαγώνιο κίνηση.

γ. Προτεραιότητα

Η Ανοιχτή λίστα υλοποιείται με τη βοήθεια ενός κωδικοποιητή προτεραιότητας (priority encoder). Για την αποδοτικότερη λειτουργία της λίστας, αυτή μπορεί να αποθηκεύει έως και οκτώ στοιχεία, όπως και στην περίπτωση της C++. Η δομή της εικονίζεται στο ακόλουθο σχήμα.



Διάγραμμα Βαθμίδων Priority Encoder

Σχεδιάζεται ώστε να διαθέτει δύο γραμμές από σύνολα των τεσσάρων θέσεων. Κάθε θέση αποτελεί και στην περίπτωση αυτή μία δομή (έναν κόμβο), η οποία αποτελείται από δύο καταχωρητές και ένα flip-flop. Οι καταχωρητές αποθηκεύουν την συντεταγμένη του κόμβου (key) και την προτεραιότητά του (value), ενώ το flip-flop πληροφορεί για το αν στον κόμβο υπάρχουν αποθηκευμένα δεδομένα ή όχι.

Οι κόμβοι που αποθηκεύονται στην κάτω σειρά, έχουν μικρότερη τιμή προτεραιότητας από αυτούς της άνω σειράς. Επιπλέον, στην πάνω σειρά η μεγαλύτερη τιμή προτεραιότητας ανήκει στον αριστερότερο κόμβο, ενώ η μεγαλύτερη τιμή προτεραιότητας της κάτω σειράς αντιστοιχεί στον δεξιότερο κόμβο. Συνεπώς, μικρότερη τιμή προτεραιότητας διαθέτει ο κάτω αριστερός κόμβος. Σχετικά με τις στήλες, οι πάνω κόμβοι έχουν μεγαλύτερη τιμή προτεραιότητας.

Με βάση την λογική αυτή, η **εισαγωγή** ενός στοιχείου στη λίστα (σήμα εισόδου add), συμβαίνει στην πρώτη γραμμή της πρώτης στήλης, εφόσον έχει προηγηθεί ένα right shift στους κόμβους που υπάρχουν στη πρώτη σειρά. Ακολουθεί σύγκριση ανάμεσα στις τιμές των στοιχείων των στηλών και, σε περίπτωση που το στοιχείο της πρώτης γραμμής έχει μικρότερη προτεραιότητα από αυτό της δεύτερης, ακολουθεί ανταλλαγή θέσης. Ενώ συντελείται η ταξινόμηση, το σήμα busy επιστρέφει λογικό '1'. Η όλη διαδικασία ολοκληρώνεται σε δύο παλμούς ρολογιού.

Η διαγραφή ενός στοιχείου (σήμα εισόδου pop) είναι παρόμοια. Προηγείται ένα left shift των στοιχείων της δεύτερης γραμμής, διαγράφοντας το μικρότερο στοιχείο. Ακολουθεί σύγκριση ανάμεσα στα στοιχεία κάθε στήλης και ανακατάταξη ώστε τα στοιχεία της πρώτης σειρά να έχουν μεγαλύτερη τιμή από αυτά της δεύτερης.

Το αν η λίστα είναι κενή, συμπεραίνεται από την κατάσταση του flip-flop της πρώτης γραμμής και πρώτης στήλης. Αν βρίσκεται σε λογικό '1', τότε είναι κενή, ειδάλλως όχι.

Η δομή του priority encoder διαθέτει κάποια σήματα καταστάσεων (ready, adding, popping). Αν εισάγεται ένα στοιχείο, όσο διαρκεί η εισαγωγή και ανακατάταξη των κόμβων, ενεργοποιείται η κατάσταση adding. Ομοίως συμβαίνει και με την κατάσταση popping. Όταν δεν εκτελούνται οι διεργασίες αυτές, είναι ενεργοποιημένη η κατάσταση ready, και μόνο τότε μπορεί να λάβει χώρα μία νέα διαδικασία εισαγωγής ή διαγραφής.

Όταν η είσοδος reset οδηγηθεί σε λογικό '1', η λίστα αδειάζει. Το σήμα PrChange, εντοπίζει ένα συγκεκριμένο key και αλλάζει τη value του, δηλαδή αλλάζει την προτεραιότητα ενός κόμβου.

d. Κλειστή Λίστα

Η φύση και οι απαιτήσεις της Κλειστής λίστας σχετικά με τον χειρισμό και την αποθήκευση των βάσεων της διαδρομής, υλοποιούνται ιδανικά μέσω της FIFO. Η FIFO επιστρέφει τα δεδομένα που καταχωρούνται με την ίδια ακριβώς σειρά που καταχωρήθηκαν, επομένως διατηρεί την ακολουθία των βάσεων.

Η υλοποιημένη FIFO διαθέτει δύο επίπεδα (processes). Ένα στο οποίο γίνεται ο έλεγχος της λογικής λειτουργίας της, και ένα στο οποίο αποθηκεύονται τα δεδομένα (μία RAM εν ολίγοις). Στην ουσία είναι η process της λογικής που αναγκάζει τη RAM να συμπεριφερθεί ως FIFO (δηλαδή ως στοίβα). Η RAM διαθέτει μία είσοδο και μία έξοδο, από τις οποίες μετακινούνται δεδομένα. Το σήμα WE (Write Enable) καθορίζει το πότε είναι επιτρεπτή η αποθήκευση των δεδομένων.

Η λογική ελέγχου FIFO διαθέτει τα βασικά σήματα που είναι απαραίτητα για τη λειτουργία του αλγορίθμου. Κάθε διεργασία εκτελείται όταν οι παλμοί του ρολογιού οδηγούνται σε λογικό '1'. Δύο επιπλέον σήματα, τα NoPop και NoAdd, είναι σήματα σφάλματος που ενεργοποιούνται :

- Όταν ενεργοποιείται η λειτουργία Add, και η FIFO είναι γεμάτη
- Όταν ενεργοποιείται η λειτουργία Pop, και η FIFO είναι κενή

Ακολουθεί το διάγραμμα βαθμίδων της FIFO:

13.3. Ρομποτική

Εφόσον γραφεί ο αλγόριθμος αναζήτησης, θα πρέπει να μεταφραστεί αυτό το μονοπάτι σε βήματα της μονάδας κίνησης. Προσοχή θα πρέπει να δοθεί κατά την κίνηση σε εξωτερικές παραμέτρους, όπως, η ταχύτητα περιστροφής των τροχών, ιδίως στη περίπτωση που η κινούμενη μονάδα στρίβει. Η κακώς ρυθμισμένη κίνηση των τροχών θα έχει ως αποτέλεσμα το όχημα να κινείται προς λάθος κατεύθυνση. Επίσης θα πρέπει να δοθεί προσοχή και στην ταχύτητα της κίνησης.

Για την υλοποίηση μιας κινούμενης μονάδας απαιτούνται:

1. Η αρχικοποίηση ενός προσομοιωμένου οχήματος/αντικειμένου, το οποίο θα μπορεί ο αλγόριθμος να το κατευθύνει
2. Η αναπαράσταση του χώρου κίνησης, από άποψη κλίμακας και προσβασιμότητας
3. Ο προσδιορισμός της κίνησης ανάμεσα σε δύο περιοχές, την αφετηρία και τον στόχο η προορισμό και η εφαρμογή της πληροφορίας αυτής το όχημα
4. Έλεγχος της ικανότητας προσδιορισμού του οχήματος
5. Υλοποίηση κάποιου είδους αρχιτεκτονικής συμπεριφοράς του robot.

13.3.1.Αντιστοίχιση Διαδρομής με Ρομποτική Κίνηση

Προκειμένου η κινούμενη μονάδα να μπορέσει να αξιοποιήσει την πληροφορία της βέλτιστης διαδρομής, έτσι όπως τη προσδιορίζει το Path Finding μέσα από τη κλειστή λίστα, είναι απαραίτητο να μεταφραστεί η αρίθμηση των κόμβων σε κατευθύνσεις κίνησης.

Το πλέγμα όπως σχεδιάστηκε επιτρέπει την κίνηση σε οχτώ κατευθύνσεις. Αυτές οι κατευθύνσεις μπορούν να μεταφραστούν σε σημεία του ορίζοντα για κίνηση στον φυσικό χώρο. Εάν το όχημα ξέρει την κατεύθυνση, μπορεί να περιστρέψει τους τροχούς του σε ανάλογη γωνία προκειμένου να εκτελέσει την κίνηση. Η αντιστοίχιση των κατευθύνσεων του πλέγματος με σημεία του ορίζοντα, μπορεί να επιτευχθεί με χρήση των συντεταγμένων του πλέγματος. Χρειάζεται μόνο μία απλή συνάρτηση για να επιτευχθεί αυτό:

```
int Movement(int Cpop, int old ){           //popc list, startNode
    int neo;
    neo = Cpop;
    if (neo== old)return 0; }                //Do nothing
    if (neo== old+1){ return 1; }            //Drive East
    if (neo== old-1){ return 2; }            //Drive West
    if (neo== old-fieldWidth){ return 3; }    //Drive North
    if (neo== old+fieldWidth){ return 4; }    //Drive South
    if (neo== old-fieldWidth+1){ return 5; }  //Drive North East
    if (neo== old-fieldWidth-1){ return 6; }  //Drive North West
    if (neo== old+fieldWidth+1){ return 7; }  //Drive South East
    if (neo== old+fieldWidth-1){ return 8; }  //Drive South West
    else {return -1;}                        //error
}
```

Η μεταβλητή Cpop στον παραπάνω κώδικα, αντιστοιχείται στην τιμή που επιστρέφει η συνάρτηση C.pop() όταν καλείται. Ανάλογα με την τιμή που επιστρέφει η συνάρτηση Movement(), Το όχημα θα πρέπει να αντιδράσει ανάλογα. Θα πρέπει να γραφεί μία συνάρτηση σύμφωνα με τις απαιτήσεις του κάθε οχήματος, που να εκμεταλλεύεται

αυτή την πληροφορία, μεταφράζοντάς την σε περιστροφή τροχών γύρω από τον άξονά τους (ταχύτητα), και γύρω από τον άξονα της κινούμενης μονάδας(στροφή).

Επιπλέον, για την ομαλότερη κίνηση της μονάδας, επιβάλλεται η εξομάλυνση της διαδρομής. Αυτό επιτυγχάνεται με τη βοήθεια της συνάρτησης κατευθύνσεων

```
List smooth;
while(!C.List_empty()){
    int path;
    int direction, olddirect;
    path=C.pop();
    P.Map_add(path,1);
    direction=Movement(path, old);
    old=path;
    if((direction==olddirect)&& (direction!=0)){
        smooth.Remove_last();
        smooth.List_add(path);
    }else{
        smooth.List_add(path);
    }
    olddirect=direction;
}
```

13.3.2. ΑΛΛΗΛΕΠΙΔΡΑΣΗ ΜΕ ΤΟΝ ΚΥΡΙΟ ΒΡΟΧΟ ΤΟΥ ΑΛΓΟΡΙΘΜΟΥ

Σε γενικές γραμμές, ο υπολογισμός του τμήματος της διαδρομής που βρίσκεται πιο κοντά στην αφετηρία είναι σημαντικότερος από τον υπολογισμό του μονοπατιού κοντά στο στόχο. Υπάρχει ένας άγραφος κανόνας που απαιτεί η μονάδα να αρχίσει να κινείται το ταχύτερο δυνατόν, ακόμα και κατά μήκος ενός μη βέλτιστου μονοπατιού, και να ο προσδιορισμός της καλύτερης πορείας να έλθει στη συνέχεια. Τέτοιου είδους συμπεριφορά όμως, αποτελεί σε πρακτικά συστήματα κίνησης, κατασπατάληση των διαθέσιμων πόρων.

Η κινούμενη μονάδα μπορεί να προγραμματιστεί -όπως και η ίδια η φύση- για να ακολουθήσει είτε το ένστικτό της (απλή κίνηση) ή το μυαλό της (προϋπολογισμένη διαδρομή). Η μονάδα θα επιτελέσει απλή κίνηση, εκτός αν το «μυαλό» της, της επιβάλει το αντίθετο.

Ακόμη, στην περίπτωση που η δομή δεδομένων η οποία επιλέχθηκε για την αποθήκευση της ανοιχτής και της κλειστής λίστας είναι μικρή, θα μπορούσε να τερματιστεί πρόωρα η αναζήτηση του αλγορίθμου, οπότε και θα προκύψει μια μερική διαδρομή. Υπό φυσιολογικές συνθήκες, ο αλγόριθμος ολοκληρώνεται όταν εντοπίσει τον κόμβο στόχου. Ωστόσο, αν έχει προσδιορίσει το μονοπάτι έως οποιαδήποτε βάση πριν τον στόχο, θα μπορεί να επιστρέψει ως αποτέλεσμα, το προσωρινό μονοπάτι που προσδιόρισε, το οποίο εκτείνεται μέχρι τον τρέχοντα κόμβο. Εν τέλει, ο αλγόριθμος έχοντας βρει ένα

τμήμα της βέλτιστης διαδρομής μπορεί να συνεχίσει την αναζήτηση της υπόλοιπης, θέτοντας ως αφετηρία τον κόμβο στον οποίο τερματίστηκε η αναζήτηση στην προηγούμενη προσπάθεια.

13.3.3.Ομαδική Κίνηση

Υπάρχει περίπτωση να υπάρχουν περισσότερες από μία κινούμενες μονάδες η οποίες κινούμενες ως σύστημα, θα έχουν κοινό στόχο. Είναι πολύ πιθανό το βέλτιστο μονοπάτι που θα υπολογιστεί για τη μία κινούμενη μονάδα, να χρησιμεύσει και στις υπόλοιπες. Αυτό μπορεί να επιτευχθεί υπολογίζοντας το μονοπάτι με αφετηρία το κέντρο του συστήματος και στόχο το κέντρο των προορισμών. Έπειτα για κάθε ξεχωριστή μονάδα κίνησης, θα πρέπει να υπολογιστεί η απόσταση που θα πρέπει να διανυθεί ως την κεντρική αυτή αφετηρία στην αρχή, και τον στόχο στο τέλος της διαδρομής.

Οι ατομικές διαδρομές που βρίσκονται για κάθε μονάδα είναι σύντομες (μέσο όρο περίπου 10 μέτρα), και η κύρια διαδρομή, που είναι μακρύτερη, είναι κοινή σε όλο το σύστημα. Επομένως το μεγαλύτερο τμήμα της διαδρομής υπολογίζεται μόνο μία φορά. Ακόμη, προκειμένου να αποφευχθεί ο κίνδυνος συγκρούσεων είναι δυνατόν η διαδρομή κάθε μονάδας να ρυθμιστεί ώστε το μονοπάτι που θα ακολουθήσει να είναι ελαφρώς διαφορετικό.

Μία μέθοδος για να επιτευχθεί αυτό, είναι η επιλογή εφαιπόμενων, προς το κύριο μονοπάτι, κόμβων. Άλλη προσέγγιση είναι το να δοθεί η δυνατότητα στις μονάδες να αναγνωρίζονται μεταξύ τους, ορίζοντας κάποια κινούμενη μονάδα ως κεφαλή του συστήματος, η οποία και θα ηγείται της κίνησης. Η κεφαλή θα προσδιορίζει το μονοπάτι για τη δική της κίνηση μόνο, ενώ οι υπόλοιπες μονάδες θα πρέπει να χρησιμοποιήσουν ένα αλγόριθμο flocking (αλγόριθμος συρροής) ο οποίος θα τις αναγκάζει να κινούνται ως σμήνος.

ΣΥΜΠΕΡΑΣΜΑΤΑ

Στην παρούσα εργασία αναπτύχθηκε κώδικας αναζήτησης βέλτιστης διαδρομής, με σκοπό την εφαρμογή του σε κινούμενη ρομποτική μονάδα. Από τις προσομοιώσεις (Παράρτημα II) γίνεται κατανοητό ότι κάθε τύπο εδάφους παρουσιάζονται διαφορετικές απαιτήσεις, και αντίστοιχα, οι λύση κάθε προβλήματος μπορεί να εξαρτάται από διαφορετική ευρετική συνάρτηση.

Οι τρόποι προσέγγισης των επιμέρους δομών του προγράμματος ποικίλουν. Κατά την ανάπτυξή τους, λήφθηκαν υπόψη οι ιδιαίτερες απαιτήσεις του αλγορίθμου, προκειμένου η υλοποίησή τους να είναι όσο το δυνατόν αποδοτικότερη. Εξαίρεση αποτελεί η δομή του χάρτη, στην οποία προτεραιότητα δόθηκε στην ευανάγνωστη μέθοδο προσομοίωσης.

ΠΑΡΑΡΤΗΜΑ Ι

ΠΡΟΓΡΑΜΜΑ C++

1. ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ

```
#include "Map_header.h"
#include <iostream>
#include <iomanip>
using namespace std;

class Map{
private:
    int height;           //σειρές
    int width;            //στήλες
    int *nodes;           //dynamic array αποθηκεύσει τιμές σε μορφή πίνακα
public:
    Map();                //constructor
    ~Map();               //destructor
    Map(int r, int c);    //constructor με παραμέτρους
    void Map_add(int, int); //πρόσθεση τιμής σε συγκεκριμένο κόμβο
    void Map_replace(int, int); //αλλαγή τιμής διερευνημένου ή μη κόμβου
    bool Map_exists(int); //επιστρέφει true αν ο κόμβος είναι διάφορος 0
    int Map_find_value(int); //επιστρέφει αποθηκευμένη τιμή του κόμβου
    void print();         //τυπώνει τον χάρτη υπό μορφή πίνακα
};

Map::Map(){
    height=0;
    width=0;
    nodes=NULL;
}

Map::~Map(){
    delete[] nodes;
}

Map::Map(int r, int c){
    height=r;
    width=c;
    nodes=new int[r*c];
    for (int i=0;i<r*c;i++){
        nodes[i]=0;}
}

void Map::Map_add(int currentNode, int addValue){
    if(!nodes[currentNode])
        nodes[currentNode]=addValue;
    return;
}

void Map::Map_replace(int currentNode, int addValue){
    nodes[currentNode]=addValue;
    return;
}

bool Map::Map_exists(int currentNode){
    if(nodes[currentNode])
        return true;
}

int Map::Map_find_value(int currentNode){
    int v;
    v=nodes[currentNode];
}
```

```
        cout<<v;
        return v;
    }
    void Map::print(){
        for (int i=0;i<height*width;i++){
            cout<<setw(5)<<nodes[i];
            if((i+1)%width==0)
                cout<<"\n\n"; //end of row
        }
        return;
    }
}
```

2.ΒΙΒΛΙΟΘΗΚΗ ΚΛΕΙΣΤΗΣ ΛΙΣΤΑΣ

```
#include "List_header.h"
#include <iostream>
using namespace std;

class List{
public:
    List(); //constructor
    ~List(); //destructor
    void List_add(int); //προσθέτει νέα τιμή στην ουρά της λίστας
    int List_exists(int); //ελέγχει αν υπάρχει στη λίστα το όρισμα
    bool List_empty(); //ελέγχει αν η λίστα είναι κενή
    int pop(); //αφαιρεί και επιστρέφει την κεφαλή της λίστας
    void Remove_last(); //αφαιρεί την ουρά της λίστας
    void clear(); //άδειασμα της λίστας, εκτός του head
    bool List_corner(int, int); //έλεγχος για ύπαρξη γωνίας
    int List_returnTail(); // επιστροφή ουράς λίστας
    void print(); //εκτύπωση λίστας
private:
    typedef struct Node{ //βασική δομή από την οποία αποτελείται η λίστα
        Node *next;
        int data;
        int position;
    }NODE;
    NODE* head;
    int List_returnPretail(); //επιστροφή προτελευταίου στοιχείου λίστας
    void List_reverse(); //αντιστροφή ακολουθίας στοιχείων λίστας
    int List_getSecond(); //επιστροφή στοιχείου μετά το head
};

List::List(){
    head=NULL;
}

List::~List(){
    if(head==NULL)return;
    NODE *cur=head;
    while(head->next){
        head=head->next;
    }
    if(head!=NULL){
        delete cur;
        head=NULL;
    }
}

void List::List_add(int value){
    if(head==NULL){
        head= new Node;
        head->data=value;
        head->next=NULL;
        return;
    }
    NODE *cur=head;
    while (cur){
        if(cur->next==NULL){
            NODE *ptr= new NODE;
            ptr->data=value;
            ptr->next=NULL;
            cur->next=ptr;
        }
    }
}
```

```

        return;
    }
    cur=cur->next;
}
}

int List::List_exists(int value){
    NODE *cur=head;
    if(head==NULL){
        cout<<"The List is empty."<<endl;
        return -1;
    }
    while (cur!=NULL){
        if(cur->data==value){
            cout<<"The value already exists!";
            return 1;
        }
        else{
            cur=cur->next;
        }
    }
    if(cur==NULL){
        cout<<"The value does not exist!"<<endl;
        return -1;
    }
}

bool List::List_empty(){
    if(head==NULL){
        return true;
    }
    else{
        return false;
    }
}

int List::pop(){
    if(head==NULL){
        return -1;
    }
    NODE *tmp=head;
    int value=head->data;
    if(head->next){
        head=head->next;
    }
    else{
        delete tmp;          //pop head
        head=NULL;
    }
    return value;
}

void List::print(){
    if(head==NULL)return;
    Node *cur=head;
    while(cur){
        cout<<cur->data<<" ";
        cur=cur->next;
    }
    cout<<endl;
}

bool List::List_corner(int cur, int width){

```

```

int tail;
tail=List_returnTail();
int pretail;
pretail=List_returnPretail();
if ((tail>0)&&(pretail>0)){
    if((tail==cur-width)&(pretail==cur-width-1)){return true;}
    if((tail==cur-width)&(pretail==cur-width+1)){return true;}
    if((tail==cur+width)&(pretail==cur+width-1)){return true;}
    if((tail==cur+width)&(pretail==cur+width+1)){return true;}
    if((tail==cur+1)&(pretail==cur-width+1)){return true;}
    if((tail==cur+1)&(pretail==cur+width+1)){return true;}
    if((tail==cur-1)&(pretail==cur-width-1)){return true;}
    if((tail==cur-1)&(pretail==cur+width-1)){return true;}
    else{ return false;}
}else{return false;}
}
void List::Remove_last(){
    if(head==NULL){
        cout<<"Empty stack!"<<endl;
        return;
    }
    NODE *tmp=head;
    NODE *cur;
    if(head->next==NULL){
        cout<<"Cannot delete startNode"<<endl;
        return;
    }
    while(tmp->next!=NULL){
        cur=tmp;
        tmp=tmp->next;
    }
    cur->next=NULL;
    delete tmp;
}
void List::clear(){
    head->next=NULL;
    return;
}
int List::List_returnTail(){
    if(head==NULL){
        return -1;
    }
    NODE *cur=head;
    int value;
    while (cur!=NULL){
        if(cur->next==NULL){
            value=cur->data;
            cout<<"Last is: "<<value<<endl;
            return value;
        }
        else{
            cur=cur->next;
        }
    }
}
int List::List_returnPretail(){
    int Pretail;
    if((head==NULL)|| (head->next==NULL)){
        cout<<"-1"<<endl;
    }
}

```

```

        return -1;
    }
    else{
        List_reverse();
        Pretail=List_getSecond();
        List_reverse();
        cout<<"Pretail is: "<<Pretail<<endl;

        return Pretail;
    }
}

void List::List_reverse(){
    NODE *startList=head;
    NODE *midList=startList->next;
    NODE *reversed=midList->next;

    startList->next=NULL;
    midList->next=head;///  
reverse pointer direction
    while(reversed!=NULL){        ///  
//reversed
        midList->next=startList;
        startList=midList;
        midList=reversed;
        reversed=reversed->next;
    }

    head=midList;
    head->next=startList;
}

int List::List_getSecond(){
    if(head==NULL){
        cout<<"Empty stack!"<<endl;
        return -1;
    }
    NODE *tmp=head->next;
    int value=tmp->data;
    cout<<value<<endl;
    return value;
}

```

3. ΒΙΒΛΙΟΘΗΚΗ ΛΙΣΤΑΣ ΠΡΟΤΕΡΑΙΟΤΗΤΑΣ

```
#include "Priority_header.h"
#include <iostream>

using namespace std;

class Priority{
public:
    Priority();           //constructor
    ~Priority();          //destructor
    void Priority_clear(); //άδειασμα της λίστας προτεραιότητας
    void Priority_add(int,int); //προσθήκη κόμβου και προτεραιότητάς του
    void Priority_change(int,int); //αλλαγή της προτεραιότητας ενός κόμβου
    int Priority_getHead(); //επιστροφή τιμής head
    int pop();             //επιστρέφει τη κεφαλή της λίστας
    bool empty();          //ελέγχει αν η λίστα είναι άδεια
    void print();          //τύπωση του περιεχομένου της λίστας
private:
    typedef struct Node{ //βασική δομή από την οποία απαρτίζεται η λίστα
        Node *next;
        Node *previous;
        int data;
        int queue;
    }NODE;
    NODE* head;
    void sort(); //κατατάσσει στοιχεία με αύξοντα αριθμό
    void change(int,int); //αλλάζει την προτεραιότητα
};

Priority::Priority(){
    head=NULL;
}

Priority::~~Priority(){
    if(head==NULL)return;
    NODE *cur=head;
    while(head->next){
        head=head->next;
    }
    if(head!=NULL){
        delete cur;
        head=NULL;
    }
}

void Priority::Priority_clear(){
    if(head==NULL) return;
    NODE *cur=head;
    while(head->next){
        head=head->next;
    }
    if(head!=NULL){
        delete cur;
        head=NULL;
    }
}

void Priority::sort(){
    int temp;
    for(NODE *i=head; i->next!=NULL;i=i->next){
        for(NODE *j=i->next;j!=NULL;j=j->next){
            if(i->queue>j->queue){
```

```

        temp=i->queue;
        i->queue=j->queue;
        j->queue=temp;
        temp=i->data;
        i->data=j->data;
        j->data=temp;
    }
}
}
}
void Priority::add(int value,int priority){
    if(head==NULL){
        head= new Node;
        head->data=value;
        head->next=NULL;
        head->queue=priority;
        return;
    }
    NODE *cur=head;

    NODE *ptr= new NODE;
    ptr->data=value;
    ptr->queue=priority;

    if((cur->queue)<priority){
        while (cur){
            if(cur->next==NULL){
                ptr->next=NULL;
                cur->next=ptr;

                return;
            }
            cur=cur->next;
        }
    }else{
        ptr->next =head;
        head = ptr;
    }
}
void Priority::change(int value, int newPrior){
    NODE *cur=head;
    while(cur){
        if(cur->data==value){
            cur->queue=newPrior;
        }
        cur=cur->next;
    }
}
void Priority::Priority_change(int value,int newPrior){
    change( value,newPrior);
    sort();
}
int Priority::Priority_getHead(){
    if(head==NULL){
        return -1;
    }
    NODE *tmp=head;
    int value=head->data;
    cout<<value<<endl;;
}

```

```

        return value;
    }
    int Priority::pop(){
        if(head==NULL){
            return -1;
        }
        NODE *tmp=head;
        int value=head->data;
        if(head->next){
            head=head->next;
        }
        else{
            delete tmp;
            head=NULL;
        }
        return value;
    }
    bool Priority::empty(){
        if(head==NULL){
            return true;
        }
        else{
            return false;
        }
    }
    void Priority::print(){
        if(head==NULL) return;
        Node *cur=head;
        while(cur){
            cout<<cur->data<<" ";
            cur=cur->next;
        }
        cout<<endl;
    }
}

```

4.ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ ΕΜΠΟΔΙΩΝ

```
#include "Walkable.h"
#include <iostream>
#include <iomanip>
using namespace std;

class Walkable{
private:
    int height;           //γραμμές
    int width;            //στήλες
    int *nodes;           //dynamic array αποθηκεύσει τιμές σε μορφή πίνακα
public:
    Walkable();            //constructor
    ~Walkable();           //destructor
    Walkable(int, int);    //constructor με παραμέτρους
    bool WallNode(int);    //εισαγωγή εμποδίου σε κάποιο κόμβο
    void Walkable_add(int, int); //ανάθεση τιμής σε εμπόδιο
    bool WallCheck(int);   //ελέγχει αν κόμβος είναι προσπελάσιμος
    void print();          //τύπωση πίνακα
};

Walkable::Walkable(){
    height=0;
    width=0;
    nodes=NULL;
}

Walkable::~Walkable(){
    delete[] nodes;
}

Walkable::Walkable(int r, int c){
    height=r;
    width=c;
    nodes=new int[r*c];
    for (int i=0;i<r*c;i++){
        nodes[i]=true;}
}

bool Walkable::WallNode(int currentNode){
    nodes[currentNode]=false;
    return false;
}

void Walkable::Walkable_add(int currentNode, int addValue){
    nodes[currentNode]=addValue;
    return;
}

bool Walkable::WallCheck(int currentNode){
    if(nodes[currentNode]==false){
        return false;
    }else{
        return true;
    }
}

void Walkable::print(){
    for (int i=0;i<height*width;i++){
        cout<<setw(5)<<nodes[i];
        if((i+1)%width==0)
            cout<<"\n\n";
    }
}
```

5. ΚΟΡΜΟΣ ΠΡΟΓΡΑΜΜΑΤΟΣ (MAIN)

```
#include "List_header.h"
#include "Map_header.h"
#include "Priority_header.h"
#include "Walkable.h"
#include <iostream>
#include <cmath>

using namespace std;

int fieldWidth=15;
int fieldHeight=15;

int Key(int &KeyX, int &KeyY){
    int r;
    r=KeyY*fieldWidth+KeyX;
    return r;
}
int KeyX(int &Key){
    int r;
    r=Key%fieldWidth;
    return r;
}
int KeyY(int &Key){
    int r;
    r=Key/fieldHeight;
    return r;
}
int max(int num1,int num2){
    int r;
    if(num1>=num2){r=num1;}
    else{r=num2;}
    return r;
}
int min(int num1,int num2){
    int r;
    if(num1>=num2){r=num2;}
    else{r=num1;}
    return r;
}
int Movement(int Cpop, int old ){
    int neo;
    neo = Cpop;
    if (neo== old){ return 0; }           //Do nothing
    if (neo== old+1){ return 1; }         //Drive East
    if (neo== old-1){ return 2; }         //Drive West
    if (neo== old-fieldWidth){ return 3; } //Drive North
    if (neo== old+fieldWidth){ return 4; } //Drive South
    if (neo== old-fieldWidth+1){ return 5; } //Drive North East
    if (neo== old-fieldWidth-1){ return 6; } //Drive North West
    if (neo== old+fieldWidth+1){ return 7; } //Drive South East
    if (neo== old+fieldWidth-1){ return 8; } //Drive South West
    else { return -1;}                   //error
}
int main(int argc, char** argv) {
    //Μετατροπή συντεταγμένων χώρου σε συντεταγμένες πλέγματος
    int startX, startY, endX, endY;
    Map H(fieldWidth,fieldHeight);
```

```

Map G(fieldWidth,fieldHeight);
Map P(fieldWidth,fieldHeight);
Map B(fieldWidth,fieldHeight);
Walkable wallMap(fieldWidth,fieldHeight);
Priority F;
List C;

/*_____ΕΙΣΑΓΩΓΗ ΒΑΣΙΚΩΝ ΠΛΗΡΟΦΟΡΙΩΝ ΧΑΡΤΗ_____*/
//Ορισμός εμποδίων
wallMap.WallNode(37); wallMap.WallNode(52); wallMap.WallNode(67);
wallMap.WallNode(82); wallMap.WallNode(97); wallMap.WallNode(112);
wallMap.WallNode(127); wallMap.WallNode(142); wallMap.WallNode(157);
wallMap.WallNode(172);wallMap.WallNode(187);

for(int wall=0;wall<=fieldWidth*fieldHeight; wall++){
    if(wallMap.WallCheck(wall)==false){
        H.Map_add(wall, 111);
        G.Map_add(wall, 111);
        P.Map_add(wall, 111);
    }
}

int startNode;
cout<<"Please enter starting node's position: ";
cin>> startNode;
if(wallMap.WallCheck(startNode)){    //startNode ≠ wallnode
    H.Map_add(startNode,11);
}
while(!wallMap.WallCheck(startNode)){
    cout<<"Start node's position cannot be approached. Please enter another start
    node's position: ";
    cin>>startNode;
    H.Map_add(startNode,11);
}

int endNode;
cout<<"Please enter target node's position: ";
cin>>endNode;
if (wallMap.WallCheck(endNode)){    //endNode ≠ wallnode
    H.Map_add(endNode,11);
}
while(!wallMap.WallCheck(endNode)){
    cout<<"Target node's position cannot be approached. Please enter another
    target node's position: ";
    cin>>endNode;
    H.Map_add(endNode,11);
}

/*_____ΑΡΧΙΚΟΠΟΙΗΣΗ_____*/
startX=KeyX(startNode); startY=KeyY(startNode); endX=KeyX(endNode);
endY=KeyY(endNode);
int curX;
curX=startX; //curx= τρέχον x - ο κόμβος που διερευνάται
int curY;
curY=startY;
int curNode;
curNode=Key(curX,curY);
G.Map_add(curNode, 11);    // το όρισμα εισάγει τον #κόμβου και την τιμή του
P.Map_add(curNode, 11);

```

```

P.Map_add(endNode, 11);
G.print();

/**_____ΑΛΓΟΡΙΘΜΟΣ_____*/
bool searching; //searching και found προσβάσιμες σε όλο το πρόγραμμα
bool found;

do{
    //1.Προσθήκη τρέχοντος κόμβου στη Κλειστή Λίστα (για αποφυγή κύκλων)
    C.List_add( Key(curX,curY));

    //2.Διερεύνηση εφαπτόμενων στον τρέχοντα κόμβων
    //σύγκριση μόνο των 8 κόμβων
    int distFromStartToCurrent; //απόσταση αφετηρίας μέχρι τον τρέχοντα
    distFromStartToCurrent=G.Map_find_value(Key(curX,curY));

    //οριοθέτηση πλέγματος ώστε να μην υπολογίζονται περιοχές έξω από το πλέγμα
    for (int i=max(0,curX-1);i<=min(fieldWidth-1,curX+1);i++){
        for(int j=max(0,curY-1);j<=min(fieldHeight-1,curY+1);j++){
            if((i==curX) && (j==curY)){continue;}//εξαιρείται ο τρέχων κόμβος
            if(Key(i,j)==endNode){ //συνθήκη κύριου βρόχου
                C.List_add(endNode);
                searching=false;
                found=true;
            }

            //η διαγώνιος κίνηση γίνεται πάντοτε σε κόμβους με περιττό ή άρτιο αριθμό
            bool diagonal;
            if(((curX+curY)%2)==((i+j)%2)){
                diagonal=true;
            }
            else{diagonal=false;
            }

            int distFromCurrentToNextNode=0; //κόστος βηματισμού
            if (diagonal){
                distFromCurrentToNextNode=14; //κόστος διαγώνιου βηματισμού
            }else{
                distFromCurrentToNextNode=10; //κόστος οριζόντιας/κάθετης κίνηση
            }

            int step;
            step=C.List_exists(Key(i,j));
            if((step==-1)&&(wallMap.WallCheck(Key(i,j))==true)){
                /*_____ΥΠΟΛΟΓΙΣΜΟΣ_____G,H,F_____*/
                int tempG;
                tempG=distFromStartToCurrent+distFromCurrentToNextNode;
                int tempB;
                tempB=B.Map_find_value(Key(i,j));

                int tempH; //εισαγωγή επιλεγμένης ευρετικής(diagonal)
                int dx, dy;
                dx=abs(i-endX); dy=abs(j-endY);
                tempH=(10*(dx+dy))+((14-2*10)*min(dx,dy));

                int tempF;
                tempF=tempH+tempG;//+tempB;
                G.Map_add(Key(i,j),tempG);
                H.Map_add(Key(i,j),tempH);
            }
        }
    }
}

```

```

        F.Priority_add(Key(i,j),tempF);
    }
}
int lowF=-1;
lowF=F.Priority_getHead();           //Επιστροφή κόμβου υψηλότερης προτεραιότητας

//Έλεγχος για το αν σχηματίζεται ορθή γωνία στο μονοπάτι και διόρθωση
if(C.List_corner(lowF, fieldWidth)){
int tail;
tail=C.List_returnTail();
wallMap.WallNode(tail);
C.Remove_last();
}

    //3. Επιλογή καλύτερου κόμβου
    int minF=-1;
    minF=F.pop();
    curX=KeyX(minF);
    curY=KeyY(minF);
    curNode=minF;
    F.Priority_clear();
}while(searching);           //Ολοκλήρωση κύριου βρόχου Αλγορίθμου

//Επιστροφή αποτελέσματος
int old=startNode;
List smooth;
while(!C.List_empty()){
    int path;
    int direction, oldirect;
    path=C.pop();
    P.Map_add(path,1);
    direction=Movement(path, old);
    old=path;
    if((direction==oldirect)&& (direction!=0)){
        smooth.Remove_last();
        smooth.List_add(path);
    }
    else{
        smooth.List_add(path);
    }
    oldirect=direction;
}
cout<<"printing B : \n";B.print();
return 0;
}

```

ΠΑΡΑΡΤΗΜΑ II

ΠΡΟΣΟΜΟΙΩΣΗ

1) ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ

Το πλέγμα στην προσομοίωση έχει ξεκάθαρα χαρακτηριστικά. Αποτελείται από ευδιάκριτες γραμμές και στήλες, όπως ακριβώς ένας πίνακας. Η αρίθμηση των θέσεων του πλέγματος ξεκινάει από μηδέν και αυξάνει συμπληρώνοντας πρώτα τις θέσεις των γραμμών και καθώς αναπτύσσεται των στηλών.

Κατά την προσομοίωση του πλέγματος, μελετάται η συμπεριφορά των συναρτήσεων. Η συνάρτηση Map_add(key, value) προσθέτει μία τιμή (value) σε μία θέση του πλέγματος (key), δεν μπορεί όμως να προσθέσει τιμή σε έναν κόμβο ο οποίος διαθέτει ήδη τιμή. Αυτό επιτυγχάνεται από τη συνάρτηση Map_replace(key, value).

```
The Map is:
0 0 0
0 0 0
0 0 0

Add element on node: 4
The Map is:
0 0 0
0 12 0
0 0 0

Add element on node: 4
The Map is:
0 0 0
0 12 0
0 0 0

The Map is:
0 0 0
0 12 0
0 0 0

Replace element of node: 4
The Map is:
0 0 0
0 8 0
0 0 0

Replace element of node: 0
The Map is:
9 0 0
0 8 0
0 0 0

Check if element in node exists: 0
The element exists!
Check if element in node exists: 3
The element does not exist!
Check the value in node: 4
The stored value is: 8
Check the value in node: 8
The stored value is: 0
9 0 0
0 8 0
0 0 0

Process exited after 38.02 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . .
```

Το πλέγμα επιλέχθηκε να έχει διαστάσεις 3Χ3, ώστε λόγω μικρού μεγέθους, να φανεί η λειτουργία των συναρτήσεων του. Η ακαθόριστη τιμή του πλέγματος είναι η 0, και δηλώνει ότι η θέση του πλέγματος με τη συγκεκριμένη τιμή δεν έχει διερευνηθεί. Η μεταγλώττιση διήρκεσε 4, 16s.

Συγκριτικά ανάμεσα στις δομές που συνιστούν τον A*, ο χάρτης είναι η πιο χρονοβόρα δομή που υλοποιήθηκε.

2) ΒΙΒΛΙΟΘΗΚΗ ΚΛΕΙΣΤΗΣ ΛΙΣΤΑΣ

Τα στοιχεία της κλειστής λίστας είναι οι συντεταγμένες των θέσεων από τις οποίες διήλθε η κινούμενη μονάδα (κόμβοι), με την σειρά ακριβώς που επιβάλλει ο βηματισμός. Κάθε στοιχείο προστίθεται στο τέλος (την ουρά) της λίστας.

Στην επόμενη προσομοίωση περιγράφεται αναλυτικά η διαδικασία και οι ενέργειες που εκτελούν οι συναρτήσεις. Οι συναρτήσεις επιστρέφουν -1 όταν η συνθήκη είναι αναληθής. Η μεταγλώττιση διήρκεσε 4,16s.

Η προσομοίωση αρχίζει με τη συνάρτηση που αφαιρεί το τελευταίο στοιχείο της λίστας, και τη συνάρτηση που ελέγχει εάν η λίστα είναι άδεια, προκειμένου να αποδειχθεί ότι η κλήση αυτών των συναρτήσεων δεν εξαρτάται από μία συγκεκριμένη μορφή της λίστας.

```
Pop tail: The list is Empty!
Does 9 exist on list? : The List is empty.
Adding elements to the list:
5
5 6
Pop tail: 6
The list is : 5
Adding elements to the list:
5 12
5 12 98
The list is:
5 12 98 1000
Pretail is: 98
Last is: 1000
The list is: 5 12 98 1000

Does 5 exist on list? : The value already exists!
Does 1000 exist on list? : The value already exists!
Does 12 exist on list? : The value already exists!
Does 100 exist on list? : -1
Does 6 exist on list? : -1
Does 98 exist on list? : The value already exists!
Does 500 exist on list? : -1

Pop head of the list:
12 98 1000
Pop head: 98 1000
Pop tail: 1000
The list is : 98
Pop head:
The list is Empty!
Pop tail: The list is Empty!
Pop head: Empty stack!
Checking for corner of current node 10:
-1
Corner is not formed
The list is :36
Checking for corner of current node 52:
Last is: 36
-1
Corner is not formed
The list is :36 37
Checking for corner of current node 52:
Last is: 37
Pretail is: 36
Corner is formed
Checking for corner of current node 10:
Last is: 37
Pretail is: 36
Corner is not formed

-----
Process exited after 0.1087 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . . _
```

3) ΛΙΣΤΑ ΠΡΟΤΕΡΑΙΟΤΗΤΑΣ

Ο έλεγχος της λειτουργίας των συναρτήσεων επιτυγχάνεται προσθέτοντας αρχικά κάποια στοιχεία στη λίστα και παρατηρώντας αν τα στοιχεία με τη μικρότερη προτεραιότητα τοποθετούνται στην κεφαλή της λίστας, και αυτά με τη μικρότερη προτεραιότητα στην ουρά. Έπειτα παρατηρείται η επαλήθευση της λειτουργίας των υπόλοιπων συναρτήσεων.

Προς επαλήθευση της σωστής λειτουργίας τυπώνεται και η προτεραιότητα των στοιχείων που προστέθηκαν στον πίνακα. Σημειωτέον ότι το πρώτο όρισμα στη συνάρτηση `Priority_add(50,50)` είναι το «κλειδί», δηλαδή οι συντεταγμένες της θέσης του χώρου που διερευνάται (ή κόμβος), και το δεύτερο όρισμα είναι η προτεραιότητα, δηλαδή στον αλγόριθμο, το F.

```
Priority F;
F.Priority_add(50,50); F.print();
F.Priority_add(20,25); F.print();
F.Priority_add(10,20); F.print();
F.Priority_add(60,102); F.print();
F.Priority_add(40,33); F.print();
F.Priority_add(30,99);
F.print();
F.pop();F.print();
cout<<endl;
```

Η μεταγλώττιση διήρκησε 4,05s. Τα αποτελέσματα της προσομοίωσης είναι τα ακόλουθα:

```
Adding elements to the list:
50
20 50
10 20 50
10 20 50 60
10 20 50 60 40
10 20 50 60 40 30
pop: 10
20 50 60 40 30

Head node is:
20
Change Priority of second element. The new priority queue is:
60 20 40 50 30

Pop head:
pop: 60
Priority Queue is now:
20 40 50 30
Is the Priority Queue empty?
The priority queue is not empty.
Pop head:
pop: 20
The new priority queue is: 40 50 30
pop: 40
The new priority queue is: 50 30
pop: 50
The new priority queue is: 30
pop: 30
The new priority queue is: Empty stack!
Is the Priority Queue empty?
The priority queue is empty.

-----
Process exited after 0.1877 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . .
```

4) ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ ΕΜΠΟΔΙΩΝ

Ο χάρτης των εμποδίων είναι μία οντότητα «σκιά» του χάρτη του πλέγματος. Κύρια διεργασία που επιτελεί είναι η αντιστοίχιση μίας δυαδικής τιμής, true ή false, σε κάθε κόμβο (θέση του πλέγματος). Αν η τιμή του κόμβου είναι αληθής τότε αυτός είναι προσπελάσιμος. Αν όχι τότε υπάρχει κάποιο εμπόδιο τοποθετημένο σε αυτόν.

```
Add Wall on node: 7
Add Wall on node: 12
Add Wall on node: 17
The Map is:
  1   1   1   1   1
  1   1   0   1   1
  1   1   0   1   1
  1   1   0   1   1
  1   1   1   1   1

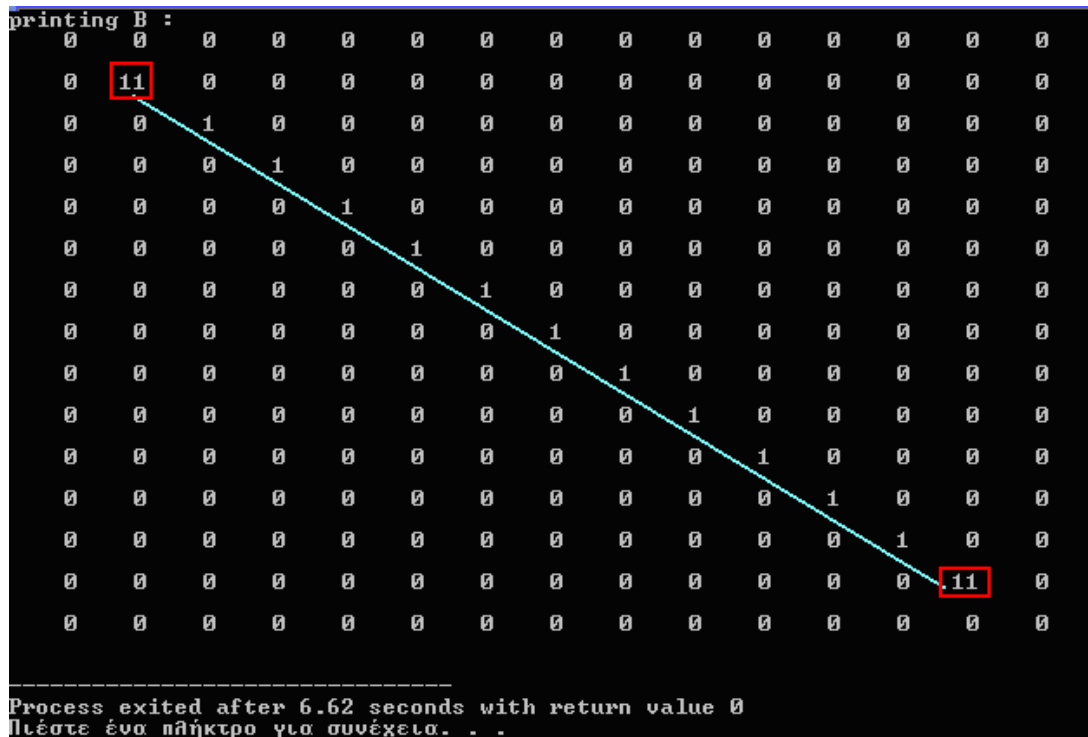
Check if there is Wall on Node: 12
This node is Unwalkable!
Check if there is Wall on Node: 23
There is no Wall here.
Check if there is Wall on Node: 7
This node is Unwalkable!
Point Walls as 111
There is no Wall here.
  1   1   1   1   1
  1   1 111   1   1
  1   1 111   1   1
  1   1 111   1   1
  1   1   1   1   1

-----
Process exited after 12.46 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . . _
```

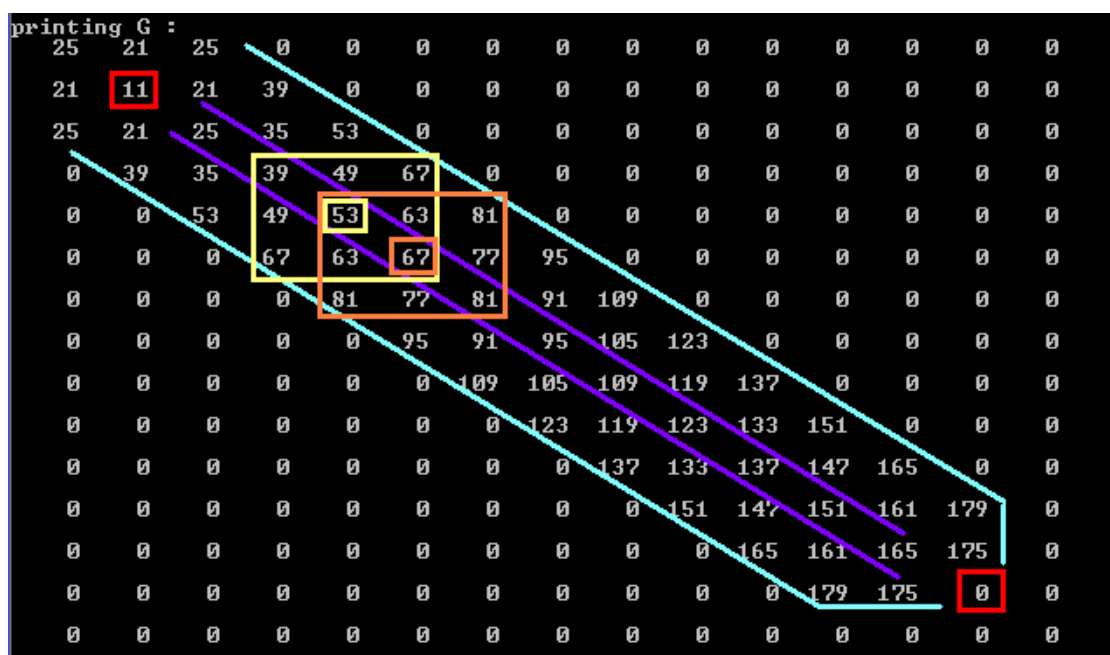
Η αρίθμηση των κόμβων είναι πανομοιότυπη με αυτή του χάρτη πλέγματος.

5) ΚΟΡΜΟΣ ΠΡΟΓΡΑΜΜΑΤΟΣ (MAIN)

Στην προσομοίωση χρησιμοποιήθηκε χάρτης διαστάσεων 15x15, ο οποίος κατέχει συνολικά 224 θέσεις. Αρχικά, πρέπει να μελετηθεί η συμπεριφορά του αλγορίθμου σε χάρτη χωρίς εμπόδια. Η τιμή 0 στον χάρτη υποδηλώνει περιοχή εκτός της διαδρομής, η τιμή 1 τα σημεία της διαδρομής και τα σημεία 11 την αφετηρία και τον στόχο.



Παρατηρούμε ότι ο αλγόριθμος εντοπίζει το βέλτιστο μονοπάτι. Προκειμένου όμως να φανεί εάν δαπανά περιττό χρόνο στον υπολογισμό του μονοπατιού πρέπει να μελετήσουμε την περιοχή διερεύνησης:



Με κόκκινο σημειώνονται η αφετηρία και ο στόχος, η μεθόριος με γαλάζιο και τα σημεία του μονοπατιού με μωβ. Παρατηρώντας την διερεύνηση δύο διαδοχικών βάσεων (κίτρινο και πορτοκαλί), συμπεραίνουμε ότι η περιοχή διερεύνησης είναι η μικρότερη δυνατή. Η αναζήτηση κατέλαβε 1,3 MiB στη μνήμη.

Ακολουθεί προσομοίωση με προσεκτική επιλογή θέσεων αφετηρίας και στόχου που δε βρίσκονται πάνω στη διαγώνιο.

```

printing path :
printing B :
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 11 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 1 1 1 1 11 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
-----
Process exited after 6.286 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . .

```

Παρατηρούμε ότι το μονοπάτι είναι συμμετρικό και βέλτιστο. Η ακολουθία των επιλεγόμενων βάσεων είναι ιδανική για την κίνηση της μονάδας καθώς αποτελείται από δύο ευθύγραμμα τμήματα. Η αναζήτηση κατέλαβε επίσης 1,3 MiB μνήμης.

Στη συνέχεια μελετάται η συμπεριφορά του αλγορίθμου σε περιοχή με εμπόδιο, όπου η αφετηρία και ο στόχος βρίσκονται στη ίδια ευθεία. Η περίπτωση αυτή παρουσιάζει ιδιαίτερη δυσκολία:

```

printing B :
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 11 1 1 1 1 0 111 0 0 0 11 0 0 0
0 0 0 0 0 0 1 111 0 0 0 1 0 0 0
0 0 0 0 0 0 1 111 0 0 0 1 0 0 0
0 0 0 0 0 0 1 111 0 0 1 0 0 0 0
0 0 0 0 0 0 1 111 0 1 0 0 0 0 0
0 0 0 0 0 0 1 111 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
-----
Process exited after 7.008 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια.

```

Προκύπτουν τα αναμενόμενα αποτελέσματα. Μόλις έρχεται αντιμέτωπος με το εμπόδιο, ο αλγόριθμος αποφεύγει την καταχώρηση ορθής γωνίας. Εξαιτίας του γεγονότος ότι η αφετηρία και ο στόχος βρίσκονται στην ίδια ευθεία ενώ το εμπόδιο διχοτομεί την περιοχή αναζήτησης, το πρώτο τμήμα του μονοπατιού δεν είναι βέλτιστο. Οποιαδήποτε μετατροπή της ευρετικής συνάρτησης, ενώ βελτιώνει το τμήμα της διαδρομής μπροστά από το εμπόδιο, αλλοιώνει τα αποτελέσματα της αναζήτησης του δεύτερου τμήματος της διαδρομής. Αυτό οφείλεται στην έλξη που ασκεί ο στόχος κατά μήκος της ευθείας. Η όλη διαδικασία κατέλαβε 1,3 MiB στη μνήμη και η μεταγλώττιση διήρκησε 3,84s.

Σημειωτέον, ότι σε περίπτωση χρήσης άλλης απόστασης αντί της Diagonal, το μονοπάτι που προκύπτει δεν είναι συμμετρικό ούτε βέλτιστο. Σε χάρτη στον οποίο επιτρέπεται η κίνηση προς 8 κατευθύνσεις, είναι η απόσταση Diagonal που συνήθως αποφέρει τα καλύτερα αποτελέσματα. Παρακάτω μελετάται και η περίπτωση, κατά την οποία κάποια άλλη ευρετική συνάρτηση κρίνεται αποδοτικότερη.

Στη συνέχεια μελετάται χάρτης στον οποίον ο στόχος δεν είναι προσβάσιμος. Όλες οι ευρετικές συναρτήσεις έπειτα από κάποια διερεύνηση ολοκληρώνουν τη διαδικασία επιστρέφοντας μήνυμα :

```

printing F :
Destination is unreachable

```

Ωστόσο είναι η Ευκλείδεια απόσταση που αντιλαμβάνεται ταχύτερα ότι ο προορισμός δεν είναι προσεγγίσιμος. Η όλη διαδικασία κατέλαβε 1,3 MiB μνήμης και η μεταγλώττιση διήρκησε 7,94s.

```

printing B :
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 0 0 0 0 0 0
0 0 0 0 0 0 0 111 0 1 1 1 0 0 0
0 0 0 0 0 0 0 111 1 111 111 111 1 0 0
0 0 0 11 1 0 111 1 111 11 111 1 0 0
0 0 0 0 0 0 1 111 1 111 111 111 1 0 0
0 0 0 0 0 0 1 111 0 1 1 1 0 0 0
0 0 0 0 0 0 1 111 1 0 0 0 0 0 0
0 0 0 0 0 0 1 111 1 0 0 0 0 0 0
0 0 0 0 0 0 1 111 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

-----
Process exited after 3.974 seconds with return value 0
Πέρας έτος πλάτος για συνέχεια

```

Ακολουθεί προσομοίωση σε κοίλο εμπόδιο το οποίο διαχωρίζει στα δύο την περιοχή διερεύνησης. Παρατηρούμε ότι η απόσταση Diagonal συμπεριφέρεται ιδανικά.

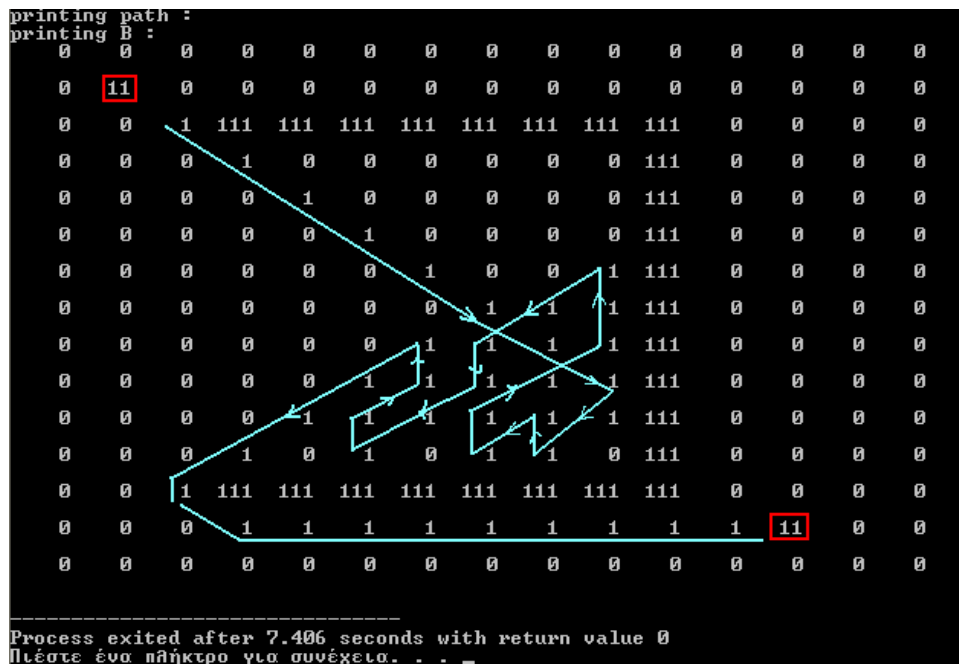
```

printing path :
printing B :
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 11 1 1 1 1 1 1 0 0 0 0
111 111 111 111 111 111 111 111 111 111 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 111 1 0 0 0
0 0 0 0 0 1 0 0 0 0 0 111 1 0 0 0
0 0 0 0 1 0 0 0 0 0 111 1 0 0 0
0 0 0 0 1 111 111 111 111 111 111 1 0 0 0
0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

-----
Process exited after 4.625 seconds with return value 0
Πέρας έτος πλάτος για συνέχεια

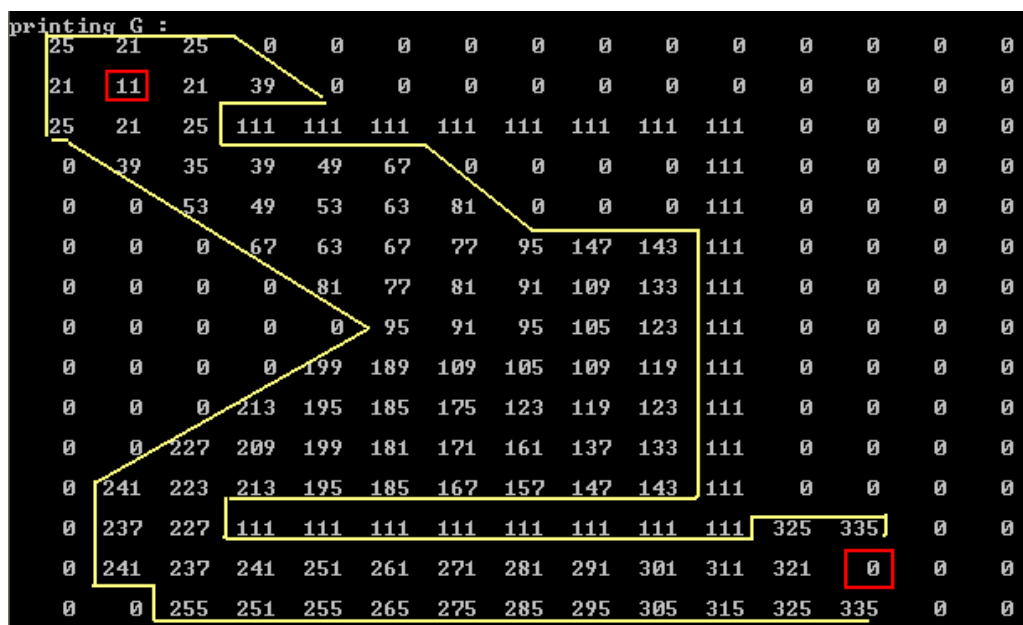
```

Ακολουθεί προσομοίωση για την περίπτωση του κοίλου εμποδίου. Αρχικά χρησιμοποιήθηκε η ευρετική συνάρτηση Diagonal και τα αποτελέσματα που προέκυψαν ήταν απογοητευτικά:



Η διαδρομή που προκύπτει είναι χαοτική και σίγουρα μη βέλτιστη. Με ρύθμιση μόνο της ευρετικής συνάρτησης και του κόστους κίνησης, είναι πρακτικά αδύνατον να βελτιωθεί το συγκεκριμένο αποτέλεσμα. Αντιθέτως, η αλλαγή του πλέγματος σε πλέγμα 4 κατευθύνσεων η οποία συνεπάγεται χρήση της απόστασης Manhattan ως ευρετικής συνάρτησης Η, βελτιώνει σημαντικά την προσδιοριζόμενη διαδρομή όπως φαίνεται στην προσομοίωση που ακολουθεί.

Ωστόσο, η περιορισμένη έκταση της περιοχή διερεύνησης παρουσιάζει ενδιαφέρον, καθώς αποδεικνύει ότι και στη Diagonal απόσταση παρά το γεγονός ότι η διαδρομή είναι μη εφαρμόσιμη, η περιοχή αναζήτησης του Α* παραμένει αρκετά μικρή, και επομένως ο αλγόριθμος είναι ταχύτατος. Η όλη διαδικασία κατέλαβε 1,3 MiB μνήμης και η μεταγλώττιση διήρκεσε 0,28s.



Ακολουθεί προσομοίωση στον ίδιο χάρτη με χρήση απόστασης Manhattan:

```
printing path :
printing B :
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 11 1 1 1 1 1 1 1 1 1 1 1 0 0
0 0 0 111 111 111 111 111 111 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 111 0 1 0 0
0 0 0 111 111 111 111 111 111 111 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 11 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

-----
Process exited after 8.389 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια...
```

Στην περίπτωση αυτή λοιπόν, ο αλγόριθμος παρακάμπτει το κοίλο εμπόδιο χωρίς καμία δυσκολία. Θα μπορούσε να χρησιμοποιηθεί και η απόσταση Diagonal με την προσθήκη μίας επιπλέον συνάρτησης η οποία θα υλοποιούνταν μέσα στη κλάση Walkable. Η συνάρτηση αυτή ελέγχοντας τον χώρο για συνεχή εμπόδια προς κάποια κατεύθυνση, θα μπορούσε να προσδιορίσει εάν υπάρχει κοίλο εμπόδιο στο χάρτη και στη συνέχεια να αυξήσει την εσωτερικά την τιμή του G των κόμβων που βρίσκονται μέσα στο κοίλο εμπόδιο.

Επίσης, θα μπορούσε η ευρετική συνάρτηση H κάθε κόμβου, να υπολογιστεί με βάση την ακριβή απόσταση από τον στόχο, λαμβάνοντας υπόψη τη διαμόρφωση του εδάφους και τα εμπόδια. Και οι δύο αυτές προτάσεις είναι χρονοβόρες, και καταλαμβάνουν πολύ χώρο.

Η παραπάνω διαδικασία κατέλαβε 1,31MiB και η μεταγλώττιση διήρκεσε 0,26s.

Συμπεραίνουμε από τις προσομοιώσεις ότι ενώ κάποια λύση είναι αποτελεσματική για κάποια περίπτωση, σε κάποια άλλη δεν είναι. Ανάλογα με τις απαιτήσεις της περιοχής θα πρέπει να προσαρμόζεται και ο αλγόριθμος.

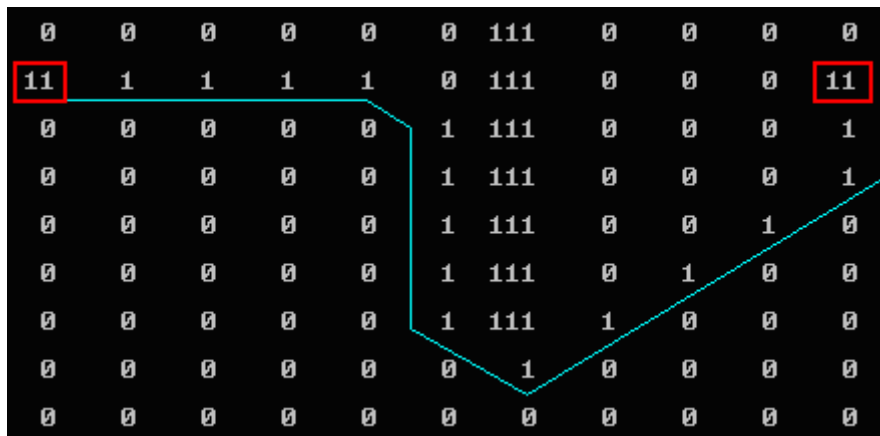
6) ΚΑΤΕΥΘΥΝΣΕΙΣ ΔΙΑΔΡΟΜΗΣ

Τέλος ελέγχεται η λειτουργία της συνάρτησης Movement(), η οποία αντιστοιχεί τις συντεταγμένες πλέγματος σε κατευθύνσεις κίνησης. Κατά τη διαδικασία χρησιμοποιήθηκε ο πρώτος χάρτης που προσομοιώθηκε στη MAIN. Τα αποτελέσματα είναι τα εξής :

```
Do nothing
Drive East
Drive East
Drive East
Drive East
Drive East
Drive South East
Drive South
Drive South
Drive South
Drive South
Drive South East
Drive North East
Drive North East
Drive North East
Drive North East
Drive North
Drive North

-----
Process exited after 5.08 seconds with return value 0
Πιέστε ένα πλήκτρο για συνέχεια. . .
```

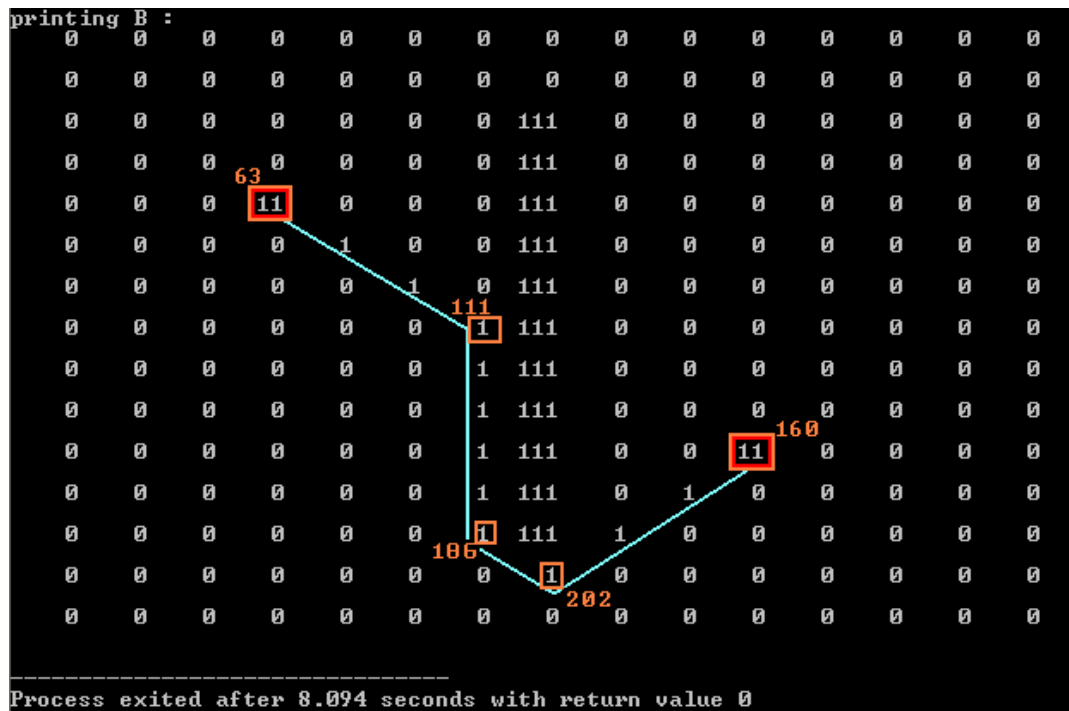
Συγκρίνοντας με το μονοπάτι, γίνεται αντιληπτό ότι υπάρχει απόλυτη ταύτιση ακολουθίας συντεταγμένων και κατευθύνσεων.



Σημειωτέον ότι στην περίπτωση αυτή που ο Αλγόριθμος δε χρειάστηκε να τυπώσει τους χάρτες η διαδικασία της αναζήτησης τερματίστηκε πολύ γρηγορότερα και η μεταγλώττιση διήρκησε μόλις 10,86s.

7) ΕΞΟΜΑΛΥΝΣΗ ΔΙΑΔΡΟΜΗΣ

Ακολουθεί προσομοίωση που ελέγχει κατά πόσο ο κώδικας της εξομάλυνσης της διαδρομής, λειτουργεί σωστά. Αρχικά εικονίζεται το προσδιοριζόμενο μονοπάτι, με σημειωμένα πάνω του τα σημεία πλοήγησης.



Συγκρίνοντας το χάρτη με τα αποτελέσματα του προγράμματος, παρατηρούμε αντιστοιχία σημείων πλοήγησης και σημείων εξομάλυνσης:

```

175 157 175 158 167 170
Do nothing
Drive South East
Drive South East
Drive South East
Drive South
Drive South
Drive South
Drive South
Drive South
Drive South East
Drive North East
Drive North East
Drive North East
printing smooth :
63 111 186 202 160
printing H :

```

ΠΑΡΑΡΤΗΜΑ ΙΙΙ

VHDL

1) ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Grid is
    Port ( clk, reset :in STD_LOGIC;
          Width, Height: in INTEGER;
          startNode, endNode : in INTEGER;
          add, replace, find : in STD_LOGIC;
          findValue : in INTEGER;
          key, value : in INTEGER;
          busy, exists : out STD_LOGIC;
          GridKey, GridValue : out INTEGER);
end Grid;

architecture Behavioral of Grid is
    type struct is record
        dp : STD_LOGIC;
        key : INTEGER;
        value : INTEGER;
    END record struct;
    type grid is array (0 TO Height, Width DOWNT0 0) OF struct;
    signal node: grid;

    type state_type is (ready, inserting);
    signal state: state_type;
begin
    process(clk) begin
        IF rising_edge(clk) THEN
            IF reset='1' THEN
                FOR i IN 0 to Width*Height-1 loop
                    node(i REM Width,i/Height).value<=0;
                    node(i REM Width,i/Height).dp<='0';
                END loop;
                state <=ready;
            ELSIF state = ready and add = '1' THEN
                IF node(key REM Width,key/Height).dp = '0' THEN
                    node(key REM Width,key/Height).value <= value;
                    state <= inserting;
                END IF;
            ELSIF state = ready and replace = '1' THEN
                node(key REM Width,key/Height).value <= value;
                state <= inserting;
            ELSIF state = ready and find = '1' THEN
                IF node(key REM Width,key/Height).dp='1' THEN
                    exists <= '1';
                    GridValue <= node(findValue
Width,findValue/Height).value;
                    REM
```

```

        state <= inserting;
    ELSE exists <= '0';
    END IF;
    state <= ready;
    ELSIF state = ready and find = '0' THEN
        GridValue <= node(key REM Width,key/Height).value;
    END IF;
    END IF;
END process;
GridKey <= node(key REM Width,key/Height).key;
busy <= '1' WHEN state /= ready ELSE '0';

end Behavioral;
```

2) ΒΙΒΛΙΟΘΗΚΗ ΚΛΕΙΣΤΗΣ ΛΙΣΤΑΣ

ΛΟΓΙΚΟ ΕΠΙΠΕΔΟ FIFO:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FIFO_list is
    Port ( clk, add, pop, clear: in STD_LOGIC;
          Node: in INTEGER;
          popValue: out INTEGER;
          NOADD, NOPOP: out STD_LOGIC;
          full, empty, exist : out STD_LOGIC);
end FIFO_list;

architecture Behavioral of FIFO_list is
    signal Wr_e : STD_LOGIC;
    signal A, dataout : INTEGER;
    COMPONENT RAM IS
        GENERIC ( words: INTEGER :=16);
        Port ( clk, wr_enable :in STD_LOGIC;
              address : in INTEGER range 0 to words-1;
              data_in: in INTEGER;
              data_out : out INTEGER);
    end COMPONENT;
    COMPONENT FIFO IS
        Port ( CLK, ADD, POP, CLEAR: in STD_LOGIC;
              data_in : in INTEGER;
              data_out : out INTEGER;
              FULL, EMPTY, WE, noAdd, noPop: buffer STD_LOGIC);
    end COMPONENT;
begin
    FL : FIFO port map (CLK =>clk, ADD=> add, POP=>pop, CLEAR =>clear,
    data_in =>Node, data_out =>dataout,
    FULL =>full, EMPTY =>empty, WE
=> Wr_e, noAdd => NOADD, noPop => NOPOP);
    R: RAM port map (clk =>clk, wr_enable =>Wr_e, address => A, data_in
=>dataout, data_out =>popValue);

end Behavioral;
```

ΕΠΙΠΕΔΟ RAM:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RAM is
    GENERIC ( words: INTEGER :=16);
    Port ( clk, wr_enable :in STD_LOGIC;
          address : in INTEGER range 0 to words-1;
          data_in: in INTEGER;
          data_out : out INTEGER);
```

```

end RAM;

architecture Behavioral of RAM is
    type list is array (0 to words-1) of INTEGER;
    signal memory: list;
begin
    process (clk, wr_enable)
    begin
        IF (wr_enable = '1') THEN
            IF (clk'event AND clk = '1') THEN
                memory(address) <= data_in;
            END IF;
        END IF;
    END process;
    data_out <= memory(address);

end Behavioral;

```

3) ΛΙΣΤΑ ΠΡΟΤΕΡΑΙΟΤΗΤΑΣ

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Priority_queue is
    Port ( clk, reset :in STD_LOGIC;
          add, pop, PrChange : in  STD_LOGIC;
          key, value : in  INTEGER;
          smallKey : out INTEGER;
          busy, full : out STD_LOGIC );
end Priority_queue;

architecture Behavioral of Priority_queue is
    constant rows: integer :=4;                                --αριθμός στηλών
    type struct is record                                       --βασική δομή, κόμβος
        ff : STD_LOGIC;                                         --flip-flop
        key : INTEGER;
        value : INTEGER;
    END record struct;
    type rowTyp is array(0 to rows-1) of struct;
    signal head, tail: rowTyp;

    type state_type is (ready, adding, popping);
    signal state: state_type;
begin
    process(clk) begin
        IF rising_edge(clk) THEN
            IF reset='1' THEN
                FOR i IN 0 to rows-1 loop
                    head(i).ff<='0';
                    tail(i).ff<='0';
                    head(i).key<=0;
                    tail(i).key<=0;
                END loop;
                state<=ready;
            ELSIF state = ready and add = '1' THEN
                IF head(rows-1).ff /= '1' THEN
                    FOR i IN 1 TO rows-1 loop
                        head(i)<= head(i-1);
                    END loop;
                    head(0)<= ('1', key, value);
                    state<= adding;
                END IF;
            ELSIF state = ready and pop = '1' THEN
                IF tail(0).ff /= '0' THEN
                    FOR i IN 0 TO rows-2 loop
                        tail(i) <= tail(i+1);
                    END loop;
                    tail(rows-1).ff <= '0';
                    state <= popping;
                END IF;
            ELSIF state = ready and PrChange = '1' THEN
                head(0).ff <='0';
                IF head(rows-1).ff /= '1' THEN
                    FOR i IN 1 TO rows-1 loop
                        head(i)<= head(i-1);
                    END loop;
                    head(0)<= ('1', key, value);
                END IF;
            END IF;
        END IF;
    end process;
end;
```

```

        state<= adding;
    END IF;
    ELSIF state = adding or state = popping THEN
        FOR i IN 0 TO rows-1 loop
            IF head(i).ff = '1' and (head(i).value<tail(i).value or
tail(i).ff = '0') THEN
                tail(i) <= head(i);
                head(i) <= tail(i);
            END IF;
        END loop;
        state <= ready;
    END IF;
END IF;
END process;
smallKey <= tail(0).key WHEN tail(0).ff = '1' ELSE 0;
full <= head(rows-1).ff;
busy <= '1' WHEN state /= ready ELSE '0';
end Behavioral;

```

4) ΒΙΒΛΙΟΘΗΚΗ ΧΑΡΤΗ ΕΜΠΟΔΙΩΝ

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Walkable is
    Port ( clk, reset :in STD_LOGIC;
          Width : in  INTEGER;
          Heigh : in  INTEGER;
          wallNode : in  STD_LOGIC;
          key : in  INTEGER;
          busy : out STD_LOGIC;
          canWalk : out  STD_LOGIC);
end Walkable;

architecture Behavioral of Walkable is
    Type struct is record          --δομή κόμβου
        dp : STD_LOGIC;           --flip-flop
        key : INTEGER;
        value : STD_LOGIC;
    END record struct;
    Type grid is array (0 TO Heigh, Width DOWNT0 0) OF struct;
    signal node: grid;

    type state_type is (ready, inserting);
    signal state: state_type;

begin
    process(clk) begin
        IF rising_edge(clk) THEN
            IF reset='1' THEN
                FOR i IN 0 to Width*Heigh-1 loop
                    node(i REM Width, i/Heigh).value<='0';
                    node(i REM Width, i/Heigh).dp<='0';
                END loop;
                state <=ready;
            ELSIF state = ready and wallNode = '1' THEN
                node(key REM Width, key/Heigh).value <= '1';
                state <= inserting;
            ELSIF state = ready and wallNode = '0' THEN
                canWalk <= node(key REM Width, key/Heigh).value;
            END IF;
        END IF;
    END process;
    busy <= '1' WHEN state /= ready ELSE '0';

end Behavioral;
```


ΒΙΒΛΙΟΓΡΑΦΙΑ

- [1] **Jones Joseph L** Robot Programming: A Practical Guide to Behaviour-Based Robotics. - USA: McGraw-Hill, Inc., 2004. - ISBN: 0-07-142778-3
- [2] **Nilsson Nils J** Problem Solving Methods in Artificial Intelligence. - USA: McGraw-Hill, Inc., 1971. - ISBN: 07-046573-8.
- [3] **Howie Choset, Kevin M. Lynch, Seth Hutchinson, George A. Kantor, Wolfram Burgard, Lydia E. Kavraki and Sebastian Thrun:** *Principles of Robot Motion* MIT Press, Inc., May 2005. - ISBN: 9780262033275

ΔΙΚΤΥΑΚΟΙ ΤΟΠΟΙ

- [1] “A star Tutorial” <http://www.policyalmanac.org/games/aStarTutorial.htm>. Ιστοσελίδα Polyalmanac. 5/2014
- [2] Ποικίλα θέματα σχετικά με τον προγραμματισμό <http://stackoverflow.com/> Ιστοσελίδα stackoverflow.
- [3] “Indroduction to Fuctions” [h/http://www.functionx.com/cpp/Lesson05.htm](http://www.functionx.com/cpp/Lesson05.htm) Ιστοσελίδα FunctionX Tutorials. 5/8/2014
- [4] “Διάφορες πληροφορίες σχετικές με τον προγραμματισμό” http://www.tutorialspoint.com/cplusplus/cpp_constructor_destructor.htm Ιστοσελίδα tutorials point, simply learning.
- [5] “Priority queue” https://en.wikipedia.org/wiki/Priority_queue Ιστοσελίδα Wikipedia.
- [6] “Path Finding Tutorial” com/Path_Finding_Tutorial#Pseudo-code_A.2A Ιστοσελίδα GAME Gardens. 7/1/2015
- [7] “Introduction to A*” <http://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html> Ιστοσελίδα Massachusetts Institute of Technology
- [9] Pathfinding using A*” <http://web.mit.edu/eranki/www/tutorials/search/> Ιστοσελίδα Massachusetts Institute of Technology. 25/9/2014